| **Hamdard University – Karachi Main Campus** |
| --- |
| **Web Engineering (SEWE-6109)** |
| **16-Week Lecture Notes with Industry-Oriented Hands-on Labs** |
| Rocky Linux 10 and Kubuntu 26.04 LTS editions · Free & open-source tooling only |
| Instructor: Iqbal Uddin Khan · BSSE 5th Semester · 3 Credit Hours (3+0) · 16 Weeks |

# Contents

Contents ..... 2
About This Document ..... 6
Course Alignment (CLO / PLO / Bloom) ..... 6
Assessment Calendar (from the OBE mechanism) ..... 6
Syllabus Modernisation Notes ..... 6
Version Policy: What “Latest” Means Here ..... 7
Conventions ..... 7
Lab 0: Common Lab Environment (Complete Before Week 1) ..... 8
System Requirements ..... 8
Version A – Rocky Linux 10 ..... 8
Version B – Kubuntu 26.04 LTS ..... 9
Lab 0 Verification ..... 10
16-Week Plan at a Glance ..... 11
Week 1: Internet, HTTP, Web Architectures and HTML5 Foundations ..... 12
Learning Outcomes ..... 12
3-Hour Lecture Plan ..... 12
Lecture Notes ..... 12
Hands-on Lab 1: Own Web Server, HTTPS and First HTML5 Page ..... 15
Review Questions ..... 19
Week 2: HTML5 Forms, Semantic Elements and Accessibility ..... 20
Learning Outcomes ..... 20
3-Hour Lecture Plan ..... 20
Lecture Notes ..... 20
Hands-on Lab 2: Accessible Student Registration Form with Automated Linting ..... 22
Review Questions ..... 24
Week 3: CSS3: Selectors, Modern Layout and Responsive Design ..... 25
Learning Outcomes ..... 25
3-Hour Lecture Plan ..... 25
Lecture Notes ..... 25
Hands-on Lab 3: Responsive Event Landing Page (Mobile-First) ..... 26
Review Questions ..... 29
Week 4: JavaScript Fundamentals (ES2025 and Beyond) ..... 30
Learning Outcomes ..... 30
3-Hour Lecture Plan ..... 30
Lecture Notes ..... 30

Hands-on Lab 4: Expense Tracker – Tested ES Module Library with a Browser UI ..... 31
Review Questions ..... 34
Week 5: Advanced JavaScript: OOP, Async, CSV Data and Canvas / 3D ..... 35
Learning Outcomes ..... 35
3-Hour Lecture Plan ..... 35
Lecture Notes ..... 35
Hands-on Lab 5: CSV Student Explorer and Rotating 3D Cube (Vite Build) ..... 37
Review Questions ..... 40
Week 6: Node.js and Modern Angular ..... 41
Learning Outcomes ..... 41
3-Hour Lecture Plan ..... 41
Lecture Notes ..... 41
Hands-on Lab 6: Node REST API and Angular Course Dashboard ..... 42
Review Questions ..... 46
Week 7: Bootstrap 5.3, UI/UX and Component-Based Interfaces ..... 47
Learning Outcomes ..... 47
3-Hour Lecture Plan ..... 47
Lecture Notes ..... 47
Hands-on Lab 7: Responsive Product Catalogue with Carousel, Pagination and Dark Mode ..... 48
Review Questions ..... 51
Week 8: Mid-Term Examination (Weeks 1–7) ..... 52
Learning Outcomes ..... 52
3-Hour Lecture Plan ..... 52
Lecture Notes ..... 52
Hands-on Lab 8: Mid-Term Practical Environment and Automatic Checker ..... 53
Review Questions ..... 55
Week 9: PHP Fundamentals and the Server-Side Stack ..... 56
Learning Outcomes ..... 56
3-Hour Lecture Plan ..... 56
Lecture Notes ..... 56
Hands-on Lab 9: PHP-FPM Stack, Grade Report and Matrix Exercises with PHPStan ..... 57
Review Questions ..... 61
Week 10: SQL, MariaDB, PDO, CAPTCHA and Secure File Upload ..... 62
Learning Outcomes ..... 62
3-Hour Lecture Plan ..... 62
Lecture Notes ..... 62

Hands-on Lab 10: MariaDB Schema, PDO Connection, CAPTCHA Registration and Avatar Upload 64
Review Questions ..... 69
Week 11: Full-Stack PHP: Templates, Authentication, Products, Cart and Error Handling ..... 70
Learning Outcomes ..... 70
3-Hour Lecture Plan ..... 70
Lecture Notes ..... 70
Hands-on Lab 11: Mini-Shop: Login, Product List, Session Cart and Transactional Checkout ..... 71
Review Questions ..... 76
Week 12: Queries, Result Handling, File Handling and Hourly Test ..... 77
Learning Outcomes ..... 77
3-Hour Lecture Plan ..... 77
Lecture Notes ..... 77
Hands-on Lab 12: Product Search, CSV Import/Export and Thumbnail Generation ..... 78
Review Questions ..... 83
Week 13: Email, Cookies, Sessions and MVC Frameworks (CodeIgniter 4 and Laravel) ..... 84
Learning Outcomes ..... 84
3-Hour Lecture Plan ..... 84
Lecture Notes ..... 84
Hands-on Lab 13: Mailpit, Secure Sessions and a CodeIgniter 4 Notes Application ..... 85
Review Questions ..... 91
Week 14: WordPress: Architecture, Installation, Plugins and the Block Editor ..... 92
Learning Outcomes ..... 92
3-Hour Lecture Plan ..... 92
Lecture Notes ..... 92
Hands-on Lab 14: WordPress with WP-CLI, Plugins and a Home Page Built from Blocks ..... 93
Review Questions ..... 97
Week 15: Creating and Configuring a Blog: Themes, Custom Content, Security and Backups ..... 98
Learning Outcomes ..... 98
3-Hour Lecture Plan ..... 98
Lecture Notes ..... 98
Hands-on Lab 15: Configure the Blog, Child Theme, “Projects” Plugin, Hardening and Automatic Backup ..... 99
Review Questions ..... 103
Week 16: Capstone Deployment with Containers, Course Revision and Final-Exam Preparation ..... 105
Learning Outcomes ..... 105
3-Hour Lecture Plan ..... 105
Lecture Notes ..... 105

Hands-on Lab 16: Containerised Capstone: Apache/PHP + MariaDB with Podman Compose and a Quadlet Service ..... 107

Review Questions ..... 111

Appendix A: Rocky 10 vs Kubuntu 26.04 Cheat Sheet ..... 112

SELinux Troubleshooting Routine (Rocky) ..... 112

Appendix B: Lab Marking Rubric (10 points per lab) ..... 112

Appendix C: References and Free / Open-Source Tool Index ..... 113

# About This Document

These notes expand the approved course outline of **Web Engineering (SEWE-6109)** into sixteen weekly packs. Each week is one **3-hour class** (equal to two 1.5-hour syllabus lectures: Week n covers Lectures 2n–1 and 2n of the lecture schedule). Every pack contains learning outcomes, a minute-by-minute lecture plan, lecture notes, one hands-on lab in **two versions** (Version A for Rocky Linux 10 and Version B for Kubuntu 26.04 LTS), a verification checklist and review questions.

## Course Alignment (CLO / PLO / Bloom)

| CLO | Statement | PLO | Bloom | Weeks |
|---|---|---|---|---|
| CLO_1 | Classify web components and differentiate MVC from two-/three-tier architectures | PLO_2 | C2 | 1–3, 13 |
| CLO_2 | Develop web pages using HTML, CSS, JavaScript, PHP and database connection | PLO_5 | C3 | 4–9, 11–12 |
| CLO_3 | Analyze and solve web development problems using WordPress and CodeIgniter | PLO_4 | C4 | 10–16 |

## Assessment Calendar (from the OBE mechanism)

| Item | Week | CLO | Marks | Note |
|---|---|---|---|---|
| Assignment A1 | 1 → due 2 | CLO_1 | 4 | Architecture report |
| Quiz Q1 | 4 | CLO_1 | 4 | Last 15 min of class |
| Assignment A2 | 5 → due 6 | CLO_2 | 0 | Formative (0 marks in OBE sheet) |
| Mid-Term (Obj 10 + Subj 10) | 8 | CLO_1 / CLO_2 | 20 | Practical + MCQs |
| Quiz Q2 | 9 | CLO_2 | 0 | Formative |
| Assignment A3 | 11 → due 12 | CLO_3 | 6 | Mini-shop extension |
| Quiz Q3 | 11 | CLO_3 | 6 | Last 15 min of class |
| Hourly Test (OHT) | 12 | CLO_2 | 10 | 60 min, Weeks 9–11 |
| Final Exam (Subj 10 + Obj 15 + Subj 25) | After 16 | CLO 1 / 2 / 3 | 50 | University schedule |

**Scheduling:** The weekly placement of A2, Q2, Q3 and the OHT is a suggestion because the source outline lists only lecture numbers. Adjust to the academic calendar.

## Syllabus Modernisation Notes

Several syllabus items use names that have been superseded in industry. The labs teach the current practice while still meeting the stated outcome:

| Syllabus item | What is taught | Reason |
|---|---|---|
| “Angular JS” | Modern **Angular** (standalone components, signals) | AngularJS 1.x reached end-of-life in January 2022 |
| Glyph icons | **Bootstrap Icons** (SVG / icon font) | Glyphicons were removed from Bootstrap 4+ |
| Jumbotron | Hero section built from Bootstrap 5 utilities | Jumbotron component removed in Bootstrap 5 |
| reCAPTCHA | Hand-built GD CAPTCHA + survey of reCAPTCHA / Turnstile / ALTCHA | Keeps the lab 100% open source and offline |

| Syllabus item | What is taught | Reason |
|---|---|---|
| jQuery | Covered as legacy with a jQuery → modern JS mapping | Native DOM APIs replace most jQuery usage |
| Laravel + CodeIgniter | CodeIgniter 4 hands-on, Laravel guided demo | CLO_3 names CodeIgniter; Laravel for comparison |
| MySQL | **MariaDB** (drop-in compatible, shipped by both distros) | Default open-source server on Rocky and Ubuntu |

## Version Policy: What "Latest" Means Here

Labs use the newest stable or LTS release available in October 2026 and avoid pinning minor versions in commands where the tool resolves "latest" itself. The two operating systems ship different system packages, which is itself a real-world skill (matching code to the runtime you are given):

| Component | Rocky Linux 10 | Kubuntu 26.04 LTS |
|---|---|---|
| Kernel / desktop | Linux 6.12 LTS-based, GNOME or server | Linux 7.0, KDE Plasma 6.6 (Wayland) |
| Apache httpd | 2.4.x (package `httpd`) | 2.4.66 (package `apache2`) |
| PHP | 8.3 (AppStream) | 8.5 |
| Database | MariaDB 10.11 | MariaDB 11.8 |
| Mandatory access control | SELinux – enforcing | AppArmor |
| Firewall | `firewalld` | `ufw` |
| Package manager | `dnf` | `apt` |
| Web user / group | `apache` | `www-data` |
| Node.js (all labs) | Node 24 LTS via nvm | Node 24 LTS via nvm |

> **Compatibility:** All lab code is written for PHP 8.3 so that it runs on both systems. Kubuntu students may try PHP 8.4/8.5 features (property hooks, `array_find`, the pipe operator) as stretch goals.

## Conventions

- **Version A / Version B** steps are complete and independent: do only your own platform. Source code under "Common Source Files" is shared.
- Commands starting with `sudo` need administrator rights; other commands run as your normal user. Prompts ($) are omitted so commands can be copied.
- All sites live under `/var/www/webeng/weekNN` and are served at `http://webeng.test/weekNN/` (Lab 0).
- Every tool is free software under an open-source licence (Apache, GPL, MIT, BSD, MPL, LGPL or similar).
- Tables use 10 pt text for fit; body text is Aptos 12 pt; code is Aptos Mono.

# Lab 0: Common Lab Environment (Complete Before Week 1)

Lab 0 builds the base system used every week: a patched OS, Apache with a dedicated virtual host `webeng.test`, Git, Node.js LTS, Firefox and a code editor. Allow 45 minutes.

## System Requirements

- 64-bit PC or VM, **4 GB RAM minimum (8 GB recommended)**, 40 GB disk, internet access.
- **Rocky Linux 10 requires an x86-64-v3 CPU** (AVX2). In VirtualBox / KVM (virt-manager) select host-passthrough CPU, otherwise the installer stops.
- Create a normal user with sudo rights during installation (do not work as root).

## Version A – Rocky Linux 10

**Step 1 – Update and install base packages.** EPEL provides extra tooling; CRB enables build dependencies.

```
sudo dnf -y upgrade --refresh
sudo dnf -y install epel-release
sudo crb enable
sudo dnf -y install httpd mod_ssl git curl wget unzip tar jq tmux bind-utils \
  nmap-ncat tcpdump openssl policycoreutils-python-utils flatpak firefox
```

**Step 2 – Start Apache and open the firewall.** firewalld is the default firewall; permanent rules survive reboot.

```
sudo systemctl enable --now httpd
sudo firewall-cmd --permanent --add-service=http --add-service=https
sudo firewall-cmd --reload
sudo firewall-cmd --list-services
```

**Step 3 – Create the web root with correct SELinux labels.** SELinux is enforcing; content must carry the type `httpd_sys_content_t`.

```
sudo mkdir -p /var/www/webeng
sudo chown $USER:apache /var/www/webeng
sudo chmod 2775 /var/www/webeng
sudo semanage fcontext -a -t httpd_sys_content_t "/var/www/webeng(/.*)?"
sudo restorecon -Rv /var/www/webeng
```

**Step 4 – Create the virtual host.** Files in /etc/httpd/conf.d/ are loaded automatically.

```
sudo tee /etc/httpd/conf.d/webeng.conf >/dev/null <<'EOF'
<VirtualHost *:80>
    ServerName webeng.test
    DocumentRoot /var/www/webeng
    <Directory /var/www/webeng>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog logs/webeng_error.log
    CustomLog logs/webeng_access.log combined
</VirtualHost>
EOF
echo "127.0.0.1 webeng.test" | sudo tee -a /etc/hosts
sudo apachectl configtest && sudo systemctl reload httpd
```

**Step 5 – Install Node.js LTS with nvm and the editor.** nvm lets each student install Node without root. VSCodium is the open-source build of VS Code.

```
NVM_TAG=$(curl -s https://api.github.com/repos/nvm-sh/nvm/releases/latest | jq -
r .tag_name)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/${NVM_TAG}/install.sh | bash
source ~/.bashrc
nvm install --lts && node -v && npm -v
flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install -y flathub com.vscodium.codium
```

**Step 6 – Smoke test.** You should see HTTP 200 and the test text.

```
echo "<h1>webeng ready</h1>" > /var/www/webeng/index.html
curl -i http://webeng.test/
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Update and install base packages.** Kubuntu already includes Firefox (snap) and Kate.

```
sudo apt update && sudo apt -y full-upgrade
sudo apt -y install apache2 git curl wget unzip jq tmux bind9-dnsutils \
  netcat-openbsd tcpdump openssl ufw flatpak plasma-discover-backend-flatpak
sudo a2enmod ssl rewrite headers
```

**Step 2 – Start Apache and configure the firewall.** Allow SSH first if you work remotely. ufw is inactive by default.

```
sudo systemctl enable --now apache2
sudo ufw allow OpenSSH
sudo ufw allow "Apache Full"
sudo ufw --force enable
sudo ufw status verbose
```

**Step 3 – Create the web root.** www-data is the Apache user; the setgid bit keeps group ownership.

```
sudo mkdir -p /var/www/webeng
sudo chown $USER:www-data /var/www/webeng
sudo chmod 2775 /var/www/webeng
```

**Step 4 – Create the virtual host.** Sites are enabled with a2ensite; the default site is disabled.

```
sudo tee /etc/apache2/sites-available/webeng.conf >/dev/null <<'EOF'
<VirtualHost *:80>
    ServerName webeng.test
    DocumentRoot /var/www/webeng
    <Directory /var/www/webeng>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/webeng_error.log
    CustomLog ${APACHE_LOG_DIR}/webeng_access.log combined
</VirtualHost>
EOF
echo "127.0.0.1 webeng.test" | sudo tee -a /etc/hosts
sudo a2dissite 000-default && sudo a2ensite webeng
sudo apache2ctl configtest && sudo systemctl reload apache2
```

**Step 5 – Install Node.js LTS with nvm and the editor.** Identical to Rocky because nvm is distribution-independent.

```
NVM_TAG=$(curl -s https://api.github.com/repos/nvm-sh/nvm/releases/latest | jq -
r .tag_name)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/${NVM_TAG}/install.sh | bash
source ~/.bashrc
nvm install --lts && node -v && npm -v
flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install -y flathub com.vscodium.codium
```

**Step 6 – Smoke test.** You should see HTTP 200 and the test text.

```
echo "<h1>webeng ready</h1>" > /var/www/webeng/index.html
curl -i http://webeng.test/
```

## Lab 0 Verification

| Check | Expected Result |
|---|---|
| `curl -sI http://webeng.test/ \| head -1` | `HTTP/1.1 200 OK` |
| `node -v` | Node 24.x (or the current LTS) |
| `git --version` | Git 2.4x or newer |
| Rocky: `getenforce` / Kubuntu: `sudo ufw status` | `Enforcing` / `Status: active` |

# 16-Week Plan at a Glance

| Wk | Topic | Lec | CLO | Hands-on Lab | Assess. |
|---|---|---|---|---|---|
| 1 | Internet, HTTP, architectures, HTML5 basics | 1–2 | 1 | Own web server, HTTPS, first HTML5 page | A1 issued |
| 2 | HTML5 forms, semantics, accessibility | 3–4 | 1 | Accessible registration form + linting | A1 due |
| 3 | CSS3 layout, responsive design | 5–6 | 1 | Responsive event landing page | — |
| 4 | JavaScript fundamentals (ES2025) | 7–8 | 2 | Expense tracker + unit tests | Q1 |
| 5 | Advanced JS: OOP, async, CSV, canvas | 9–10 | 2 | CSV explorer + Three.js cube (Vite) | A2 issued |
| 6 | Node.js and Angular | 11–12 | 2 | Node REST API + Angular dashboard | A2 due |
| 7 | Bootstrap 5.3 and UI/UX | 13–14 | 2 | Product catalogue with carousel | — |
| 8 | Mid-Term examination | 15–16 | 1, 2 | Practical exam environment + auto-checker | Mid-Term |
| 9 | PHP fundamentals | 17–18 | 2 | PHP-FPM stack, grades, arrays, PHPStan | Q2 |
| 10 | SQL, MariaDB, PDO, CAPTCHA, uploads | 19–20 | 3 | Secure registration with CAPTCHA + avatar | — |
| 11 | PHP + MariaDB: auth, cart, errors | 21–22 | 3 | Mini-shop with login and cart | A3, Q3 |
| 12 | Queries, file handling, CSV | 23–24 | 3 | Search, CSV import/export, thumbnails | OHT |
| 13 | Email, cookies, sessions, MVC frameworks | 25–26 | 1, 3 | Mailpit + CodeIgniter 4 notes app | — |
| 14 | WordPress install, plugins, block editor | 27–28 | 3 | WP-CLI install, plugins, block page | — |
| 15 | Blog creation, themes, security | 29–30 | 3 | Child theme, custom post type, backups | — |
| 16 | Containers, deployment, revision | 31–32 | 1–3 | Podman compose capstone + mock exam | Final prep |

# Week 1: Internet, HTTP, Web Architectures and HTML5 Foundations

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 1–2 | **CLO** | CLO_1 |
| **PLO** | PLO_2 | **Bloom's Level** | C2 (Understanding) |
| **Reading** | Ch. 1 Internet Technologies; Ch. 2 HTML Basics | **Assessment** | A1 issued (due Week 2) |

## Learning Outcomes

- Explain how a page travels from server to browser (DNS, TCP/QUIC, TLS, HTTP).
- Decompose a URI into scheme, host, port, path, query and fragment, and distinguish URI, URL and URN.
- **Classify web components** and **differentiate MVC from two-tier and three-tier architectures** (CLO_1).
- Write valid HTML5 with headings, text formatting, links, lists, tables, div/span, comments.
- Deploy a page on Apache, observe HTTP with curl / tcpdump / DevTools and enable HTTPS.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | Internet, DNS, URL, HTTP/HTTPS, TCP/IP |
| 0:45 – 1:20 | 35 | Concept Block 2 | Web architectures (1/2/3-tier, MVC) and HTML5 elements |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 1 – Web server, HTTPS and first HTML5 page |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 1.1 What Is Web Engineering?

Web engineering applies systematic, measurable engineering practice to the design, development, deployment and maintenance of web applications. Unlike a desktop program, a web application is delivered continuously, is exposed to the public internet, runs on thousands of device and browser combinations and is judged by performance, accessibility and security as much as by features.

| Aspect | Traditional software | Web application |
|---|---|---|
| Release cycle | Versioned installers | Continuous deployment, often daily |
| Client | Known OS / hardware | Any browser, phone or desktop |
| Attack surface | Local machine | Public internet (OWASP Top 10) |
| Quality drivers | Correctness | Correctness + speed + accessibility + SEO |
| Content | Mostly code | Code + content + media |

## 1.2 Internet vs World Wide Web

The **Internet** is the global network of networks that moves packets using TCP/IP. The **World Wide Web** is one service on top of it: hyperlinked resources identified by URLs and transferred with HTTP. E-mail (SMTP) and file transfer (FTP/SFTP) are other services on the same Internet.

| Year | Milestone |
|---|---|
| 1969 / 1983 | ARPANET starts / TCP/IP becomes the standard protocol suite |
| 1989–1991 | Tim Berners-Lee proposes the Web; first server, browser and page |
| 1995 | JavaScript and PHP appear |
| 2015 | HTTP/2 (multiplexing over one TCP connection), now RFC 9113 |
| 2018 | TLS 1.3 (RFC 8446): faster, safer handshake |
| 2022 | HTTP/3 over QUIC (RFC 9114); HTTP semantics unified in RFC 9110 |

## 1.3 URI, URL and URN

A **URI** identifies a resource. A **URL** says where it is and how to fetch it; a **URN** names it independent of location (for example `urn:isbn:9780133775983`). Every URL is a URI, not every URI is a URL.

```
https://student@www.example.com:8443/docs/index.html?lang=en&page=2#intro
```

| Part | Value | Meaning |
|---|---|---|
| Scheme | `https` | Protocol to use |
| Userinfo | `student` | Optional credentials (avoid) |
| Host | `www.example.com` | Resolved by DNS to an IP address |
| Port | `8443` | Default 80 (HTTP) / 443 (HTTPS) |
| Path | `/docs/index.html` | Resource on the server |
| Query | `?lang=en&page=2` | Parameters sent to the server |
| Fragment | `#intro` | Position inside the page; never sent to the server |

## 1.4 What Happens When You Open a Page

1. The browser parses the URL and checks its caches.
2. **DNS** resolves the host name: stub resolver → recursive resolver → root → TLD → authoritative server.
3. A **TCP** connection is opened (three-way handshake), or **QUIC** for HTTP/3.
4. A **TLS** handshake authenticates the server certificate and negotiates keys.
5. The browser sends an **HTTP request**; the server returns static content or runs application code and a database query.
6. The browser builds the **DOM** from HTML, fetches CSS / JS / images, builds the CSSOM, lays out, paints and runs JavaScript.

## 1.5 HTTP in Detail

**Request (top) and response (bottom)**

```
GET /week01/index.html HTTP/1.1
Host: webeng.test
Accept: text/html
User-Agent: curl/8.x

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

```
Content-Length: 1234
Cache-Control: max-age=3600
```

| Method | Purpose | Safe | Idempotent |
|---|---|---|---|
| GET | Read a resource | Yes | Yes |
| HEAD | Headers only | Yes | Yes |
| POST | Create / submit data | No | No |
| PUT | Replace a resource | No | Yes |
| PATCH | Partial update | No | No |
| DELETE | Remove a resource | No | Yes |
| OPTIONS | Capabilities / CORS pre-flight | Yes | Yes |

| Class | Common codes |
|---|---|
| 1xx Informational | 101 Switching Protocols |
| 2xx Success | 200 OK, 201 Created, 204 No Content |
| 3xx Redirection | 301 Moved Permanently, 302 Found, 304 Not Modified |
| 4xx Client error | 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed |
| 5xx Server error | 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable |

| Protocol | Layer | Role |
|---|---|---|
| HTTP / HTTPS | Application | Web transfer; HTTPS = HTTP inside TLS |
| FTP / SFTP | Application | File transfer; FTP is unencrypted, prefer SFTP |
| TCP / UDP / QUIC | Transport | Reliable stream / datagrams / encrypted multiplexed transport on UDP |
| IP (v4, v6) | Internet | Addressing and routing |
| Ethernet, Wi-Fi | Link | Local delivery |

## 1.6 Web Architectures (CLO_1)

| Style | Structure | Example | Strength / weakness |
|---|---|---|---|
| 1-tier | UI, logic, data in one program | Local HTML file with JS and localStorage | Simple / no sharing, no security |
| 2-tier | Client ↔ database server | Old client-server app with SQL in the page | Fast to build / DB exposed, logic duplicated |
| 3-tier | Presentation ↔ application ↔ data tiers | Browser → Apache + PHP → MariaDB | Scalable, secure, replaceable tiers / more moving parts |
| MVC | Model (data/rules), View (output), Controller (input handling) | CodeIgniter, Laravel, Rails | Clean code organisation / learning curve |

> **Key distinction:** N-tier describes how a system is **deployed and layered**. MVC describes how code **inside** the application tier is **organised**. A modern site is usually 3-tier at runtime and MVC in its code.

**Web components** you must be able to classify: client (browser), web server (Apache, Nginx, Caddy), application runtime (PHP-FPM, Node.js), database (MariaDB), protocols (HTTP, TLS, DNS) and content languages (HTML, CSS, JavaScript).

### 1.7 HTML5 Essentials

**Minimal valid HTML5 document**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title</title>
</head>
<body>
  <h1>Heading</h1>
  <p>Paragraph with <strong>strong</strong>, <em>emphasis</em> and a <a
href="about.html">link</a>.</p>
</body>
</html>
```

| Group | Elements |
|---|---|
| Headings / text | `h1`–`h6`, `p`, `strong`, `em`, `mark`, `small`, `sub`, `sup`, `br`, `hr`, `pre`, `code`, `blockquote` |
| Links / media | `a` (`href`, `target`, `rel`), `img` (`src`, `alt`, `width`, `height`) |
| Lists | `ul`, `ol`, `li`, `dl`, `dt`, `dd` |
| Tables | `table`, `caption`, `thead`, `tbody`, `tfoot`, `tr`, `th scope`, `td`, `colspan`, `rowspan` |
| Generic | `div` (block), `span` (inline), global attributes `id`, `class`, `lang`, `title`, `hidden`, `data-*` |
| Comments | `<!-- comment -->` (visible in page source, never put secrets in them) |

External links that open a new tab should include `rel="noopener noreferrer"`. Always provide meaningful `alt` text. Tables are for tabular data only, never for page layout.

## Hands-on Lab 1: Own Web Server, HTTPS and First HTML5 Page

Publish an HTML5 student profile on your own Apache virtual host, capture and explain real HTTP traffic, resolve DNS names and upgrade the site to HTTPS with a self-signed certificate.

### Free & Open-Source Tools

- Apache HTTP Server 2.4 (Apache-2.0)
- curl (MIT-style), OpenSSL (Apache-2.0), tcpdump (BSD), dig (MPL-2.0)
- Firefox DevTools (MPL-2.0), Kate / VSCodium (LGPL / MIT)

### Deliverables

- `/var/www/webeng/week01/index.html` served over HTTP and HTTPS
- Text file `http-notes.txt` with 3 captured requests (200, 301/404, 304) and a one-line explanation of each
- Screenshot of DevTools Network tab showing HTTP version and timing
- Starts Assignment A1 (see below)

## Common Source Files (identical on both operating systems)

Create `/var/www/webeng/week01/index.html` with the content below. Add a second page `about.html` that links back so you can test relative links and a 404.

**index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Student Profile - Week 1</title>
</head>
<body>
  <!-- Header block -->
  <div id="header">
    <h1>Ayesha Khan</h1>
    <p><strong>BSSE, 5th Semester</strong> · Hamdard University, Karachi</p>
  </div>
  <hr>
  <h2>About me</h2>
  <p>I am learning <em>Web Engineering</em>. See <a href="about.html">more</a> or visit
     <a href="https://www.w3.org/" target="_blank" rel="noopener noreferrer">W3C</a>.</p>
  <h2>Skills</h2>
  <ul><li>HTML5</li><li>Linux</li><li>Git</li></ul>
  <h2>Courses</h2>
  <table border="1">
    <caption>Fall semester</caption>
    <thead><tr><th scope="col">Code</th><th scope="col">Title</th><th
scope="col">Credits</th></tr></thead>
    <tbody>
      <tr><td>SEWE-6109</td><td>Web Engineering</td><td>3</td></tr>
      <tr><td>SEDB-5101</td><td>Databases</td><td>3</td></tr>
    </tbody>
  </table>
  <p>Updated: <span style="color:#0a7">2026</span></p>
</body>
</html>
```

## Version A – Rocky Linux 10

**Step 1 – Create and label the lab folder.** Files you create inherit the label of the folder; files you move in with mv keep their old label, so run restorecon.

```
mkdir -p /var/www/webeng/week01 && cd /var/www/webeng/week01
kate index.html about.html &      # or: flatpak run com.vscodium.codium .
ls -Zd /var/www/webeng/week01     # expect httpd_sys_content_t
sudo restorecon -Rv /var/www/webeng
```

**Step 2 – Request the page and read the headers.** Compare a normal request, a header-only request and a missing page.

```
curl -i http://webeng.test/week01/
curl -sI http://webeng.test/week01/ | head -n 5
curl -sI http://webeng.test/week01/missing.html | head -n 1
```

**Step 3 – Capture the traffic.** Terminal 1 captures on the loopback interface; terminal 2 sends the request. Plain HTTP is readable, which is why HTTPS exists.

```
sudo tcpdump -i lo -A -nn port 80 -c 20        # terminal 1
curl -s http://webeng.test/week01/ > /dev/null   # terminal 2
```

**Step 4 – Resolve names and check listening ports.** Document the DNS chain and which process owns port 80.

```
dig +short hamdard.edu.pk A
dig +trace hamdard.edu.pk | tail -n 12
sudo ss -tlnp | grep -E ':80|:443'
sudo firewall-cmd --list-all
```

**Step 5 – Create a certificate.** Keys go to /etc/pki/tls/private and must keep their SELinux label.

```
sudo openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -keyout /etc/pki/tls/private/webeng.key -out /etc/pki/tls/certs/webeng.crt \
  -subj "/C=PK/ST=Sindh/L=Karachi/O=Hamdard/CN=webeng.test" \
  -addext "subjectAltName=DNS:webeng.test"
sudo restorecon -v /etc/pki/tls/private/webeng.key /etc/pki/tls/certs/webeng.crt
```

**Step 6 – Add an HTTPS virtual host with HTTP/2.** Append to the existing configuration file.

```
sudo tee -a /etc/httpd/conf.d/webeng.conf >/dev/null <<'EOF'
<VirtualHost *:443>
    ServerName webeng.test
    DocumentRoot /var/www/webeng
    Protocols h2 http/1.1
    SSLEngine on
    SSLCertificateFile /etc/pki/tls/certs/webeng.crt
    SSLCertificateKeyFile /etc/pki/tls/private/webeng.key
    <Directory /var/www/webeng>
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
EOF
sudo apachectl configtest && sudo systemctl reload httpd
```

**Step 7 – Verify TLS and HTTP/2.** `-k` accepts the self-signed certificate for testing only.

```
curl -skI --http2 https://webeng.test/week01/ | head -n 3
openssl s_client -connect webeng.test:443 -servername webeng.test </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -dates
sudo ausearch -m avc -ts recent        # must show no denials
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Create the lab folder.** The web root is group-writable by your user and www-data (Lab 0).

```
mkdir -p /var/www/webeng/week01 && cd /var/www/webeng/week01
kate index.html about.html &      # or: flatpak run com.vscodium.codium .
ls -ld /var/www/webeng/week01
```

**Step 2 – Request the page and read the headers.** Compare a normal request, a header-only request and a missing page.

```
curl -i http://webeng.test/week01/
curl -sI http://webeng.test/week01/ | head -n 5
curl -sI http://webeng.test/week01/missing.html | head -n 1
```

**Step 3 – Capture the traffic.** Terminal 1 captures on the loopback interface; terminal 2 sends the request.

```
sudo tcpdump -i lo -A -nn port 80 -c 20        # terminal 1
curl -s http://webeng.test/week01/ > /dev/null   # terminal 2
```

**Step 4 – Resolve names and check listening ports.** Document the DNS chain and which process owns port 80.

```
dig +short hamdard.edu.pk A
dig +trace hamdard.edu.pk | tail -n 12
sudo ss -tlnp | grep -E ':80|:443'
sudo ufw status verbose
```

**Step 5 – Create a certificate.** Keys live in /etc/ssl/private (group ssl-cert).

```
sudo openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -keyout /etc/ssl/private/webeng.key -out /etc/ssl/certs/webeng.crt \
  -subj "/C=PK/ST=Sindh/L=Karachi/O=Hamdard/CN=webeng.test" \
  -addext "subjectAltName=DNS:webeng.test"
```

**Step 6 – Add an HTTPS virtual host with HTTP/2.** Enable the http2 module and the new site.

```
sudo tee /etc/apache2/sites-available/webeng-ssl.conf >/dev/null <<'EOF'
<VirtualHost *:443>
    ServerName webeng.test
    DocumentRoot /var/www/webeng
    Protocols h2 http/1.1
    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/webeng.crt
    SSLCertificateKeyFile /etc/ssl/private/webeng.key
    <Directory /var/www/webeng>
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
EOF
sudo a2enmod http2 && sudo a2ensite webeng-ssl
sudo apache2ctl configtest && sudo systemctl reload apache2
```

**Step 7 – Verify TLS and HTTP/2.** `-k` accepts the self-signed certificate for testing only.

```
curl -skI --http2 https://webeng.test/week01/ | head -n 3
openssl s_client -connect webeng.test:443 -servername webeng.test </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -dates
sudo tail -n 5 /var/log/apache2/error.log
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `curl -sI http://webeng.test/week01/ \| head -1` | `HTTP/1.1 200 OK` |
| 2 | `curl -sI .../missing.html \| head -1` | `HTTP/1.1 404 Not Found` |
| 3 | `curl -skI --http2 https://webeng.test/week01/ \| head -1` | `HTTP/2 200` |
| 4 | tcpdump output | Request line and headers visible in clear text on port 80 |
| 5 | Firefox DevTools → Network | Protocol column shows `h2` for the HTTPS request |
| 6 | Rocky: `sudo ausearch -m avc -ts recent` | No SELinux denials (Version B: no errors in error.log) |

### Stretch Goals (Industry Practice)

- Add `Header always set Strict-Transport-Security "max-age=300"` and `X-Content-Type-Options: nosniff`; confirm with `curl -sI`.
- Redirect HTTP to HTTPS with a `Redirect permanent` rule and observe the 301.
- Replace the self-signed certificate with a locally trusted one using `mkcert` (BSD-3).

### Assignment A1 (CLO_1, 4 marks) – issued this week, due Week 2

Write a 2-page report on the architecture of an **online university portal**. Draw the 1-tier, 2-tier and 3-tier versions and then show how MVC organises the application tier. Support the report with the HTTP evidence captured in this lab and justify one trade-off (security, scalability or cost).

## Review Questions

1. Differentiate Internet and WWW with one example service of each.
2. Break `https://api.shop.pk:8443/v1/items?id=5#top` into its URI components.
3. Why is a URL fragment never sent to the server?
4. Explain, with an example, why a site can be three-tier at runtime and still use MVC in code.
5. What do status codes 301, 304, 403 and 502 tell you about where a problem lies?

# Week 2: HTML5 Forms, Semantic Elements and Accessibility

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 3–4 | **CLO** | CLO_1 |
| **PLO** | PLO_2 | **Bloom's Level** | C2 (Understanding) |
| **Reading** | Ch. 2 HTML Basics; Ch. 3 Forms and Controls | **Assessment** | A1 due |

## Learning Outcomes

- Structure a page with semantic elements (`header`, `nav`, `main`, `article`, `section`, `aside`, `footer`).
- Build forms using the modern input types and attributes, and use native constraint validation.
- Apply WCAG 2.2 AA basics: labels, landmarks, contrast, keyboard access, alt text.
- Lint and format HTML in an automated toolchain (Node.js, html-validate, Prettier).

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | A1 submission and discussion of architecture reports |
| 0:10 – 0:45 | 35 | Concept Block 1 | Semantic elements, metadata, media and accessibility |
| 0:45 – 1:20 | 35 | Concept Block 2 | Forms: elements, input types, attributes, validation, GET vs POST |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 2 – Accessible registration form with linting |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 2.1 Semantic HTML

Semantic elements describe **meaning**, not appearance. They give screen readers landmarks, improve SEO and make the code self-documenting. Use `div` and `span` only when no semantic element fits.

| Element | Meaning |
|---|---|
| `header` / `footer` | Introductory / closing content of a page or section |
| `nav` | Major navigation links |
| `main` | Dominant content, exactly one per page |
| `article` / `section` | Self-contained item / thematic grouping with a heading |
| `aside` | Tangential content (sidebar) |
| `figure` / `figcaption` | Illustration with caption |
| `details` / `summary` | Native disclosure widget (no JavaScript) |
| `dialog` | Modal or non-modal dialog (`showModal()`) |
| `search` | Landmark that wraps a search form |
| `time`, `mark`, `address` | Machine-readable date, highlight, contact info |

## 2.2 Forms

A form collects user input and sends it as name=value pairs. Key attributes: `action` (URL), `method` (`get` puts data in the query string and is for searches; `post` puts it in the body and is for changes), `enctype="multipart/form-data"` for file uploads, `autocomplete`, `novalidate`.

| Input type | Typical attributes | Benefit |
|---|---|---|
| `text`, `search` | `required`, `minlength`, `maxlength`, `pattern` | Basic text |
| `email`, `url`, `tel` | `multiple`, `inputmode`, `autocomplete` | Mobile keyboards + validation |
| `number`, `range` | `min`, `max`, `step` | Numeric constraints |
| `date`, `time`, `datetime-local`, `month` | `min`, `max` | Native pickers |
| `checkbox`, `radio` | `name`, `value`, `checked` | Choices (radios share a name) |
| `file` | `accept`, `multiple`, `capture` | Uploads |
| `color`, `hidden`, `password` | `autocomplete="new-password"` | Specialised |

Other form elements: `label` (always bind with `for`/`id`), `fieldset` + `legend` (group related controls), `select` / `option`, `datalist` (suggestions), `textarea`, `output`, `button type="submit|reset|button"`, `progress`, `meter`.

## 2.3 Constraint Validation and the Client–Server Rule

Attributes such as `required` and `pattern` give instant feedback and can be styled with `:user-invalid`. The JavaScript Constraint Validation API (`input.validity`, `setCustomValidity()`, `form.reportValidity()`) allows custom messages.

> **Security rule:** Client-side validation is a convenience, never security. Anyone can bypass it with curl. The server must validate again (Weeks 9–11).

## 2.4 Accessibility (WCAG 2.2 AA Essentials)

- **Perceivable:** `alt` text, captions, text contrast at least 4.5:1, do not rely on colour alone.
- **Operable:** everything usable by keyboard (Tab, Enter, Space), visible focus, “skip to content” link, target size at least 24×24 CSS px.
- **Understandable:** `lang` attribute, labels and instructions, helpful error messages, `autocomplete` tokens.
- **Robust:** valid HTML first; use ARIA only when native elements cannot do the job (“no ARIA is better than bad ARIA”).

## 2.5 Metadata and Responsive Media

```
<meta name="description" content="Short page summary for search results">
<meta property="og:title" content="Registration">
<link rel="icon" href="favicon.svg" type="image/svg+xml">
<picture>
  <source srcset="hero.avif" type="image/avif">
  <img src="hero.jpg" alt="Students in a lab" width="800" height="450" loading="lazy">
</picture>
```

# Hands-on Lab 2: Accessible Student Registration Form with Automated Linting

Build a semantic, accessible, natively validated registration form using Pakistani CNIC and mobile patterns, then lint and format it using a Node.js toolchain.

## Free & Open-Source Tools

- Node.js 24 LTS + npm (MIT)
- html-validate (MIT), Prettier (MIT)
- Firefox Accessibility Inspector (MPL-2.0)

## Deliverables

- `register.html` and `thanks.html` under `/var/www/webeng/week02/`
- `npx html-validate` reports 0 errors
- Screenshot of the Firefox Accessibility tree showing landmarks
- Form submits with GET and the query string is shown on `thanks.html`

## Common Source Files (identical on both operating systems)

**register.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Student Registration</title>
</head>
<body>
  <a href="#main">Skip to content</a>
  <header><h1>Hamdard Student Registration</h1>
    <nav aria-label="Primary"><a href="index.html">Home</a> <a
href="register.html">Register</a></nav>
  </header>
  <main id="main">
    <form action="thanks.html" method="get">
      <fieldset>
        <legend>Personal details</legend>
        <p><label for="name">Full name</label>
           <input id="name" name="name" required minlength="3" autocomplete="name"></p>
        <p><label for="email">Email</label>
           <input id="email" name="email" type="email" required autocomplete="email"></p>
        <p><label for="cnic">CNIC (xxxxx-xxxxxxx-x)</label>
           <input id="cnic" name="cnic" required inputmode="numeric" pattern="[0-9]{5}-[0-
9]{7}-[0-9]"></p>
        <p><label for="mobile">Mobile (03xx-xxxxxxx)</label>
           <input id="mobile" name="mobile" type="tel" required pattern="03[0-9]{2}-[0-
9]{7}" autocomplete="tel"></p>
        <p><label for="dob">Date of birth</label>
           <input id="dob" name="dob" type="date" required max="2010-12-31"></p>
      </fieldset>
      <fieldset>
        <legend>Programme</legend>
        <p><label for="dept">Department</label>
           <input id="dept" name="dept" list="depts" required>
           <datalist id="depts"><option value="Software Engineering"><option
value="Computer Science"><option value="Electrical Engineering"></datalist></p>
        <p>Semester:
           <label><input type="radio" name="sem" value="5" checked> 5th</label>
           <label><input type="radio" name="sem" value="6"> 6th</label></p>
```

```
        <p><label for="gpa">Expected GPA</label>
           <input id="gpa" name="gpa" type="range" min="2" max="4" step="0.1"
value="3"></p>
        <p><label><input type="checkbox" name="terms" required> I accept the code of
conduct</label></p>
      </fieldset>
      <button type="submit">Register</button> <button type="reset">Clear</button>
    </form>
  </main>
  <footer><small>&copy; 2026 Hamdard University</small></footer>
</body>
</html>
```

**thanks.html**

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8"><title>Thanks</title></head>
<body><main><h1>Received</h1><p id="out"></p></main>
<script>document.getElementById('out').textContent =
decodeURIComponent(location.search);</script>
</body></html>
```

## Version A – Rocky Linux 10

**Step 1 – Create the folder and the files.** Save the two files above, then refresh SELinux labels.

```
mkdir -p /var/www/webeng/week02 && cd /var/www/webeng/week02
kate register.html thanks.html &
sudo restorecon -Rv /var/www/webeng/week02
```

**Step 2 – Initialise a Node project with tooling.** Dev dependencies stay local to the project.

```
npm init -y
npm install --save-dev html-validate prettier
npx prettier --write "*.html"
```

**Step 3 – Create the validator configuration.** Extends the recommended rule set and adds accessibility rules.

```
cat > .htmlvalidate.json <<'EOF'
{ "extends": ["html-validate:recommended", "html-validate:a11y"] }
EOF
npx html-validate register.html thanks.html
```

**Step 4 – Test in the browser.** Use Tab only, no mouse, and open the accessibility inspector.

```
firefox http://webeng.test/week02/register.html &
# Firefox: F12 → Accessibility tab → Check for issues
```

**Step 5 – Verify the server side.** Confirm GET puts the data in the query string.

```
curl -s "http://webeng.test/week02/thanks.html?name=Ali&cnic=42101-1234567-1" | head -n 3
curl -sI http://webeng.test/week02/register.html | head -n 1
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Create the folder and the files.** Save the two files above.

```
mkdir -p /var/www/webeng/week02 && cd /var/www/webeng/week02
kate register.html thanks.html &
```

**Step 2 – Initialise a Node project with tooling.** Dev dependencies stay local to the project.

```
npm init -y
npm install --save-dev html-validate prettier
npx prettier --write "*.html"
```

**Step 3 – Create the validator configuration.** Extends the recommended rule set and adds accessibility rules.

```
cat > .htmlvalidate.json <<'EOF'
{ "extends": ["html-validate:recommended", "html-validate:a11y"] }
EOF
npx html-validate register.html thanks.html
```

**Step 4 – Test in the browser.** Use Tab only, no mouse, and open the accessibility inspector.

```
firefox http://webeng.test/week02/register.html &
# Firefox: F12 → Accessibility tab → Check for issues
```

**Step 5 – Verify the server side.** Confirm GET puts the data in the query string; check Apache access log.

```
curl -s "http://webeng.test/week02/thanks.html?name=Ali&cnic=42101-1234567-1" | head -n 3
sudo tail -n 3 /var/log/apache2/webeng_access.log
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `npx html-validate register.html thanks.html` | Exit code 0, no errors |
| 2 | Submit with an invalid CNIC | Browser blocks submit and shows a validation bubble |
| 3 | Tab through the form | Logical order, visible focus ring, skip link works |
| 4 | Submit valid data | `thanks.html?name=...&cnic=...` shows decoded values |
| 5 | Accessibility inspector | Landmarks banner / navigation / main / contentinfo present; no unlabeled inputs |

## Stretch Goals (Industry Practice)

- Add a `<dialog>` that confirms the entry before submission.
- Style invalid fields with `input:user-invalid { outline: 2px solid #b00020 }`.
- Add a Prettier + html-validate `npm run check` script and a Git pre-commit hook.

## Review Questions

1. Why is `<main>` preferable to `<div id="main">`?
2. When should a form use GET and when POST? Give one example of each.
3. Write a pattern for a Pakistani mobile number such as 0312-1234567 and explain each part.
4. Why must the server validate data that the browser already validated?
5. List four WCAG 2.2 AA checks you performed in the lab.

# Week 3: CSS3: Selectors, Modern Layout and Responsive Design

| Syllabus Lectures | Lec 5–6 | CLO | CLO_1 |
|---|---|---|---|
| PLO | PLO_2 | Bloom's Level | C2 (Understanding) |
| Reading | Ch. 3–4 CSS | Assessment | — |

## Learning Outcomes

- Insert CSS (inline, internal, external) and predict the cascade using specificity and `@layer`.
- Use the box model, custom properties, `clamp()` and modern colour functions.
- Build layouts with Flexbox and Grid; adapt them with media and container queries.
- Create animations, transitions, pseudo-classes and a CSS-only loader respecting `prefers-reduced-motion`.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | Syntax, selectors, specificity, cascade layers, box model, units |
| 0:45 – 1:20 | 35 | Concept Block 2 | Flexbox, Grid, queries, animation, pseudo-classes |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 3 – Responsive event landing page |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 3.1 Ways to Insert CSS

| Method | Syntax | Use |
|---|---|---|
| External | `<link rel="stylesheet" href="style.css">` | Default for real projects (cacheable) |
| Internal | `<style>…</style>` in head | Single page demos, critical CSS |
| Inline | `style="color:red"` | Avoid; highest specificity, hard to maintain |

### 3.2 Selectors, Specificity and the Cascade

A rule has a selector and declarations: `selector { property: value; }`. When rules conflict the browser decides by (1) origin and `!important`, (2) **cascade layer** order, (3) **specificity** (id > class/attribute/pseudo-class > element), (4) source order.

```
@layer reset, base, components;       /* later layers win */
@layer base { a { color: var(--link); } }
.card { padding: 1rem; &:hover { box-shadow: 0 2px 8px #0003; } }   /* native nesting */
li:first-child { font-weight: 700; }
.list:has(> li.active) { border-color: teal; }       /* parent selector */
:where(h1, h2, h3) { margin-block: 0.5em; }          /* zero specificity */
```

## 3.3 Box Model, Units and Custom Properties

- `box-sizing: border-box` makes `width` include padding and border; set it globally.
- Units: `rem` (root font size), `em`, `%`, `vw/vh`, `dvh` (mobile-safe viewport), `ch`; fluid type with `clamp(1rem, 2vw + .5rem, 2rem)`.
- Colour: `rgb()`, `hsl()`, `oklch()` for perceptual uniformity, `color-mix(in oklch, red, blue)`.
- Custom properties: `:root { --brand: #0f766e; }` then `color: var(--brand);` they cascade and can change at runtime (dark mode).

## 3.4 Flexbox and Grid

| Feature | Flexbox | Grid |
|---|---|---|
| Dimension | One-dimensional (row **or** column) | Two-dimensional (rows **and** columns) |
| Best for | Navbars, button rows, centring | Page layouts, card galleries |
| Key properties | `display:flex; gap; justify-content; align-items; flex-wrap` | `display:grid; grid-template-columns; gap; grid-area; repeat(auto-fit, minmax())` |

```
.cards { display: grid; gap: 1rem;
         grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr)); }
.nav   { display: flex; gap: 1rem; justify-content: space-between; align-items: center; }
```

The old “clearfix” hack (`.clearfix::after { content:""; display:table; clear:both }`) existed because floated children collapse their parent. Today `display: flow-root`, Flexbox or Grid replace it.

## 3.5 Responsive Design: Media and Container Queries

```
@media (min-width: 48rem) { .layout { grid-template-columns: 1fr 3fr; } }   /* mobile-
first */
@media (prefers-color-scheme: dark) { :root { --bg: #0b1220; --fg: #e5e7eb; } }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; } }
.card-wrap { container-type: inline-size; }
@container (min-width: 28rem) { .card { display: flex; } }               /* component-
level */
```

## 3.6 Transitions, Animations and Loaders

```
.btn { transition: transform .2s ease, background-color .2s; }
.btn:hover { transform: translateY(-2px); }
@keyframes spin { to { transform: rotate(360deg); } }
.loader { width: 2rem; aspect-ratio: 1; border: .3rem solid #ddd; border-top-color: var(--
brand);
          border-radius: 50%; animation: spin 1s linear infinite; }
```

Animate only `transform` and `opacity` for smooth 60 fps rendering; animating `width` or `top` triggers layout. Text effects such as `text-shadow: 1px 1px 2px #0005`; and borders such as `border: 2px solid` complete the syllabus list.

# Hands-on Lab 3: Responsive Event Landing Page (Mobile-First)

Build a “Karachi Dev Meetup” landing page using Grid, Flexbox, custom properties, dark mode, a container-query card and a CSS-only loader, linted with Stylelint.

## Free & Open-Source Tools

- Stylelint + stylelint-config-standard (MIT)
- Firefox Responsive Design Mode (Ctrl+Shift+M)
- Prettier (MIT)

## Deliverables

- `index.html` + `style.css` in `/var/www/webeng/week03/`
- No horizontal scrolling at 360 px width
- Dark mode follows the OS; animations stop with reduced-motion
- Stylelint reports 0 problems

## Common Source Files (identical on both operating systems)

**index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-
scale=1">
  <title>Karachi Dev Meetup</title><link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="top"><strong>KDM</strong>
    <nav class="nav"><a href="#talks">Talks</a><a href="#venue">Venue</a><a
href="#join">Join</a></nav></header>
  <main>
    <section class="hero"><h1>Build the modern web</h1><p>Saturday talks on Linux, PHP and
cloud.</p>
      <a class="btn" href="#join">Reserve a seat</a><div class="loader" role="status"
aria-label="Loading"></div></section>
    <section id="talks" class="cards">
      <div class="card-wrap"><article class="card"><h2>Rootless Containers</h2><p>Podman
in production.</p></article></div>
      <div class="card-wrap"><article class="card"><h2>PHP 8.5 Tour</h2><p>Pipes and URI
API.</p></article></div>
      <div class="card-wrap"><article class="card"><h2>Angular Signals</h2><p>Reactive
UIs.</p></article></div>
    </section>
    <section id="venue"><h2>Venue</h2><p>Hamdard University, Karachi Campus.</p></section>
  </main>
  <footer><small>&copy; 2026 KDM</small></footer>
</body>
</html>
```

**style.css**

```
@layer reset, base, layout;
@layer reset { *, *::before, *::after { box-sizing: border-box; } body { margin: 0; } }
:root { --brand: #0f766e; --bg: #ffffff; --fg: #0f172a; --card: #f1f5f9; }
@media (prefers-color-scheme: dark) { :root { --bg: #0b1220; --fg: #e5e7eb; --card:
#1e293b; } }
@layer base {
  body { font-family: system-ui, sans-serif; background: var(--bg); color: var(--fg);
line-height: 1.6; }
  h1 { font-size: clamp(1.75rem, 4vw + 1rem, 3.5rem); text-shadow: 1px 1px 2px #0003; }
  a { color: var(--brand); }
  .btn { display: inline-block; padding: .75rem 1.25rem; border-radius: .5rem; background:
var(--brand);
         color: #fff; text-decoration: none; transition: transform .2s; }
  .btn:hover, .btn:focus-visible { transform: translateY(-2px); }
```

```
}
@layer layout {
  .top { display: flex; justify-content: space-between; align-items: center; padding:
1rem; }
  .nav { display: flex; gap: 1rem; }
  .hero { padding: 3rem 1rem; text-align: center; }
  .cards { display: grid; gap: 1rem; padding: 1rem; grid-template-columns: repeat(auto-
fit, minmax(16rem, 1fr)); }
  .card-wrap { container-type: inline-size; }
  .card { background: var(--card); padding: 1rem; border: 2px solid transparent; border-
radius: .75rem; }
  .card:hover { border-color: var(--brand); }
  @container (min-width: 22rem) { .card { display: flex; gap: 1rem; align-items:
center; } }
  #venue, footer { padding: 1rem; }
}
.loader { margin: 1.5rem auto 0; width: 2rem; aspect-ratio: 1; border: .3rem solid
#cbd5e1;
          border-top-color: var(--brand); border-radius: 50%; animation: spin 1s linear
infinite; }
@keyframes spin { to { transform: rotate(360deg); } }
@media (prefers-reduced-motion: reduce) { .loader { animation: none; } .btn { transition:
none; } }
```

## Version A – Rocky Linux 10

**Step 1 – Create files and fix labels.** Save index.html and style.css, then relabel.

```
mkdir -p /var/www/webeng/week03 && cd /var/www/webeng/week03
kate index.html style.css &
sudo restorecon -Rv .
```

**Step 2 – Install the linter.** Stylelint checks CSS the way a compiler checks code.

```
npm init -y && npm install --save-dev stylelint stylelint-config-standard prettier
echo '{ "extends": "stylelint-config-standard" }' > .stylelintrc.json
npx prettier --write style.css && npx stylelint style.css
```

**Step 3 – Test responsiveness.** Open the page, press Ctrl+Shift+M, choose 360×640, then 1280×800. Toggle dark mode in Firefox about:preferences → Colors.

```
firefox http://webeng.test/week03/ &
```

**Step 4 – Check delivered size and caching.** Industry teams track transferred bytes.

```
curl -s -o /dev/null -w "%{size_download} bytes, HTTP %{http_code}\n"
http://webeng.test/week03/style.css
curl -sI http://webeng.test/week03/style.css | grep -iE 'etag|last-modified|content-type'
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Create files.** Save index.html and style.css.

```
mkdir -p /var/www/webeng/week03 && cd /var/www/webeng/week03
kate index.html style.css &
```

**Step 2 – Install the linter.** Stylelint checks CSS the way a compiler checks code.

```
npm init -y && npm install --save-dev stylelint stylelint-config-standard prettier
echo '{ "extends": "stylelint-config-standard" }' > .stylelintrc.json
npx prettier --write style.css && npx stylelint style.css
```

**Step 3 – Test responsiveness.** Open the page, press Ctrl+Shift+M, choose 360×640, then 1280×800. Switch Plasma to the dark theme (System Settings → Colors & Themes) to test dark mode.

```
firefox http://webeng.test/week03/ &
```

**Step 4 – Check delivered size and caching.** Industry teams track transferred bytes.

```
curl -s -o /dev/null -w "%{size_download} bytes, HTTP %{http_code}\n"
http://webeng.test/week03/style.css
curl -sI http://webeng.test/week03/style.css | grep -iE 'etag|last-modified|content-type'
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `npx stylelint style.css` | No problems reported |
| 2 | Width 360 px | Single column, no horizontal scrollbar |
| 3 | Width ≥ 1000 px | Three card columns; cards switch to row layout via container query |
| 4 | OS dark mode on | Background and text colours invert using custom properties |
| 5 | Reduced motion enabled | Loader stops spinning |

## Stretch Goals (Industry Practice)

- Add a `@media print` stylesheet.
- Replace the colour palette using `oklch()` and `color-mix()`.
- Use `@view-transition` or scroll-driven animation (progressive enhancement).

## Review Questions

1. Order these selectors by specificity: `#a .b`, `ul li.active`, `a:hover`, `:where(.x)`.
2. What problem do cascade layers solve compared with `!important`?
3. When would you pick Grid over Flexbox? Give two examples.
4. Explain the difference between a media query and a container query.
5. Why animate `transform` instead of `left`?

# Week 4: JavaScript Fundamentals (ES2025 and Beyond)

| Syllabus Lectures | Lec 7–8 | CLO | CLO_2 |
|---|---|---|---|
| PLO | PLO_5 | Bloom's Level | C3 (Applying) |
| Reading | Ch. 5 Introduction to JavaScript | Assessment | Q1 (CLO_1, 4 marks) |

## Learning Outcomes

- Declare variables with `let` / `const`, explain hoisting and use the main data types.
- Write functions (declaration, arrow, default / rest parameters) and use closures.
- Use arrays, objects, destructuring, spread, getters/setters and ES modules.
- Apply conditionals, `switch`, loops and `continue` / `break`.
- Unit-test code with the built-in Node.js test runner and lint with ESLint.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of CSS, then Quiz Q1 (15 min) on Weeks 1–3 |
| 0:10 – 0:45 | 35 | Concept Block 1 | Runtime, types, variables, hoisting, operators, control flow |
| 0:45 – 1:20 | 35 | Concept Block 2 | Functions, arrays, objects, modules, getters/setters, DOM basics |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 4 – Expense tracker with tests |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 4.1 How JavaScript Runs

JavaScript is single-threaded and event-driven. Engines (V8 in Chrome and Node.js, SpiderMonkey in Firefox, JavaScriptCore in Safari) compile it just-in-time. The same language runs in the **browser** (DOM, `fetch`, Web APIs) and on the **server** (Node.js, also Deno and Bun) with different libraries. The ECMAScript standard is updated every year ("ES2025" etc.).

### 4.2 Variables, Types and Hoisting

```
const PI = 3.14159;          // cannot be reassigned
let count = 0;               // block scoped, reassignable
// var is function-scoped and hoisted: avoid it
console.log(typeof 42, typeof 'a', typeof true, typeof undefined, typeof null, typeof {},
typeof 10n);
// number string boolean undefined object(!) object bigint   (Symbol is the 8th type)
console.log(a); var a = 1;   // undefined (declaration hoisted)
console.log(b); let b = 1;   // ReferenceError (temporal dead zone)
```

Equality: always use `===`. Falsy values: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Useful operators: `??` (null/undefined fallback), `?.` (optional chaining), `**` (power), `&&=`, `||=`, `??=`.

### 4.3 Control Flow

```
for (let i = 0; i < 10; i++) { if (i % 2) continue; if (i > 6) break; console.log(i); }
for (const x of [10, 20]) console.log(x);          // iterate values
for (const k in { a: 1 }) console.log(k);          // iterate keys
let n = 3; while (n--) console.log(n);
switch (grade) { case 'A': case 'B': msg = 'good'; break; default: msg = 'review'; }
```

### 4.4 Functions and Closures

```
function add(a, b = 0) { return a + b; }
const mul = (a, b) => a * b;                 // arrow function (lexical this)
const sum = (...nums) => nums.reduce((s, n) => s + n, 0);
function counter() { let c = 0; return () => ++c; }   // closure keeps c alive
const next = counter(); next(); next();                // 2
```

### 4.5 Arrays and Objects

```
const nums = [5, 3, 8];
nums.map(n => n * 2); nums.filter(n => n > 4); nums.reduce((a, b) => a + b, 0);
nums.toSorted((a, b) => a - b);          // non-mutating (ES2023)
nums.at(-1); nums.findLast(n => n < 8);
Object.groupBy(items, i => i.category);  // ES2024

const user = { name: 'Sara', address: { city: 'Karachi' } };
const { name, address: { city } } = user;          // destructuring
const copy = { ...user, name: 'Ali' };             // spread
Object.entries(user); structuredClone(user);

const acct = { _bal: 0, get balance() { return this._bal; },
               set balance(v) { if (v < 0) throw new RangeError('negative'); this._bal =
v; } };
```

Newer built-ins to know: `Set` methods (`union`, `intersection`), iterator helpers (`.map()` / `.take()` on iterators), `Promise.withResolvers()` and the `Temporal` date API (shipping in current runtimes; check support before relying on it).

### 4.6 Modules and the DOM

```
// math.js
export const sq = n => n * n;
// app.js  (load with <script type="module" src="app.js"></script>)
import { sq } from './math.js';
document.querySelector('#out').textContent = sq(7);
document.querySelector('button').addEventListener('click', e => console.log(e.type));
```

> **Security:** Never build HTML by concatenating user input into `innerHTML` (XSS). Use `textContent` or create elements.

## Hands-on Lab 4: Expense Tracker – Tested ES Module Library with a Browser UI

Write a pure-function library for expenses, test it with `node --test`, lint it with ESLint, and attach a small browser UI served by Apache.

## Free & Open-Source Tools

- Node.js 24 LTS test runner `node:test` (MIT)
- ESLint 9 flat config + `@eslint/js` (MIT)
- Prettier (MIT)

## Deliverables

- `expenses.js`, `expenses.test.js`, `app.js`, `index.html` in `/var/www/webeng/week04/`
- All tests pass (`node --test`)
- ESLint clean
- Working UI that adds expenses and displays totals per category

## Common Source Files (identical on both operating systems)

**expenses.js**

```
export function addExpense(list, { title, amount, category }) {
  if (!title?.trim()) throw new TypeError('title required');
  if (!Number.isFinite(amount) || amount <= 0) throw new RangeError('amount must be > 0');
  return [...list, { id: list.length + 1, title: title.trim(), amount, category:
category ?? 'other' }];
}
export function totalsByCategory(list) {
  const out = {};
  for (const e of list) out[e.category] = (out[e.category] ?? 0) + e.amount;
  return out;
}
export const biggest = list => list.reduce((m, e) => (e.amount > (m?.amount ?? 0) ? e :
m), null);
export function grade(total) {
  switch (true) { case total < 5000: return 'frugal'; case total < 20000: return 'normal';
default: return 'high'; }
}
```

**expenses.test.js**

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { addExpense, totalsByCategory, biggest, grade } from './expenses.js';

test('addExpense validates input', () => {
  assert.throws(() => addExpense([], { title: '', amount: 5 }), TypeError);
  assert.throws(() => addExpense([], { title: 'x', amount: -1 }), RangeError);
});
test('totalsByCategory groups amounts', () => {
  let l = addExpense([], { title: 'Bus', amount: 100, category: 'travel' });
  l = addExpense(l, { title: 'Lunch', amount: 300, category: 'food' });
  l = addExpense(l, { title: 'Taxi', amount: 400, category: 'travel' });
  assert.deepEqual(totalsByCategory(l), { travel: 500, food: 300 });
  assert.equal(biggest(l).title, 'Taxi');
});
test('grade thresholds', () => { assert.equal(grade(100), 'frugal');
assert.equal(grade(30000), 'high'); });
```

**index.html**

```
<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"><title>Expenses</title>
<meta name="viewport" content="width=device-width, initial-scale=1"></head><body><main>
<h1>Expense Tracker</h1>
<form id="f"><input name="title" placeholder="Title" required>
<input name="amount" type="number" min="1" step="1" placeholder="Amount" required>
```

```
<select
name="category"><option>food</option><option>travel</option><option>study</option></select
>
<button>Add</button></form>
<ul id="list"></ul><p id="summary"></p></main>
<script type="module" src="app.js"></script></body></html>
```

**app.js**

```
import { addExpense, totalsByCategory, grade } from './expenses.js';
let expenses = [];
const list = document.querySelector('#list'), summary =
document.querySelector('#summary');
document.querySelector('#f').addEventListener('submit', e => {
  e.preventDefault();
  const d = new FormData(e.target);
  try { expenses = addExpense(expenses, { title: d.get('title'), amount:
Number(d.get('amount')), category: d.get('category') }); }
  catch (err) { alert(err.message); return; }
  list.replaceChildren(...expenses.map(x => Object.assign(document.createElement('li'),
{ textContent: `${x.title} – ${x.amount} (${x.category})` })));
  const total = expenses.reduce((s, x) => s + x.amount, 0);
  summary.textContent = `Total ${total} → ${grade(total)} |
${JSON.stringify(totalsByCategory(expenses))}`;
  e.target.reset();
});
```

## Version A – Rocky Linux 10

**Step 1 – Create the project.** ES modules need `"type": "module"` in package.json.

```
mkdir -p /var/www/webeng/week04 && cd /var/www/webeng/week04
npm init -y && npm pkg set type=module
npm install --save-dev eslint @eslint/js prettier
kate expenses.js expenses.test.js app.js index.html &
```

**Step 2 – Configure ESLint (flat config).** ESLint 9 uses eslint.config.js.

```
cat > eslint.config.js <<'EOF'
import js from '@eslint/js';
export default [js.configs.recommended,
  { languageOptions: { globals: { document: 'readonly', alert: 'readonly', FormData:
'readonly' } } }];
EOF
```

**Step 3 – Run tests and lint.** Tests first, then static analysis, then formatting.

```
node --test
npx eslint .
npx prettier --check .
```

**Step 4 – Publish with correct labels.** node_modules does not need to be served; keep it out of the web root in real projects.

```
sudo restorecon -Rv /var/www/webeng/week04
firefox http://webeng.test/week04/ &
```

**Step 5 – Debug in DevTools.** Set a breakpoint inside the submit handler and watch `expenses`.

```
# Firefox: F12 → Debugger → app.js → click line number → submit the form
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Create the project.** ES modules need `"type": "module"` in package.json.

```
mkdir -p /var/www/webeng/week04 && cd /var/www/webeng/week04
npm init -y && npm pkg set type=module
npm install --save-dev eslint @eslint/js prettier
kate expenses.js expenses.test.js app.js index.html &
```

**Step 2 – Configure ESLint (flat config).** ESLint 9 uses eslint.config.js.

```
cat > eslint.config.js <<'EOF'
import js from '@eslint/js';
export default [js.configs.recommended,
  { languageOptions: { globals: { document: 'readonly', alert: 'readonly', FormData:
'readonly' } } }];
EOF
```

**Step 3 – Run tests and lint.** Tests first, then static analysis, then formatting.

```
node --test
npx eslint .
npx prettier --check .
```

**Step 4 – Publish.** Apache serves modules with the correct MIME type (text/javascript).

```
curl -sI http://webeng.test/week04/app.js | grep -i content-type
firefox http://webeng.test/week04/ &
```

**Step 5 – Debug in DevTools.** Set a breakpoint inside the submit handler and watch `expenses`.

```
# Firefox: F12 → Debugger → app.js → click line number → submit the form
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `node --test` | 3 tests pass, 0 fail |
| 2 | `npx eslint .` | No errors |
| 3 | Add expense with empty title in the UI | Alert “title required” |
| 4 | Add three expenses | Totals per category and grade label appear |
| 5 | `curl -sI .../app.js` | `Content-Type: text/javascript` |

## Stretch Goals (Industry Practice)

- Persist expenses with `localStorage` and test the serialiser.
- Add `npm test` / `npm run lint` scripts and a Git pre-commit hook.
- Implement `totalsByCategory` with `Object.groupBy` and compare readability.

# Review Questions

1. Explain hoisting and the temporal dead zone with a short example.
2. What is the difference between == and ===? Predict `0 == ""` and `0 === ""`.
3. How does a closure keep a private counter?
4. Rewrite a `for` loop using `map` and `filter`.
5. Why should unit tests target pure functions rather than DOM code?

# Week 5: Advanced JavaScript: OOP, Async, CSV Data and Canvas / 3D

| Syllabus Lectures | Lec 9–10 | CLO | CLO_2 |
|---|---|---|---|
| PLO | PLO_5 | Bloom's Level | C3 (Applying) |
| Reading | Ch. 5–7 JavaScript, jQuery | Assessment | A2 issued (formative, 0 marks in OBE) |

## Learning Outcomes

- Model data with classes: inheritance, polymorphism, private fields, static members.
- Handle errors with `try/catch/finally` and custom error classes.
- Use Promises, `async/await` and `fetch`, and validate e-mail with a regular expression.
- Parse, edit and export CSV data in an editable table.
- Animate 2D shapes on `<canvas>` and render a rotating 3D cube with Three.js built by Vite.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | Classes, inheritance, polymorphism, errors, async/await, fetch |
| 0:45 – 1:20 | 35 | Concept Block 2 | Regex validation, CSV, editable tables, canvas 2D/3D, jQuery vs modern JS |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 5 – CSV explorer and 3D cube with Vite |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 5.1 Classes, Inheritance and Polymorphism

```
class Person {
  #cnic;                                  // private field
  static count = 0;
  constructor(name, cnic) { this.name = name; this.#cnic = cnic; Person.count++; }
  get masked() { return '*****-*******-' + this.#cnic.slice(-1); }
  describe() { return `Person ${this.name}`; }
}
class Student extends Person {
  constructor(name, cnic, marks) { super(name, cnic); this.marks = marks; }
  describe() { return `${super.describe()} (marks ${this.marks})`; }   // overriding =
polymorphism
}
for (const p of [new Person('Ali', '42101-1234567-1'), new Student('Sara', '42101-7654321-
9', 88)])
  console.log(p.describe());                // each object answers in its own way
```

Prefer **composition over inheritance** when behaviour is shared across unrelated classes (for example a `Serializable` helper object). JavaScript uses prototypes underneath; `class` is syntax over them.

## 5.2 Exceptions

```
class ValidationError extends Error {
  constructor(field, msg, options) { super(msg, options); this.name = 'ValidationError';
this.field = field; }
}
try { throw new ValidationError('email', 'Invalid e-mail'); }
catch (e) { if (e instanceof ValidationError) console.warn(e.field, e.message); else throw
e; }
finally { console.log('always runs'); }
```

## 5.3 Asynchronous JavaScript

Browsers and Node.js never block on I/O. Work is queued and the **event loop** runs callbacks when the call stack is empty. Evolution: callbacks → Promises → `async`/`await`.

```
async function loadCsv(url) {
  const ctrl = new AbortController();
  const t = setTimeout(() => ctrl.abort(), 5000);
  try {
    const res = await fetch(url, { signal: ctrl.signal });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.text();
  } finally { clearTimeout(t); }
}
const [a, b] = await Promise.all([loadCsv('a.csv'), loadCsv('b.csv')]);   // parallel
```

## 5.4 Regular Expressions and E-mail Validation

```
const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;     // pragmatic, not RFC-complete
EMAIL.test('ali@hamdard.edu.pk');                    // true
'2026-10-29'.match(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/).groups.y;   // named groups
```

> **Industry practice:** No regular expression can prove that a mailbox exists. Use `type="email"` for UX, a light regex for sanity and a **confirmation e-mail** (Week 13) for truth. Beware catastrophic backtracking on user-supplied patterns.

## 5.5 CSV Files and Editable Tables

CSV looks trivial but has quoting rules (commas and newlines inside quotes, doubled quotes). Never split on commas manually; use a parser such as **PapaParse**. Edit cells with `contenteditable` or `<input>` elements, validate on `change` and export with a `Blob` and a temporary download link.

## 5.6 Canvas 2D and 3D

```
const ctx = canvas.getContext('2d'); let angle = 0;
function frame() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.save(); ctx.translate(150, 150); ctx.rotate(angle += 0.02);
  ctx.fillStyle = 'teal'; ctx.fillRect(-40, -40, 80, 80); ctx.restore();
  requestAnimationFrame(frame);
}
frame();
```

3D in the browser uses **WebGL / WebGPU**. Libraries such as **Three.js** provide a scene graph (scene, camera, mesh = geometry + material, renderer) and hide the low-level API.

## 5.7 jQuery as Legacy Knowledge

| jQuery | Modern equivalent |
|---|---|
| `$("#id"), $(".c")` | `document.querySelector("#id"), querySelectorAll(".c")` |
| `$(el).on("click", fn)` | `el.addEventListener("click", fn)` |
| `$(el).addClass("on")` | `el.classList.add("on")` |
| `$.ajax()` / `$.get()` | `fetch()` + `await res.json()` |
| `$(el).hide()` / `.fadeIn()` | `el.hidden = true`; CSS transitions |

Event delegation (one listener on a parent that inspects `event.target`) is still essential: `table.addEventListener("click", e => { const b = e.target.closest("button[data-del]"); … }).`

# Hands-on Lab 5: CSV Student Explorer and Rotating 3D Cube (Vite Build)

Build a Vite project that loads a CSV file, validates e-mail addresses, lets users edit marks in an HTML table, exports the result and shows a Three.js rotating cube; deploy the production build on Apache.

## Free & Open-Source Tools

- Vite (MIT) for dev server and production build
- PapaParse (MIT) for CSV
- Three.js (MIT) for 3D
- Node.js 24 LTS

## Deliverables

- Vite project in `~/webeng/week05` (source, not in web root)
- Production build copied to `/var/www/webeng/week05/`
- Editable table with e-mail validation and CSV export
- Rotating cube and rotating 2D square on the same page

## Common Source Files (identical on both operating systems)

**public/students.csv (row 3 has an invalid e-mail on purpose)**

```
id,name,email,marks
1,Ayesha Khan,ayesha@hamdard.edu.pk,88
2,"Siddiqui, Bilal",bilal@hamdard.edu.pk,74
3,Hira Ali,hira.hamdard.edu.pk,91
```

**index.html**

```
<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"><title>CSV Explorer</title>
<meta name="viewport" content="width=device-width, initial-scale=1"></head><body><main>
<h1>CSV Student Explorer</h1>
<table id="t" border="1"><thead><tr><th>ID</th><th>Name</th><th>E-
mail</th><th>Marks</th><th>Grade</th></tr></thead><tbody></tbody></table>
<p><button id="export">Export CSV</button> <span id="msg" role="status"></span></p>
<canvas id="c2d" width="200" height="200"></canvas> <div id="c3d"></div></main>
<script type="module" src="/main.js"></script></body></html>
```

**main.js**

```
import Papa from 'papaparse';
import * as THREE from 'three';
```

```
const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
class Student {
  #marks;
  constructor(row) { Object.assign(this, { id: row.id, name: row.name, email:
row.email }); this.marks = row.marks; }
  get marks() { return this.#marks; }
  set marks(v) { const n = Number(v); if (!Number.isFinite(n) || n < 0 || n > 100) throw
new RangeError('0-100'); this.#marks = n; }
  get grade() { return this.marks >= 85 ? 'A' : this.marks >= 70 ? 'B' : 'C'; }
}
const base = import.meta.env.BASE_URL;
const text = await (await fetch(base + 'students.csv')).text();
const rows = Papa.parse(text, { header: true, skipEmptyLines: true }).data;
const students = rows.map(r => new Student(r));
const tbody = document.querySelector('#t tbody'), msg = document.querySelector('#msg');

function render() {
  tbody.replaceChildren(...students.map((s, i) => {
    const tr = document.createElement('tr');
    const ok = EMAIL.test(s.email);
    tr.innerHTML = '<td></td><td></td><td></td><td contenteditable="true" data-i="' + i +
'"></td><td></td>';
    const c = tr.children;
    c[0].textContent = s.id; c[1].textContent = s.name; c[2].textContent = s.email + (ok ?
'' : '  ⚠ invalid');
    c[3].textContent = s.marks; c[4].textContent = s.grade;
    return tr;
  }));
}
tbody.addEventListener('focusout', e => {            // event delegation
  const td = e.target.closest('td[data-i]'); if (!td) return;
  try { students[td.dataset.i].marks = td.textContent; msg.textContent = 'saved'; }
  catch { msg.textContent = 'Marks must be 0-100'; }
  render();
});
document.querySelector('#export').addEventListener('click', () => {
  const csv = Papa.unparse(students.map(s => ({ id: s.id, name: s.name, email: s.email,
marks: s.marks })));
  const a = Object.assign(document.createElement('a'), { href: URL.createObjectURL(new
Blob([csv], { type: 'text/csv' })), download: 'students-out.csv' });
  a.click(); URL.revokeObjectURL(a.href);
});
render();

// 2D canvas rotation
const ctx = document.querySelector('#c2d').getContext('2d'); let ang = 0;
(function frame() { ctx.clearRect(0, 0, 200, 200); ctx.save(); ctx.translate(100, 100);
ctx.rotate(ang += 0.03);
  ctx.fillStyle = '#0f766e'; ctx.fillRect(-35, -35, 70, 70); ctx.restore();
requestAnimationFrame(frame); })();

// 3D cube
const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
renderer.setSize(220, 220);
document.querySelector('#c3d').append(renderer.domElement);
const scene = new THREE.Scene(), cam = new THREE.PerspectiveCamera(60, 1, 0.1, 10);
cam.position.z = 2.2;
const cube = new THREE.Mesh(new THREE.BoxGeometry(1, 1, 1), new
THREE.MeshNormalMaterial()); scene.add(cube);
renderer.setAnimationLoop(() => { cube.rotation.x += 0.01; cube.rotation.y += 0.015;
renderer.render(scene, cam); });
```

## Version A – Rocky Linux 10

**Step 1 – Scaffold the project manually.** Keeping source outside the web root is the professional pattern; only the built files are served.

```
mkdir -p ~/webeng/week05/public && cd ~/webeng/week05
npm init -y && npm pkg set type=module
npm install papaparse three && npm install --save-dev vite
kate index.html main.js public/students.csv &
```

**Step 2 – Run the dev server with hot reload.** Vite listens on 5173; open the firewall only if testing from another machine.

```
npx vite --host 127.0.0.1
# browser: http://127.0.0.1:5173/      (Ctrl+C to stop)
```

**Step 3 – Build for production under /week05/.** The --base option rewrites asset URLs.

```
npx vite build --base=/week05/
ls -lh dist dist/assets
```

**Step 4 – Deploy and relabel.** rsync copies and --delete removes stale files; restorecon fixes labels for SELinux.

```
rsync -a --delete dist/ /var/www/webeng/week05/
sudo restorecon -Rv /var/www/webeng/week05
curl -sI http://webeng.test/week05/assets/ | head -n 1
```

**Step 5 – Enable compression for text assets.** mod_deflate is built in; add a rule and compare transfer size.

```
echo 'AddOutputFilterByType DEFLATE text/html text/css text/javascript' | sudo tee
/etc/httpd/conf.d/deflate.conf
sudo systemctl reload httpd
curl -s -H 'Accept-Encoding: gzip' -o /dev/null -w "%{size_download}\n"
http://webeng.test/week05/assets/$(ls dist/assets | grep js | head -1)
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Scaffold the project manually.** Keeping source outside the web root is the professional pattern; only the built files are served.

```
mkdir -p ~/webeng/week05/public && cd ~/webeng/week05
npm init -y && npm pkg set type=module
npm install papaparse three && npm install --save-dev vite
kate index.html main.js public/students.csv &
```

**Step 2 – Run the dev server with hot reload.** Vite listens on 5173.

```
npx vite --host 127.0.0.1
# browser: http://127.0.0.1:5173/      (Ctrl+C to stop)
```

**Step 3 – Build for production under /week05/.** The --base option rewrites asset URLs.

```
npx vite build --base=/week05/
ls -lh dist dist/assets
```

**Step 4 – Deploy.** rsync copies and --delete removes stale files.

```
rsync -a --delete dist/ /var/www/webeng/week05/
```

```
curl -sI http://webeng.test/week05/ | head -n 1
```

**Step 5 – Enable compression for text assets.** Enable mod_deflate and compare transfer size.

```
sudo a2enmod deflate && sudo systemctl reload apache2
curl -s -H 'Accept-Encoding: gzip' -o /dev/null -w "%{size_download}\n"
http://webeng.test/week05/assets/$(ls dist/assets | grep js | head -1)
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | Open `http://webeng.test/week05/` | Table with 3 rows; row 3 shows "⚠ invalid" |
| 2 | Edit marks to 150 | Message "Marks must be 0-100", value reverts |
| 3 | Edit marks to 95 | Grade changes to A |
| 4 | Click Export CSV | `students-out.csv` downloads with the edited marks; the quoted name stays intact |
| 5 | Cube and square | Both rotate continuously |
| 6 | Compression test | Transferred size is much smaller than the file size on disk |

## Stretch Goals (Industry Practice)

- Add Vitest unit tests for the `Student` class.
- Save edits to `localStorage` and reload.
- Replace the cube with a loaded glTF model.

## Assignment A2 (CLO_2, formative) – issued this week, due Week 6

Extend the CSV explorer with sorting by column, a text filter, a count of invalid e-mails and an `AbortController` timeout on the CSV request. Submit the Git repository link and the `dist` folder.

# Review Questions

1. Show how method overriding provides polymorphism using two classes of your own.
2. Why is a `#private` field better than an underscore prefix?
3. What does `await Promise.all([...])` do differently from awaiting each promise in turn?
4. Why is splitting CSV text on commas unsafe? Give an example row.
5. Map three jQuery calls to their modern equivalents.

# Week 6: Node.js and Modern Angular

| Syllabus Lectures | Lec 11–12 | CLO | CLO_2 |
|---|---|---|---|
| PLO | PLO_5 | Bloom's Level | C3 (Applying) |
| Reading | Ch. 8–9 Frameworks, Node.js | Assessment | A2 due |

## Learning Outcomes

- Describe the Node.js architecture (V8, libuv, event loop) and use core modules.
- Distinguish events, callbacks and promises and use `EventEmitter`.
- Build a dependency-free REST API with `node:http`.
- Create an Angular application with standalone components, signals, services and tables, and consume the API.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | A2 review and recap of async/await |
| 0:10 – 0:45 | 35 | Concept Block 1 | Node.js architecture, modules, events vs callbacks, HTTP server, JSON API |
| 0:45 – 1:20 | 35 | Concept Block 2 | Angular vs AngularJS, components, signals, templates, services, HttpClient |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 6 – Node REST API and Angular dashboard |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 6.1 Node.js Architecture

Node.js embeds the **V8** engine and **libuv**, a C library that provides the event loop, a thread pool and asynchronous I/O. JavaScript runs on one thread; blocking I/O is delegated to the OS or the pool, and completion callbacks are queued. Therefore never run long CPU loops in a request handler.

| Event-loop phase | What runs |
|---|---|
| timers | `setTimeout`, `setInterval` callbacks |
| poll | I/O callbacks (network, files) |
| check | `setImmediate` callbacks |
| between phases | `process.nextTick` and Promise microtasks |

### 6.2 Core Modules, Events and Callbacks

| Concept | Example | When used |
|---|---|---|
| Callback | `fs.readFile(p, (err, data) => …)` | Legacy APIs; error-first convention |
| Promise | `const data = await fs.promises.readFile(p)` | One-off async result (preferred) |
| Event | `emitter.on("order", fn)` | Repeated notifications, many listeners |

```
import { EventEmitter } from 'node:events';
import { readFile } from 'node:fs/promises';
const bus = new EventEmitter();
bus.on('order', (id) => console.log('log order', id));
bus.on('order', (id) => console.log('email for', id));
bus.emit('order', 101);                       // both listeners run synchronously
console.log((await readFile('package.json', 'utf8')).length);
```

Useful built-ins: `node:fs`, `node:path`, `node:os`, `node:http`, `node:crypto`, `node:stream`, `node:test`, global `fetch`, and the `--watch` flag for auto-restart.

### 6.3 Angular vs AngularJS

The syllabus lists "Angular JS". **AngularJS (1.x)** is a different, obsolete framework (end-of-life since January 2022). Today's **Angular** is a TypeScript framework maintained by Google with a CLI, dependency injection, routing, forms and an HTTP client. Current Angular favours **standalone components** (no NgModules), **signals** for reactivity and built-in control flow (`@if`, `@for`).

```
@Component({ selector: 'app-root', imports: [FormsModule], templateUrl: './app.html' })
export class App {
  items = signal<string[]>(['A', 'B']);
  q = signal('');
  shown = computed(() => this.items().filter(i => i.includes(this.q())));   // auto-
updates
}
// template:  @for (i of shown(); track i) { <li>{{ i }}</li> } @empty { <li>none</li> }
```

| Angular building block | Purpose |
|---|---|
| Component | Class + template + styles; the unit of UI |
| Signal / computed / effect | Fine-grained reactive state |
| Service + `inject()` | Shared logic and data access via dependency injection |
| `HttpClient` | Typed HTTP calls returning Observables |
| Router | URL ↔ component mapping, lazy loading |
| Pipes | Template formatting (`date`, `currency`, `uppercase`) |

> **Best practice:** Avoid touching the DOM directly with `document.querySelector` inside Angular. Bind data in templates; if you must use the DOM use `ElementRef` / `Renderer2`, which keeps the code testable and safe from XSS.

## Hands-on Lab 6: Node REST API and Angular Course Dashboard

Create a no-dependency Node.js JSON API with event logging, then an Angular app that lists, filters and adds courses using signals and HttpClient. Build and deploy the Angular app on Apache.

### Free & Open-Source Tools

- Node.js 24 LTS (`node:http`, `node:events`)
- Angular CLI (MIT) via `npx @angular/cli@latest`
- curl, jq for API testing

## Deliverables

- `api/server.js` running on port 3000 with GET and POST `/api/courses`
- Angular app in `~/webeng/week06/dashboard`
- Production build deployed to `/var/www/webeng/week06/`
- Callback-vs-promise comparison file `compare.js`

## Common Source Files (identical on both operating systems)

**api/server.js**

```
import { createServer } from 'node:http';
import { EventEmitter } from 'node:events';

const bus = new EventEmitter();
bus.on('request', ({ method, url }) => console.log(new Date().toISOString(), method,
url));
const courses = [
  { id: 1, code: 'SEWE-6109', title: 'Web Engineering', credits: 3 },
  { id: 2, code: 'SEDB-5101', title: 'Databases', credits: 3 },
];
const json = (res, code, body) => {
  res.writeHead(code, { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin':
'*',
    'Access-Control-Allow-Headers': 'Content-Type' });
  res.end(JSON.stringify(body));
};
createServer(async (req, res) => {
  bus.emit('request', req);
  if (req.method === 'OPTIONS') return json(res, 204, '');
  if (req.url === '/api/courses' && req.method === 'GET') return json(res, 200, courses);
  if (req.url === '/api/courses' && req.method === 'POST') {
    let raw = ''; for await (const chunk of req) raw += chunk;
    try {
      const { code, title, credits } = JSON.parse(raw);
      if (!code || !title || !(credits > 0)) return json(res, 400, { error: 'invalid' });
      const c = { id: courses.length + 1, code, title, credits }; courses.push(c); return
json(res, 201, c);
    } catch { return json(res, 400, { error: 'bad json' }); }
  }
  json(res, 404, { error: 'not found' });
}).listen(3000, '127.0.0.1', () => console.log('API on http://127.0.0.1:3000'));
```

**compare.js**

```
import { readFile } from 'node:fs';
import { readFile as readP } from 'node:fs/promises';
// 1) callback style (error-first, nesting grows)
readFile('package.json', 'utf8', (err, data) => { if (err) return console.error(err);
console.log('cb', data.length); });
// 2) promise style
readP('package.json', 'utf8').then(d => console.log('promise',
d.length)).catch(console.error);
// 3) async/await
try { console.log('await', (await readP('package.json', 'utf8')).length); } catch (e)
{ console.error(e); }
```

**src/app/app.ts**

```
import { Component, inject, signal, computed } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { FormsModule } from '@angular/forms';

interface Course { id: number; code: string; title: string; credits: number; }
```

```
@Component({ selector: 'app-root', imports: [FormsModule], templateUrl: './app.html',
styleUrl: './app.css' })
export class App {
  private http = inject(HttpClient);
  private api = 'http://127.0.0.1:3000/api/courses';
  courses = signal<Course[]>([]);
  query = signal('');
  draft = { code: '', title: '', credits: 3 };
  filtered = computed(() => this.courses().filter(c => (c.code +
c.title).toLowerCase().includes(this.query().toLowerCase())));
  totalCredits = computed(() => this.filtered().reduce((s, c) => s + c.credits, 0));
  constructor() { this.load(); }
  load() { this.http.get<Course[]>(this.api).subscribe(d => this.courses.set(d)); }
  add() { this.http.post<Course>(this.api, this.draft).subscribe(c =>
{ this.courses.update(l => [...l, c]); this.draft = { code: '', title: '', credits:
3 }; }); }
}
```

**src/app/app.html**

```
<main>
  <h1>Courses</h1>
  <input placeholder="Filter…" [ngModel]="query()" (ngModelChange)="query.set($event)">
  <table>
    <thead><tr><th>Code</th><th>Title</th><th>Credits</th></tr></thead>
    <tbody>
      @for (c of filtered(); track c.id)
{ <tr><td>{{ c.code }}</td><td>{{ c.title }}</td><td>{{ c.credits }}</td></tr> }
      @empty { <tr><td colspan="3">No courses</td></tr> }
    </tbody>
    <tfoot><tr><td colspan="2">Total
credits</td><td>{{ totalCredits() }}</td></tr></tfoot>
  </table>
  <h2>Add course</h2>
  <input placeholder="Code" [(ngModel)]="draft.code" name="code">
  <input placeholder="Title" [(ngModel)]="draft.title" name="title">
  <input type="number" min="1" [(ngModel)]="draft.credits" name="credits">
  <button (click)="add()">Add</button>
</main>
```

**src/app/app.config.ts (keep any router provider your CLI version generated)**

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideHttpClient, withFetch } from '@angular/common/http';
export const appConfig: ApplicationConfig = {
  providers: [provideBrowserGlobalErrorListeners(), provideHttpClient(withFetch())]
};
```

## Version A – Rocky Linux 10

**Step 1 – Create the API project.** Create files from the listings above.

```
mkdir -p ~/webeng/week06/api && cd ~/webeng/week06/api
npm init -y && npm pkg set type=module
kate server.js ../compare.js &
node --watch server.js &              # background; use "kill %1" to stop
```

**Step 2 – Test the API from the shell.** jq pretty-prints JSON.

```
curl -s http://127.0.0.1:3000/api/courses | jq .
curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"code":"SEWE-7000","title":"Cloud","credits":3}' http://127.0.0.1:3000/api/courses
-i | head -n 1
curl -s -X POST -d 'x' http://127.0.0.1:3000/api/courses -w "%{http_code}\n"
```

**Step 3 – Create the Angular app.** The CLI asks no questions with --defaults.

```
cd ~/webeng/week06
npx --yes @angular/cli@latest new dashboard --defaults --skip-git
cd dashboard && ls src/app
```

**Step 4 – Replace the component and config files.** Overwrite app.ts, app.html and app.config.ts with the listings above, then run the dev server.

```
npx ng serve --host 127.0.0.1
# browser: http://127.0.0.1:4200/
```

**Step 5 – Build and deploy.** Output is in dist/dashboard/browser.

```
npx ng build --base-href /week06/
rsync -a --delete dist/dashboard/browser/ /var/www/webeng/week06/
sudo restorecon -Rv /var/www/webeng/week06
curl -sI http://webeng.test/week06/ | head -n 1
```

**Step 6 – Understand the browser restriction (SELinux/CORS).** The deployed page at webeng.test calls the API at 127.0.0.1:3000 from the browser, so the API's CORS header is what permits it. Apache is not involved.

```
curl -sI -X OPTIONS http://127.0.0.1:3000/api/courses | grep -i access-control
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Create the API project.** Create files from the listings above.

```
mkdir -p ~/webeng/week06/api && cd ~/webeng/week06/api
npm init -y && npm pkg set type=module
kate server.js ../compare.js &
node --watch server.js &              # background; use "kill %1" to stop
```

**Step 2 – Test the API from the shell.** jq pretty-prints JSON.

```
curl -s http://127.0.0.1:3000/api/courses | jq .
curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"code":"SEWE-7000","title":"Cloud","credits":3}' http://127.0.0.1:3000/api/courses
-i | head -n 1
curl -s -X POST -d 'x' http://127.0.0.1:3000/api/courses -w "%{http_code}\n"
```

**Step 3 – Create the Angular app.** The CLI asks no questions with --defaults.

```
cd ~/webeng/week06
npx --yes @angular/cli@latest new dashboard --defaults --skip-git
cd dashboard && ls src/app
```

**Step 4 – Replace the component and config files.** Overwrite app.ts, app.html and app.config.ts with the listings above, then run the dev server.

```
npx ng serve --host 127.0.0.1
# browser: http://127.0.0.1:4200/
```

**Step 5 – Build and deploy.** Output is in dist/dashboard/browser.

```
npx ng build --base-href /week06/
rsync -a --delete dist/dashboard/browser/ /var/www/webeng/week06/
curl -sI http://webeng.test/week06/ | head -n 1
```

**Step 6 – Check the CORS header.** The browser (not Apache) enforces CORS against the API on port 3000.

```
curl -sI -X OPTIONS http://127.0.0.1:3000/api/courses | grep -i access-control
sudo ss -tlnp | grep -E ':3000|:80'
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `curl http://127.0.0.1:3000/api/courses` | JSON array with 2 courses |
| 2 | POST with invalid body | `400` and `{"error":"invalid"}` or `bad json` |
| 3 | API console | One log line per request produced by the EventEmitter listener |
| 4 | `node compare.js` | Three outputs (cb, promise, await) show the same length |
| 5 | Angular page | Table shows courses, filter box narrows rows, total credits updates, adding a course appears at once |
| 6 | Deployed build | `http://webeng.test/week06/` loads and lists courses while the API is running |

## Stretch Goals (Industry Practice)

- Add routing with a lazy-loaded “About” page.
- Replace `node:http` with Fastify or Hono and compare code size.
- Add a Vitest/Jasmine test for the `filtered` computed signal.

## Review Questions

1. Explain why one slow synchronous loop blocks every user of a Node.js server.
2. When would you choose an `EventEmitter` over a Promise?
3. List three differences between AngularJS 1.x and current Angular.
4. What does `computed()` do and why is it better than recalculating in the template?
5. Why does the browser need the `Access-Control-Allow-Origin` header in this lab?

# Week 7: Bootstrap 5.3, UI/UX and Component-Based Interfaces

| Syllabus Lectures | Lec 13–14 | CLO | CLO_2 |
|---|---|---|---|
| PLO | PLO_5 | Bloom's Level | C3 (Applying) |
| Reading | Ch. 10 UI Design Using Frameworks | Assessment | — |

## Learning Outcomes

- Lay out pages with the Bootstrap grid, breakpoints and utility classes.
- Use components: navbar, cards, carousel, pagination, modal; add icons with Bootstrap Icons.
- Replace removed features (Jumbotron, Glyphicons) with current techniques.
- Apply core UX heuristics and colour modes (light/dark) and wireframe with Penpot.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | UX principles, wireframing, grid, breakpoints, utilities |
| 0:45 – 1:20 | 35 | Concept Block 2 | Components: hero, carousel, pagination, icons, colour modes, accessibility |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 7 – Product catalogue with Bootstrap |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 7.1 UI/UX Foundations

- **Nielsen's heuristics:** visibility of system status, match with the real world, user control, consistency, error prevention, recognition over recall, flexibility, minimalist design, help users recover from errors, documentation.
- **Laws of UX:** Fitts's law (bigger, closer targets), Hick's law (fewer choices decide faster), Jakob's law (users expect your site to work like others).
- **Process:** user research → wireframe → prototype → usability test → build. **Penpot** (open-source design tool, browser-based) is used for wireframes.

### 7.2 Bootstrap Grid and Utilities

Bootstrap 5.3 is mobile-first and flexbox/grid-based. A `container` holds `rows` of up to 12 `cols`. Breakpoint prefixes: `sm` ≥576, `md` ≥768, `lg` ≥992, `xl` ≥1200, `xxl` ≥1400 px.

```
<div class="container">
  <div class="row g-3">
    <div class="col-12 col-md-6 col-lg-4">…</div>       <!-- 1 / 2 / 3 per row -->
  </div>
</div>
<p class="text-center fw-bold mt-3 mb-0 text-primary">Utilities replace custom CSS</p>
```

| Utility group | Examples |
|---|---|
| Spacing | `m-3`, `px-4`, `mt-auto`, `gap-2` |
| Display / flex | `d-flex`, `justify-content-between`, `align-items-center`, `d-none d-md-block` |
| Text / colour | `text-center`, `fs-4`, `text-body-secondary`, `bg-body-tertiary` |
| Sizing / border | `w-100`, `rounded-3`, `border`, `shadow-sm` |

### 7.3 Components and Replaced Features

| Syllabus item | Status in Bootstrap 5.3 | What to use |
|---|---|---|
| Jumbotron | Removed in v5 | `<div class="p-5 mb-4 bg-body-tertiary rounded-3">` hero using utilities |
| Glyphicons | Removed in v4 | Bootstrap Icons: `<i class="bi bi-cart"></i>` |
| Pagination | Available | `ul.pagination > li.page-item > a.page-link`; mark current with `.active` and `aria-current="page"` |
| Carousel plugin | Available | `.carousel.slide` with `data-bs-ride`, controls and indicators; add `alt` text |

### 7.4 Colour Modes, Customisation and Accessibility

Bootstrap 5.3 supports light and dark modes with `data-bs-theme="dark"` on `<html>` or any element. Components are themed by CSS variables such as `--bs-primary`; override them in your own stylesheet or compile Sass for deep customisation. Bootstrap's JavaScript components (modal, carousel, dropdown) include keyboard handling and ARIA roles; keep the required attributes (`aria-label`, `aria-current`, `aria-controls`).

> **Trade-off:** A framework speeds delivery but increases page weight. Import only what you use in real projects and measure with the browser Network tab.

## Hands-on Lab 7: Responsive Product Catalogue with Carousel, Pagination and Dark Mode

Build a Vite + Bootstrap 5.3 storefront front page with a hero section, image carousel, 24 generated product cards, JavaScript pagination (8 per page), icons and a light/dark toggle.

### Free & Open-Source Tools

- Bootstrap 5.3 + Popper (MIT)
- Bootstrap Icons (MIT)
- Vite (MIT)
- Penpot (MPL-2.0) for the wireframe

### Deliverables

- Penpot wireframe screenshot (mobile + desktop)
- Vite project in `~/webeng/week07`, build deployed to `/var/www/webeng/week07/`
- Working carousel, pagination and theme toggle

### Common Source Files (identical on both operating systems)

**`index.html`**

```
<!DOCTYPE html><html lang="en" data-bs-theme="light"><head><meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1"><title>PK
Shop</title></head><body>
<nav class="navbar navbar-expand-lg bg-body-tertiary"><div class="container">
  <a class="navbar-brand" href="#"><i class="bi bi-bag-heart"></i> PK Shop</a>
  <button class="btn btn-outline-secondary ms-auto" id="theme" aria-label="Toggle
theme"><i class="bi bi-moon-stars"></i></button>
</div></nav>
<header class="container py-4"><div class="p-5 bg-body-tertiary rounded-3">
  <h1 class="display-5 fw-bold">Eid Collection</h1><p class="lead">Free delivery in
Karachi.</p>
  <a class="btn btn-primary btn-lg" href="#grid">Shop now</a></div></header>
<div id="hero" class="carousel slide container mb-4" data-bs-ride="carousel">
  <div class="carousel-inner rounded-3"></div>
  <button class="carousel-control-prev" type="button" data-bs-target="#hero" data-bs-
slide="prev"><span class="carousel-control-prev-icon"></span><span class="visually-
hidden">Previous</span></button>
  <button class="carousel-control-next" type="button" data-bs-target="#hero" data-bs-
slide="next"><span class="carousel-control-next-icon"></span><span class="visually-
hidden">Next</span></button>
</div>
<main class="container" id="grid"><div class="row g-3" id="cards"></div>
  <nav class="my-4" aria-label="Products"><ul class="pagination justify-content-center"
id="pager"></ul></nav></main>
<script type="module" src="/main.js"></script></body></html>
```

**main.js**

```
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap-icons/font/bootstrap-icons.css';
import { Carousel } from 'bootstrap';

const products = Array.from({ length: 24 }, (_, i) => ({ id: i + 1, name: 'Product ' + (i
+ 1), price: 500 + i * 125 }));
const PER = 8; let page = 1;
const cards = document.querySelector('#cards'), pager = document.querySelector('#pager');

function render() {
  const slice = products.slice((page - 1) * PER, page * PER);
  cards.innerHTML = slice.map(p => `<div class="col-6 col-md-4 col-lg-3"><div class="card
h-100 shadow-sm">
    <div class="card-body d-flex flex-column"><h2 class="h6 card-title">${p.name}</h2>
    <p class="text-body-secondary mb-3">Rs ${p.price.toLocaleString('en-PK')}</p>
    <button class="btn btn-outline-primary mt-auto" data-id="${p.id}"><i class="bi bi-
cart-plus"></i> Add</button></div></div></div>`).join('');
  const pages = Math.ceil(products.length / PER);
  pager.innerHTML = Array.from({ length: pages }, (_, i) => i + 1).map(n =>
    `<li class="page-item ${n === page ? 'active' : ''}"><a class="page-link" href="#"
data-p="${n}" ${n === page ? 'aria-current="page"' : ''}>${n}</a></li>`).join('');
}
pager.addEventListener('click', e => { const a = e.target.closest('[data-p]'); if (!a)
return; e.preventDefault(); page = +a.dataset.p; render(); });

const slides = ['#0f766e', '#7c3aed', '#b45309'].map((c, i) => `<div class="carousel-item
${i ? '' : 'active'}">
  <svg class="d-block w-100" viewBox="0 0 800 260" role="img" aria-label="Offer ${i +
1}"><rect width="800" height="260" fill="${c}"/>
  <text x="400" y="145" fill="#fff" font-size="42" text-anchor="middle">Offer ${i +
1}</text></svg></div>`).join('');
document.querySelector('.carousel-inner').innerHTML = slides;
new Carousel('#hero', { interval: 4000 });

document.querySelector('#theme').addEventListener('click', () => {
  const h = document.documentElement; h.dataset.bsTheme = h.dataset.bsTheme === 'dark' ?
'light' : 'dark';
});
render();
```

## Version A – Rocky Linux 10

**Step 1 – Wireframe first in Penpot.** Open penpot.app (free) or your self-hosted instance and sketch a mobile and a desktop frame with hero, grid and pager.

```
firefox https://design.penpot.app &
```

**Step 2 – Create the project.** Install Bootstrap with its Popper peer dependency and the icon font.

```
mkdir -p ~/webeng/week07 && cd ~/webeng/week07
npm init -y && npm pkg set type=module
npm install bootstrap @popperjs/core bootstrap-icons && npm install --save-dev vite
kate index.html main.js &
npx vite --host 127.0.0.1
```

**Step 3 – Build and deploy.** Icon font files are bundled into dist/assets automatically.

```
npx vite build --base=/week07/
rsync -a --delete dist/ /var/www/webeng/week07/
sudo restorecon -Rv /var/www/webeng/week07
```

**Step 4 – Measure page weight.** Compare uncompressed and gzip transfer; this is the framework trade-off.

```
du -sh dist; ls -lh dist/assets | sort -k5 -h | tail -n 4
curl -s -H 'Accept-Encoding: gzip' -o /dev/null -w "HTML: %{size_download} bytes\n"
http://webeng.test/week07/
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Wireframe first in Penpot.** Open penpot.app (free) or your self-hosted instance and sketch a mobile and a desktop frame with hero, grid and pager.

```
firefox https://design.penpot.app &
```

**Step 2 – Create the project.** Install Bootstrap with its Popper peer dependency and the icon font.

```
mkdir -p ~/webeng/week07 && cd ~/webeng/week07
npm init -y && npm pkg set type=module
npm install bootstrap @popperjs/core bootstrap-icons && npm install --save-dev vite
kate index.html main.js &
npx vite --host 127.0.0.1
```

**Step 3 – Build and deploy.** Icon font files are bundled into dist/assets automatically.

```
npx vite build --base=/week07/
rsync -a --delete dist/ /var/www/webeng/week07/
```

**Step 4 – Measure page weight.** Compare uncompressed and gzip transfer; this is the framework trade-off.

```
du -sh dist; ls -lh dist/assets | sort -k5 -h | tail -n 4
curl -s -H 'Accept-Encoding: gzip' -o /dev/null -w "HTML: %{size_download} bytes\n"
http://webeng.test/week07/
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | Resize to 360 / 800 / 1200 px | 2 / 3 / 4 cards per row |
| 2 | Pagination | Pages 1–3 each show 8 products; active page highlighted and has `aria-current` |
| 3 | Carousel | Auto-advances every 4 s; arrow buttons and keyboard work |
| 4 | Theme toggle | All components switch between light and dark |
| 5 | Penpot file | Wireframe matches the implemented layout |

## Stretch Goals (Industry Practice)

- Add a Bootstrap modal showing product details.
- Use `@use` Sass customisation to change `$primary` and rebuild.
- Run a Lighthouse accessibility audit in Firefox/Chromium DevTools and fix findings.

# Review Questions

1. Write the classes for a card that is full width on phones, half on tablets and one-third on desktops.
2. Why was the Jumbotron removed and how do you reproduce it in Bootstrap 5?
3. Which UX law explains why “Add to cart” buttons are large and near the product?
4. How does `data-bs-theme` implement dark mode without a second stylesheet?
5. List two accessibility attributes required for the carousel and pagination.

# Week 8: Mid-Term Examination (Weeks 1–7)

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 15–16 | **CLO** | CLO_1, CLO_2 |
| **PLO** | PLO_2, PLO_5 | **Bloom's Level** | C2–C3 |
| **Reading** | All material of Weeks 1–7 | **Assessment** | Mid-Term: Objective 10 (CLO_1) + Subjective/Practical 10 (CLO_2) |

## Learning Outcomes

- Demonstrate understanding of web fundamentals, HTML5, CSS3, JavaScript, Node.js/Angular and Bootstrap.
- Deliver a working, validated, responsive web page in a controlled Linux exam environment.
- Use automated checks to verify requirements (functional, validity, deployment).

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:20 | 20 | Briefing & environment check | Exam rules, run `check.sh --env`, fix login/network problems |
| 0:20 – 0:50 | 30 | Objective paper (MCQ) | 10 MCQs × 1 mark, CLO_1, closed book |
| 0:50 – 2:20 | 90 | Practical / subjective paper | Build and deploy a mini site, open web docs (MDN) only |
| 2:20 – 2:40 | 20 | Auto-check & submission | Run `check.sh`, save the report, push to Git, submit |
| 2:40 – 3:00 | 20 | Post-exam discussion | Model solution walkthrough, common mistakes |

## Lecture Notes

### 8.1 Examination Blueprint

| Part | CLO | Marks | Time | Coverage |
|---|---|---|---|---|
| Objective (MCQ) | CLO_1 | 10 | 30 min | Internet, HTTP, DNS, URL, architectures, HTML5 semantics, forms |
| Subjective / practical | CLO_2 | 10 | 90 min | HTML5 + CSS3 responsive page + JavaScript behaviour + deployment |

### 8.2 Revision Map

| Week | Must-know items |
|---|---|
| 1 | URL parts, HTTP methods / status codes, 1/2/3-tier vs MVC, HTML5 tags |
| 2 | Semantic elements, input types, validation attributes, GET vs POST, accessibility basics |
| 3 | Specificity, box model, Flexbox vs Grid, media / container queries, animation |
| 4 | `let/const`, hoisting, closures, array methods, destructuring, modules |
| 5 | Classes, inheritance, async/await, regex, CSV, canvas |
| 6 | Event loop, EventEmitter, callbacks vs promises, Angular signals / services |
| 7 | Grid classes, components, pagination, carousel, Jumbotron / Glyphicon replacements |

## 8.3 Sample Objective Questions (with answer key)

| # | Question | Options | Key |
|---|---|---|---|
| 1 | Which part of a URL is never sent to the server? | A scheme B query C fragment D port | C |
| 2 | HTTP status for “resource moved permanently”? | A 200 B 301 C 404 D 500 | B |
| 3 | MVC is best described as… | A a network topology B a code-organisation pattern C a database type D a protocol | B |
| 4 | Which element wraps the dominant page content? | A `section` B `div` C `main` D `article` | C |
| 5 | Which attribute makes a field mandatory? | A `needed` B `required` C `validate` D `must` | B |
| 6 | Which CSS feature lets a component respond to its parent’s width? | A media query B container query C `@import` D `float` | B |
| 7 | Result of `typeof null`? | A "null" B "undefined" C "object" D "number" | C |
| 8 | Which keyword declares a block-scoped, reassignable variable? | A var B const C let D static | C |
| 9 | Node.js handles I/O concurrency mainly through… | A threads per request B the event loop C CGI D forking | B |
| 10 | Which Bootstrap class gives 3 columns per row on large screens? | A `col-lg-3` B `col-lg-4` C `col-3` D `col-md-12` | B |

## 8.4 Sample Practical Question (10 marks, CLO_2)

Create a one-page site “Library Corner” in `/var/www/webeng/mid/<rollno>/` with: (a) semantic header, nav, main, footer **[2]**; (b) a search form with `required` input and a number input for “max price” **[1]**; (c) a responsive CSS Grid of at least six book cards that becomes a single column below 600 px, with a hover effect **[3]**; (d) JavaScript that filters the cards by title as the user types and shows the visible count **[3]**; (e) no HTML validation errors and no console errors **[1]**.

| Criterion | Marks | Evidence |
|---|---|---|
| Semantic structure | 2 | header / nav / main / footer present, one `h1` |
| Form attributes | 1 | `required`, `type="number"`, `min`, labels |
| Responsive grid | 3 | Grid, media query, hover, no overflow at 360 px |
| JavaScript filter + count | 3 | Works while typing; textContent used, not innerHTML |
| Validity & cleanliness | 1 | `html-validate` exit code 0; no console errors |

# Hands-on Lab 8: Mid-Term Practical Environment and Automatic Checker

Prepare per-student exam folders with correct permissions, deploy the exam site and verify it with a shell-based auto-checker. The same script is used by the instructor to pre-test the model solution.

## Free & Open-Source Tools

- Bash, curl, grep, diff (GPL)
- html-validate (MIT)
- Git (GPL-2.0)

## Deliverables

- Exam folder `/var/www/webeng/mid/<rollno>/` with `index.html`, `style.css`, `app.js`
- `check.sh` report saved as `report.txt`

- Git commit of the final submission

## Common Source Files (identical on both operating systems)

**check.sh**

```
#!/usr/bin/env bash
# usage: ./check.sh <rollno>      (run from ~/webeng/mid-check, project must have html-validate installed)
set -u
ROLL=${1:?rollno required}; DIR=/var/www/webeng/mid/$ROLL;
URL=http://webeng.test/mid/$ROLL/
score=0; ok(){ echo "PASS  $1"; score=$((score+$2)); }; bad(){ echo "FAIL  $1"; }
[ "$(curl -s -o /dev/null -w '%{http_code}' $URL)" = 200 ] && ok "page served (HTTP 200)" 0 || bad "page not served"
H=$(curl -s $URL)
for t in header nav main footer; do grep -qi "<$t" <<<"$H" && ok "<$t> present" 0 || bad "<$t> missing"; done
[ "$(grep -io '<h1' <<<"$H" | wc -l)" = 1 ] && ok "exactly one h1" 0 || bad "h1 count"
grep -qi 'required' <<<"$H" && grep -qi 'type="number"' <<<"$H" && ok "form attributes" 1 || bad "form attributes"
grep -Eq 'display: *grid|display:grid' $DIR/*.css && ok "CSS grid used" 1 || bad "no grid"
grep -Eq '@media' $DIR/*.css && ok "media query used" 1 || bad "no media query"
grep -Eq 'addEventListener' $DIR/*.js && ok "JS event listener" 1 || bad "no JS listener"
grep -Eq 'innerHTML' $DIR/*.js && bad "innerHTML used (XSS risk)" || ok "no innerHTML" 1
(cd ~/webeng/mid-check && npx html-validate "$DIR/index.html" >/dev/null 2>&1) && ok "html-validate clean" 1 || bad "html-validate errors"
echo "Automatic score: $score (instructor adds manual marks)"
```

## Version A – Rocky Linux 10

**Step 1 – Create exam folders and permissions.** Each student owns a folder; Apache can read it. New files inherit the SELinux type from the parent directory.

```
sudo mkdir -p /var/www/webeng/mid && sudo chown $USER:apache /var/www/webeng/mid
sudo semanage fcontext -a -t httpd_sys_content_t "/var/www/webeng/mid(/.*)?"
for r in 24-001 24-002; do mkdir -p /var/www/webeng/mid/$r; done
sudo restorecon -Rv /var/www/webeng/mid
chmod -R g+rX /var/www/webeng/mid
```

**Step 2 – Install the checker.** html-validate is installed once in a helper project.

```
mkdir -p ~/webeng/mid-check && cd ~/webeng/mid-check
npm init -y && npm install --save-dev html-validate
echo '{ "extends": ["html-validate:recommended"] }' > .htmlvalidate.json
kate check.sh && chmod +x check.sh
```

**Step 3 – Pre-test with the model solution.** Copy the model answer into a test roll number and run the checker.

```
cp -r ~/webeng/model/* /var/www/webeng/mid/24-001/
sudo restorecon -Rv /var/www/webeng/mid/24-001
./check.sh 24-001 | tee /var/www/webeng/mid/24-001/report.txt
```

**Step 4 – Collect submissions.** Each student commits in their folder; the instructor archives all folders.

```
cd /var/www/webeng/mid/24-001 && git init -q && git add . && git commit -qm "Mid-term submission 24-001"
tar czf ~/mid-submissions.tgz -C /var/www/webeng mid && sha256sum ~/mid-submissions.tgz
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Create exam folders and permissions.** Each student owns a folder; www-data can read it.

```
sudo mkdir -p /var/www/webeng/mid && sudo chown $USER:www-data /var/www/webeng/mid
for r in 24-001 24-002; do mkdir -p /var/www/webeng/mid/$r; done
chmod -R g+rX /var/www/webeng/mid
```

**Step 2 – Install the checker.** html-validate is installed once in a helper project.

```
mkdir -p ~/webeng/mid-check && cd ~/webeng/mid-check
npm init -y && npm install --save-dev html-validate
echo '{ "extends": ["html-validate:recommended"] }' > .htmlvalidate.json
kate check.sh && chmod +x check.sh
```

**Step 3 – Pre-test with the model solution.** Copy the model answer into a test roll number and run the checker.

```
cp -r ~/webeng/model/* /var/www/webeng/mid/24-001/
./check.sh 24-001 | tee /var/www/webeng/mid/24-001/report.txt
```

**Step 4 – Collect submissions.** Each student commits in their folder; the instructor archives all folders.

```
cd /var/www/webeng/mid/24-001 && git init -q && git add . && git commit -qm "Mid-term
submission 24-001"
tar czf ~/mid-submissions.tgz -C /var/www/webeng mid && sha256sum ~/mid-submissions.tgz
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `./check.sh 24-001` with the model solution | All automatic checks PASS |
| 2 | Empty folder | Page not served; checks FAIL gracefully without script errors |
| 3 | `git log --oneline` | At least one commit with the roll number |
| 4 | `sha256sum ~/mid-submissions.tgz` | Hash recorded for integrity |

## Stretch Goals (Industry Practice)

- Add a Playwright test that types in the filter box and counts visible cards.
- Generate an HTML report from check.sh output.
- Run the checker inside a Podman container for a clean, repeatable environment.

# Review Questions

1. Which CLO does each part of the mid-term assess and why?
2. Explain MVC versus three-tier in three sentences.
3. What is the difference between a block-scoped and a function-scoped variable?
4. Which tools in the practical enforce quality automatically?
5. Name two mistakes from this exam you will avoid in the final.

# Week 9: PHP Fundamentals and the Server-Side Stack

| Syllabus Lectures | Lec 17–18 | CLO | CLO_2 |
|---|---|---|---|
| PLO | PLO_5 | Bloom's Level | C3 (Applying) |
| Reading | Ch. 11 Introduction to PHP, PHP Basics | Assessment | Q2 (formative) |

## Learning Outcomes

- Explain the PHP request lifecycle and the Apache + PHP-FPM architecture.
- Write PHP using variables, types, strings, operators, `if`/`else`, `switch`, `match` and nested switch.
- Use indexed, associative, 2D and 3D arrays with `for`, `while`, `do-while` and `foreach`.
- Write typed functions with `declare(strict_types=1)` and escape output safely.
- Install PHP-FPM, Composer and PHPStan and run static analysis.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Quiz Q2 (15 min, formative) on Weeks 4–7, then recap of CSS/JS |
| 0:10 – 0:45 | 35 | Concept Block 1 | How PHP runs, syntax, types, strings, operators, control flow |
| 0:45 – 1:20 | 35 | Concept Block 2 | Arrays (1D/2D/3D), loops, functions, superglobals, errors, tooling |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 9 – PHP-FPM stack, grades and matrices |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 9.1 How PHP Runs

PHP is a server-side language: the browser never sees PHP code, only its output. In current production setups Apache (or Nginx) passes `.php` requests to **PHP-FPM**, a pool of worker processes, over FastCGI (a Unix socket). **OPcache** stores compiled bytecode so files are not re-parsed on each request. PHP is stateless between requests; state is kept in sessions, cookies or the database.

| Version | Notable features |
|---|---|
| 8.0 – 8.2 | Named arguments, `match`, union types, enums, readonly properties, constructor promotion |
| 8.3 (Rocky 10) | Typed class constants, `#[\Override]`, `json_validate()` |
| 8.4 | Property hooks, asymmetric visibility, `array_find()`, `array_any()`, `array_all()` |
| 8.5 (Kubuntu 26.04) | Pipe operator `\|>`, `array_first()` / `array_last()`, built-in URI extension |

### 9.2 Syntax, Types and Strings

```
<?php
declare(strict_types=1);                    // no silent type juggling for scalar arguments
$name = 'Ayesha'; $age = 21; $gpa = 3.45; $ok = true; $none = null;
echo "Hello, {$name}! Next year: " . ($age + 1) . PHP_EOL;     // double quotes
interpolate
echo 'Single quotes do not $interpolate', PHP_EOL;
$text = <<<TXT
Heredoc keeps formatting for {$name}
```

```
TXT;
var_dump(strlen($name), strtoupper($name), str_contains($name, 'ye'), substr($name, 0,
3));
printf("%s has %.1f GPA (%05d)\n", $name, $gpa, 42);
```

| Operator group | Examples |
|---|---|
| Arithmetic | + - * / % **, integer division intdiv() |
| Comparison | == (loose) vs === (strict), <=> spaceship |
| Logical | && \|\| ! xor, and low-precedence and, or |
| Null handling | ?? (null coalescing), ??=, ?-> (null-safe) |
| String | . concatenation, .= |

## 9.3 Control Structures

```
$score = 72;
if ($score >= 85) { $g = 'A'; } elseif ($score >= 70) { $g = 'B'; } else { $g = 'C'; }
$g2 = match (true) { $score >= 85 => 'A', $score >= 70 => 'B', default => 'C' };   //
expression, strict
switch ($day) {                                    // nested switch
    case 'Mon': switch ($slot) { case 'lab': echo 'Lab A'; break; default: echo 'Hall'; }
break;
    case 'Tue': case 'Wed': echo 'Theory'; break;
    default: echo 'Free';
}
for ($i = 1; $i <= 3; $i++) echo $i;      $n = 3; while ($n > 0) echo $n--;
do { echo 'runs once'; } while (false);
```

## 9.4 Arrays: 1D, 2D and 3D

```
$langs = ['PHP', 'JS'];                            // indexed
$user  = ['name' => 'Ali', 'city' => 'Karachi'];     // associative
$grid  = [[1, 2, 3], [4, 5, 6]];                    // 2D: $grid[1][2] = 6
$cube  = ['Fall' => ['Web' => [8, 9], 'DB' => [7, 6]]];   // 3D: $cube['Fall']['Web'][0]
foreach ($cube as $term => $courses)
    foreach ($courses as $course => $marks)
        echo "$term / $course: " . array_sum($marks) / count($marks) . PHP_EOL;
array_map(fn($x) => $x * 2, [1, 2]); array_filter($langs, fn($l) => $l !== 'JS');
usort($rows, fn($a, $b) => $a['m'] <=> $b['m']);
[$a, $b] = [10, 20]; ['name' => $n] = $user;           // destructuring
```

## 9.5 Functions, Superglobals and Output Escaping

Functions should declare parameter and return types. Input arrives in $_GET, $_POST, $_COOKIE, $_FILES, $_SERVER and $_SESSION. Always treat input as untrusted: validate with filter_var() / filter_input() and **escape on output** with htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8") to prevent cross-site scripting.

> **Security:** Set display_errors = Off and log_errors = On in production; errors shown to users leak paths and queries. During development you may enable them.

# Hands-on Lab 9: PHP-FPM Stack, Grade Report and Matrix Exercises with PHPStan

Install Apache + PHP-FPM, Composer and PHPStan; write typed PHP functions, a grade report page and CLI exercises that use match, nested switch, loops and 2D/3D arrays.

## Free & Open-Source Tools

- PHP 8.x with PHP-FPM (PHP licence)
- Composer (MIT)
- PHPStan (MIT)
- Apache `mod_proxy_fcgi`

## Deliverables

- Project `/var/www/webeng/week09` with `public/`, `src/`, `bin/`
- Web page `http://webeng.test/week09/public/` showing the grade table
- `php bin/matrix.php` output for 2D and 3D arrays
- PHPStan level 6 with 0 errors

## Common Source Files (identical on both operating systems)

**src/grades.php**

```
<?php
declare(strict_types=1);

function letterGrade(float $pct): string {
    return match (true) {
        $pct >= 85 => 'A',
        $pct >= 70 => 'B',
        $pct >= 55 => 'C',
        $pct >= 40 => 'D',
        default => 'F',
    };
}
/** @param list<int|float> $marks */
function average(array $marks): float {
    return $marks === [] ? 0.0 : array_sum($marks) / count($marks);
}
function e(string $s): string {
    return htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

**public/index.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/grades.php';

$students = [
    ['name' => 'Ayesha <b>Khan</b>', 'marks' => [88, 92, 79]],
    ['name' => 'Bilal Siddiqui',     'marks' => [64, 58, 71]],
    ['name' => 'Hira Ali',           'marks' => [35, 42, 39]],
];
$pass = filter_input(INPUT_GET, 'pass', FILTER_VALIDATE_INT, ['options' => ['min_range' =>
0, 'max_range' => 100]]);
if (!is_int($pass)) { $pass = 40; }
?>
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8"><title>Grades</title></head>
<body><main><h1>Grade report (pass mark <?= $pass ?>)</h1>
<table
border="1"><thead><tr><th>Name</th><th>Average</th><th>Grade</th><th>Status</th></tr></the
ad><tbody>
<?php foreach ($students as $s): $avg = average($s['marks']); ?>
  <tr><td><?= e($s['name']) ?></td><td><?= number_format($avg, 1) ?></td>
      <td><?= letterGrade($avg) ?></td><td><?= $avg >= $pass ? 'Pass' :
'Fail' ?></td></tr>
```

```
<?php endforeach; ?>
</tbody></table></main></body></html>
```

**bin/matrix.php**

```
<?php
declare(strict_types=1);
$timetable = [
    'Mon' => ['08:30' => 'Web Engineering', '10:00' => 'Databases'],
    'Tue' => ['08:30' => 'Web Engineering'],
    'Wed' => ['08:30' => 'Web Engineering', '14:30' => 'Web Lab'],
];
foreach ($timetable as $day => $slots) {                  // 2D associative
    foreach ($slots as $time => $course) { printf("%-4s %s  %s\n", $day, $time,
$course); }
}
$marks = ['Fall' => ['Web' => [8, 9, 10], 'DB' => [7, 6, 9]],        // 3D: term / course
/ quizzes
          'Spring' => ['Net' => [9, 9, 8]]];
foreach ($marks as $term => $courses) {
    foreach ($courses as $course => $quizzes) {
        $i = 0; $sum = 0;
        do { $sum += $quizzes[$i]; $i++; } while ($i < count($quizzes));    // do-while
        printf("%s/%s average %.2f\n", $term, $course, $sum / count($quizzes));
    }
}
$n = 5; $fact = 1; while ($n > 1) { $fact *= $n--; } echo "5! = $fact", PHP_EOL;
foreach ([1, 3, 5] as $slot) {
    switch ($slot) { case 1: echo "first\n"; break; case 3: switch (true) { case $slot >
2: echo "third+\n"; break; } break; default: echo "other\n"; }
}
```

**.htaccess (in /var/www/webeng/week09)**

```
# block direct web access to non-public folders (needs AllowOverride All from Lab 0)
RedirectMatch 404 "/(vendor|src|config|storage|\.git)(/|$)"
```

## Version A – Rocky Linux 10

**Step 1 – Install PHP-FPM and extensions.** Rocky 10 provides PHP 8.3 in AppStream; extensions are separate packages.

```
sudo dnf -y install php-fpm php-cli php-mysqlnd php-mbstring php-xml php-gd php-intl php-
opcache
sudo systemctl enable --now php-fpm
php -v && php -m | grep -E 'mbstring|pdo_mysql|gd|intl|fileinfo'
```

**Step 2 – Connect Apache to PHP-FPM.** Apache talks to the FPM socket created by the pool (/run/php-fpm/www.sock).

```
sudo tee /etc/httpd/conf.d/webeng-php.conf >/dev/null <<'EOF'
<FilesMatch \.php$>
    SetHandler "proxy:unix:/run/php-fpm/www.sock|fcgi://localhost"
</FilesMatch>
DirectoryIndex index.php index.html
EOF
sudo apachectl configtest && sudo systemctl restart httpd
ps -eZ | grep php-fpm | head -n 2         # domain should be httpd_t
```

**Step 3 – Install Composer (verified installer).** The installer signature is checked before running it.

```
cd /tmp
php -r "copy('https://getcomposer.org/installer','composer-setup.php');"
```

```
php -r "exit(hash_file('sha384','composer-setup.php') ===
trim(file_get_contents('https://composer.github.io/installer.sig')) ? 0 : 1);" && echo
VERIFIED
sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer && rm
composer-setup.php
composer --version
```

**Step 4 – Create the project and files.** Create the four files from the listings; then relabel and install PHPStan.

```
mkdir -p /var/www/webeng/week09/{public,src,bin} && cd /var/www/webeng/week09
kate src/grades.php public/index.php bin/matrix.php .htaccess &
composer init --no-interaction --name=student/week09 && composer require --dev
phpstan/phpstan
sudo restorecon -Rv /var/www/webeng/week09
```

**Step 5 – Run, test and analyse.** CLI and web must give the same logic results.

```
php bin/matrix.php
curl -s "http://webeng.test/week09/public/?pass=60" | grep -E 'Pass|Fail'
curl -s -o /dev/null -w "%{http_code}\n" http://webeng.test/week09/src/grades.php      #
expect 404
vendor/bin/phpstan analyse -l 6 src public bin
```

**Step 6 – Read the logs.** PHP errors go to the FPM log; web errors to the Apache log.

```
sudo tail -n 5 /var/log/php-fpm/www-error.log /var/log/httpd/error_log
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Install PHP-FPM and extensions.** Kubuntu 26.04 provides PHP 8.5. Do not install the `php` metapackage because it pulls mod_php.

```
sudo apt -y install php-fpm php-cli php-mysql php-mbstring php-xml php-gd php-intl php-
curl
PHPV=$(php -r 'echo PHP_MAJOR_VERSION.".".PHP_MINOR_VERSION;'); echo $PHPV
sudo systemctl enable --now php${PHPV}-fpm
php -v && php -m | grep -E 'mbstring|pdo_mysql|gd|intl|fileinfo'
```

**Step 2 – Connect Apache to PHP-FPM.** proxy_fcgi plus the packaged configuration snippet.

```
sudo a2enmod proxy_fcgi setenvif
sudo a2enconf php${PHPV}-fpm
echo 'DirectoryIndex index.php index.html' | sudo tee /etc/apache2/conf-available/webeng-
index.conf && sudo a2enconf webeng-index
sudo apache2ctl configtest && sudo systemctl reload apache2
ls /run/php/                            # php8.5-fpm.sock
```

**Step 3 – Install Composer (verified installer).** The installer signature is checked before running it.

```
cd /tmp
php -r "copy('https://getcomposer.org/installer','composer-setup.php');"
php -r "exit(hash_file('sha384','composer-setup.php') ===
trim(file_get_contents('https://composer.github.io/installer.sig')) ? 0 : 1);" && echo
VERIFIED
sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer && rm
composer-setup.php
composer --version
```

**Step 4 – Create the project and files.** Create the four files from the listings; then install PHPStan.

```
mkdir -p /var/www/webeng/week09/{public,src,bin} && cd /var/www/webeng/week09
kate src/grades.php public/index.php bin/matrix.php .htaccess &
composer init --no-interaction --name=student/week09 && composer require --dev
phpstan/phpstan
```

**Step 5 – Run, test and analyse.** CLI and web must give the same logic results.

```
php bin/matrix.php
curl -s "http://webeng.test/week09/public/?pass=60" | grep -E 'Pass|Fail'
curl -s -o /dev/null -w "%{http_code}\n" http://webeng.test/week09/src/grades.php      #
expect 404
vendor/bin/phpstan analyse -l 6 src public bin
```

**Step 6 – Read the logs.** PHP-FPM logs to /var/log; Apache to its own log.

```
sudo tail -n 5 /var/log/php${PHPV}-fpm.log /var/log/apache2/error.log
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `php bin/matrix.php` | Timetable lines, three term/course averages, `5! = 120`, nested switch output |
| 2 | `curl .../public/?pass=60` | Ayesha Pass, Bilal Pass or Fail by average, Hira Fail; name shown escaped (`<b>`) |
| 3 | `curl .../src/grades.php` | 404 (protected by .htaccess) |
| 4 | `vendor/bin/phpstan analyse -l 6 …` | `[OK] No errors` |
| 5 | Rocky: `ps -eZ \| grep php-fpm` | Process runs in `httpd_t` (confined) |

## Stretch Goals (Industry Practice)

- Kubuntu (PHP 8.4+): replace a `foreach` search with `array_find()`.
- Raise PHPStan to level 8 and fix findings.
- Add a PHPUnit test for `letterGrade()` boundaries (84.9, 85).

# Review Questions

1. Describe the path of a request from the browser to PHP-FPM and back.
2. Why is `match` safer than `switch` for returning values?
3. Show how to read the mark of the second course of the first term in the 3D array above.
4. What does `declare(strict_types=1)` change? Give a failing example.
5. Why must output be escaped even if input was validated?

# Week 10: SQL, MariaDB, PDO, CAPTCHA and Secure File Upload

| Syllabus Lectures | Lec 19–20 | CLO | CLO_3 |
|---|---|---|---|
| PLO | PLO_4 | Bloom's Level | C4 (Analyzing) |
| Reading | Ch. 11–13 PHP and MySQL | Assessment | — |

## Learning Outcomes

- Design a normalised relational schema with keys, constraints and indexes.
- Write DDL (CREATE, ALTER, DROP) and DML (INSERT, UPDATE, DELETE, SELECT with JOIN / GROUP BY).
- Connect from PHP with PDO and prevent SQL injection using prepared statements.
- Implement and evaluate a CAPTCHA and compare it with reCAPTCHA, Turnstile and ALTCHA.
- Receive uploaded files safely and apply OS-level security (SELinux labels, permissions).

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | Relational model, normalisation, DDL, DML, joins, transactions |
| 0:45 – 1:20 | 35 | Concept Block 2 | PDO and prepared statements, SQL injection, CAPTCHA, secure upload |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 10 – Secure registration with CAPTCHA and avatar |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 10.1 Relational Concepts

A relational database stores data in tables of rows (records) and columns (attributes). A **primary key** uniquely identifies a row; a **foreign key** references another table and enforces referential integrity. **MariaDB** is the community fork of MySQL; it uses the InnoDB engine (transactions, row locks, foreign keys) and is the default open-source server on both distributions. Use `utf8mb4` for full Unicode (including Urdu and emoji).

| Normal form | Rule | Fix example |
|---|---|---|
| 1NF | Atomic values, no repeating groups | Split "phone1, phone2" into a `phones` table |
| 2NF | No partial dependency on part of a composite key | Move product name out of `order_items` |
| 3NF | No transitive dependency (non-key → non-key) | Move city / province to a `cities` table |

### 10.2 DDL and DML

```
CREATE TABLE demo (id INT PRIMARY KEY AUTO_INCREMENT, title VARCHAR(50) NOT NULL);
ALTER TABLE demo ADD COLUMN created_at DATETIME DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE demo MODIFY title VARCHAR(100) NOT NULL;
DROP TABLE demo;

INSERT INTO categories (name) VALUES ('Toys');
```

```
UPDATE products SET price = price * 1.10 WHERE category_id = 2;
DELETE FROM products WHERE stock = 0;

SELECT c.name AS category, COUNT(*) AS items, ROUND(AVG(p.price),2) AS avg_price
FROM products p JOIN categories c ON c.id = p.category_id
GROUP BY c.id HAVING items > 1 ORDER BY avg_price DESC LIMIT 5;
```

Transactions group statements: `START TRANSACTION; … COMMIT;` or `ROLLBACK;` give the ACID guarantees (atomicity, consistency, isolation, durability). Never run application code as the database `root`; grant the application user only the privileges it needs.

## 10.3 PDO and SQL Injection

```
// VULNERABLE: input becomes part of the SQL text
$sql = "SELECT * FROM users WHERE email = '$email'";        // email = ' OR '1'='1  →
returns every user

// SAFE: SQL and data travel separately
$st = db()->prepare('SELECT id, name FROM users WHERE email = :e');
$st->execute([':e' => $email]);
$user = $st->fetch();                                       // false when not found
```

> **Rule:** Prepared statements protect values, not identifiers. For dynamic column names or `ORDER BY` use an allow-list array.

## 10.4 CAPTCHA

A CAPTCHA distinguishes humans from bots, mainly on registration, login and contact forms. Always verify on the **server**; make the challenge single-use and time-limited.

| Solution | Open source? | Notes |
|---|---|---|
| Image CAPTCHA (this lab) | Yes (own code, GD) | Weak against modern OCR; teaches the mechanism |
| Google reCAPTCHA v2/v3 | No (free service) | Syllabus topic; third-party tracking, needs keys |
| Cloudflare Turnstile | No (free service) | Privacy-focused, invisible challenge |
| hCaptcha | No (free tier) | Alternative to reCAPTCHA |
| ALTCHA | Yes (MIT) | Self-hosted proof-of-work, no tracking |

In production combine a CAPTCHA with rate limiting, e-mail confirmation and a honeypot field.

## 10.5 Secure File Uploads

- Check `$_FILES[..]["error"]` first, then size.
- Verify the real type with `finfo` / `getimagesize()`; never trust the client-supplied name or MIME type.
- Generate a random file name; never use the original name or extension.
- Store uploads **outside** the web root and serve them through a PHP script, or disable script execution in the upload folder.
- Limit sizes in `php.ini` (`upload_max_filesize`, `post_max_size`) and in the application.

# Hands-on Lab 10: MariaDB Schema, PDO Connection, CAPTCHA Registration and Avatar Upload

Install and secure MariaDB, create the `shop` schema, practise SQL, then build `register.php` with an image CAPTCHA, password hashing, validation and a safe avatar upload served by a PHP script.

## Free & Open-Source Tools

- MariaDB Server 10.11 / 11.8 (GPL-2.0)
- PHP PDO + GD
- DBeaver Community (Apache-2.0) as GUI client
- SELinux tools (Rocky) / ls, chgrp (Kubuntu)

## Deliverables

- Database `shop` with 5 tables and sample data; SQL practice file with 8 queries
- Working `register.php` with CAPTCHA, validation, hashing and avatar
- Avatar stored outside web root and served by `avatar.php`
- Proof that SQL injection fails (screenshot or curl output)

## Common Source Files (identical on both operating systems)

**week10/schema.sql**

```
CREATE TABLE users (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(80) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  avatar VARCHAR(64) NULL,
  role ENUM('customer','admin') NOT NULL DEFAULT 'customer',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
CREATE TABLE categories (
  id TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB;
CREATE TABLE products (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  category_id TINYINT UNSIGNED NOT NULL,
  name VARCHAR(120) NOT NULL,
  description TEXT NULL,
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  stock INT UNSIGNED NOT NULL DEFAULT 0,
  image VARCHAR(64) NULL,
  CONSTRAINT fk_prod_cat FOREIGN KEY (category_id) REFERENCES categories(id),
  INDEX idx_prod_cat (category_id)
) ENGINE=InnoDB;
CREATE TABLE orders (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id INT UNSIGNED NOT NULL,
  total DECIMAL(10,2) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_ord_user FOREIGN KEY (user_id) REFERENCES users(id)
) ENGINE=InnoDB;
CREATE TABLE order_items (
  order_id INT UNSIGNED NOT NULL,
  product_id INT UNSIGNED NOT NULL,
  qty INT UNSIGNED NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id),
```

```
  CONSTRAINT fk_oi_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
  CONSTRAINT fk_oi_prod FOREIGN KEY (product_id) REFERENCES products(id)
) ENGINE=InnoDB;
INSERT INTO categories (name) VALUES
('Electronics'),('Books'),('Clothing'),('Stationery');
INSERT INTO products (category_id,name,description,price,stock) VALUES
 (1,'USB-C Cable','1 m braided cable',650.00,50),(1,'Wireless Mouse','2.4 GHz
optical',1850.00,25),
 (1,'Power Bank 10000','Dual USB output',4200.00,15),(2,'PHP and MySQL Guide','Dynamic
sites',3200.00,10),
 (2,'Learning Web Design','HTML, CSS, JS',2800.00,12),(3,'Hamdard T-Shirt','Cotton, size
M',1200.00,40),
 (4,'A4 Notebook','200 pages',350.00,100),(4,'Gel Pen Pack','Pack of 10',280.00,80);
```

**week10/config/db.php**

```
<?php
return ['dsn' => 'mysql:host=localhost;dbname=shop;charset=utf8mb4', 'user' => 'shop_app',
'pass' => 'ChangeMe_2026!'];
```

**week10/src/db.php**

```
<?php
declare(strict_types=1);
function db(): PDO {
    static $pdo = null;
    if ($pdo === null) {
        $c = require __DIR__ . '/../config/db.php';
        $pdo = new PDO($c['dsn'], $c['user'], $c['pass'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    }
    return $pdo;
}
function e(string $s): string { return htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE,
'UTF-8'); }
function saveAvatar(array $f, array &$errors): ?string {
    if ($f['error'] !== UPLOAD_ERR_OK) { $errors[] = 'Upload failed (code ' .
$f['error'] . ').'; return null; }
    if ($f['size'] > 2 * 1024 * 1024) { $errors[] = 'Image must be 2 MB or smaller.';
return null; }
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);
    $ext = ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/webp' => 'webp'][$mime] ??
null;
    if ($ext === null || @getimagesize($f['tmp_name']) === false) { $errors[] = 'Only
JPEG, PNG or WebP images.'; return null; }
    $name = bin2hex(random_bytes(16)) . '.' . $ext;
    if (!move_uploaded_file($f['tmp_name'], __DIR__ . '/../storage/avatars/' . $name))
{ $errors[] = 'Could not save file.'; return null; }
    return $name;
}
```

**week10/public/captcha.php**

```
<?php
declare(strict_types=1);
session_start();
$chars = 'ABCDEFGHJKMNPQRSTUVWXYZ23456789';           // no 0/O/1/I confusion
$code = '';
for ($i = 0; $i < 5; $i++) { $code .= $chars[random_int(0, strlen($chars) - 1)]; }
$_SESSION['captcha'] = $code;
$im = imagecreatetruecolor(90, 28);
imagefilledrectangle($im, 0, 0, 90, 28, imagecolorallocate($im, 245, 247, 250));
for ($i = 0; $i < 5; $i++) {
```

```
    $c = imagecolorallocate($im, random_int(120, 200), random_int(120, 200),
random_int(120, 200));
    imageline($im, random_int(0, 90), random_int(0, 28), random_int(0, 90), random_int(0,
28), $c);
}
foreach (str_split($code) as $i => $ch) {
    $c = imagecolorallocate($im, random_int(0, 90), random_int(0, 90), random_int(0, 90));
    imagestring($im, 5, 6 + $i * 16 + random_int(-1, 1), 6 + random_int(-3, 3), $ch, $c);
}
$big = imagescale($im, 180, 56);
header('Content-Type: image/png');
header('Cache-Control: no-store');
imagepng($big);
```

**week10/public/register.php**

```
<?php
declare(strict_types=1);
session_start();
require __DIR__ . '/../src/db.php';
$errors = []; $old = ['name' => '', 'email' => '']; $done = isset($_GET['ok']);
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $old['name'] = trim((string)($_POST['name'] ?? ''));
    $old['email'] = trim((string)($_POST['email'] ?? ''));
    $pass = (string)($_POST['password'] ?? '');
    $cap = strtoupper(trim((string)($_POST['captcha'] ?? '')));
    $expected = (string)($_SESSION['captcha'] ?? '');
    unset($_SESSION['captcha']);                                    // single use
    if ($expected === '' || !hash_equals($expected, $cap)) { $errors[] = 'CAPTCHA is
incorrect.'; }
    if (mb_strlen($old['name']) < 3) { $errors[] = 'Name must have at least 3
characters.'; }
    if (!filter_var($old['email'], FILTER_VALIDATE_EMAIL)) { $errors[] = 'E-mail is not
valid.'; }
    if (strlen($pass) < 10) { $errors[] = 'Password must have at least 10 characters.'; }
    $avatar = null;
    if (!$errors && ($_FILES['avatar']['error'] ?? UPLOAD_ERR_NO_FILE) !==
UPLOAD_ERR_NO_FILE) {
        $avatar = saveAvatar($_FILES['avatar'], $errors);
    }
    if (!$errors) {
        try {
            $st = db()->prepare('INSERT INTO users (name, email, password_hash, avatar)
VALUES (:n, :e, :p, :a)');
            $st->execute([':n' => $old['name'], ':e' => $old['email'], ':p' =>
password_hash($pass, PASSWORD_DEFAULT), ':a' => $avatar]);
            header('Location: register.php?ok=1'); exit;
        } catch (PDOException $ex) {
            if ($ex->getCode() === '23000') { $errors[] = 'E-mail is already
registered.'; } else { throw $ex; }
        }
    }
}
?>
<!DOCTYPE html><html lang="en"><head><meta charset="utf-
8"><title>Register</title></head><body><main>
<h1>Create account</h1>
<?php if ($done): ?><p role="status">Account created.</p><?php endif; ?>
<?php foreach ($errors as $m): ?><p role="alert" style="color:#b00020"><?=
e($m) ?></p><?php endforeach; ?>
<form method="post" enctype="multipart/form-data">
  <p><label>Name <input name="name" required value="<?= e($old['name']) ?>"></label></p>
  <p><label>E-mail <input name="email" type="email" required value="<?=
e($old['email']) ?>"></label></p>
  <p><label>Password <input name="password" type="password" minlength="10" required
autocomplete="new-password"></label></p>
```

```
  <p><label>Avatar <input name="avatar" type="file"
accept="image/png,image/jpeg,image/webp"></label></p>
  <p><img src="captcha.php" alt="CAPTCHA code"> <label>Type the code <input name="captcha"
required autocomplete="off"></label></p>
  <button>Register</button>
</form></main></body></html>
```

**week10/public/avatar.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/db.php';
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
$st = db()->prepare('SELECT avatar FROM users WHERE id = ?');
$st->execute([$id ?: 0]);
$file = $st->fetchColumn();
if (!is_string($file) || !preg_match('/^[a-f0-9]{32}\.(jpg|png|webp)$/', $file))
{ http_response_code(404); exit; }
$types = ['jpg' => 'image/jpeg', 'png' => 'image/png', 'webp' => 'image/webp'];
header('Content-Type: ' . $types[pathinfo($file, PATHINFO_EXTENSION)]);
header('X-Content-Type-Options: nosniff');
readfile(__DIR__ . '/../storage/avatars/' . $file);
```

**week10/.htaccess**

```
# block direct web access to non-public folders (needs AllowOverride All from Lab 0)
RedirectMatch 404 "/(vendor|src|config|storage|\.git)(/|$)"
```

**week10/practice.sql (run each query and record the result)**

```
SELECT * FROM products WHERE price > 1000 ORDER BY price;
SELECT c.name, COUNT(*) n FROM products p JOIN categories c ON c.id = p.category_id GROUP
BY c.id;
SELECT name FROM products WHERE stock < 20;
UPDATE products SET stock = stock - 1 WHERE id = 1;
SELECT category_id, SUM(price * stock) AS stock_value FROM products GROUP BY category_id;
SELECT p.name FROM products p LEFT JOIN order_items oi ON oi.product_id = p.id WHERE
oi.order_id IS NULL;
START TRANSACTION; UPDATE products SET stock = 0 WHERE id = 2; ROLLBACK; SELECT stock FROM
products WHERE id = 2;
EXPLAIN SELECT * FROM products WHERE category_id = 2;
```

## Version A – Rocky Linux 10

**Step 1 – Install and start MariaDB.** Rocky 10 ships MariaDB 10.11. The root account uses socket authentication, so `sudo mariadb` needs no password.

```
sudo dnf -y install mariadb-server
sudo systemctl enable --now mariadb
sudo mariadb-secure-installation        # answers: n (keep unix_socket), Y, Y, Y, Y
sudo mariadb -e "SELECT VERSION();"
```

**Step 2 – Create the database and least-privilege user.** The application user cannot create or drop tables.

```
sudo mariadb <<'SQL'
CREATE DATABASE shop CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'shop_app'@'localhost' IDENTIFIED BY 'ChangeMe_2026!';
GRANT SELECT, INSERT, UPDATE, DELETE ON shop.* TO 'shop_app'@'localhost';
SQL
mkdir -p /var/www/webeng/week10/{public,src,config,storage/avatars} && cd
/var/www/webeng/week10
kate schema.sql config/db.php src/db.php public/*.php .htaccess practice.sql &
sudo mariadb shop < schema.sql
sudo mariadb shop -e "SELECT COUNT(*) FROM products;"
```

**Step 3 – Practise SQL and install a GUI.** Run practice.sql line by line, then open DBeaver and view the ER diagram.

```
sudo mariadb shop < practice.sql | head -n 30
flatpak install -y flathub io.dbeaver.DBeaverCommunity
sudo mariadb -e "SELECT user,host FROM mysql.user WHERE user='shop_app';"
```

**Step 4 – Prove the upload fails under SELinux, then fix it.** The web server may write only to files labelled `httpd_sys_rw_content_t`. Try an upload first and read the denial.

```
chmod 2775 storage storage/avatars && sudo restorecon -Rv /var/www/webeng/week10
# register with an avatar in the browser → upload fails
sudo ausearch -m avc -ts recent | tail -n 3
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/var/www/webeng/week10/storage(/.*)?"
sudo restorecon -Rv storage && ls -Zd storage/avatars
```

**Step 5 – Test registration and injection attempts.** Use the browser for the CAPTCHA; use curl to prove that hostile input is harmless.

```
firefox http://webeng.test/week10/public/register.php &
curl -s -o /dev/null -w "%{http_code}\n"
"http://webeng.test/week10/public/avatar.php?id=1%20OR%201=1"
curl -s -o /dev/null -w "%{http_code}\n" http://webeng.test/week10/config/db.php
sudo mariadb shop -e "SELECT id,name,email,LEFT(password_hash,7) hash,avatar FROM users;"
ls -l storage/avatars
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Install and start MariaDB.** Kubuntu 26.04 ships MariaDB 11.8. The root account uses socket authentication.

```
sudo apt -y install mariadb-server
sudo systemctl enable --now mariadb
sudo mariadb-secure-installation        # answers: n (keep unix_socket), Y, Y, Y, Y
sudo mariadb -e "SELECT VERSION();"
```

**Step 2 – Create the database and least-privilege user.** The application user cannot create or drop tables.

```
sudo mariadb <<'SQL'
CREATE DATABASE shop CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'shop_app'@'localhost' IDENTIFIED BY 'ChangeMe_2026!';
GRANT SELECT, INSERT, UPDATE, DELETE ON shop.* TO 'shop_app'@'localhost';
SQL
mkdir -p /var/www/webeng/week10/{public,src,config,storage/avatars} && cd
/var/www/webeng/week10
kate schema.sql config/db.php src/db.php public/*.php .htaccess practice.sql &
sudo mariadb shop < schema.sql
sudo mariadb shop -e "SELECT COUNT(*) FROM products;"
```

**Step 3 – Practise SQL and install a GUI.** Run practice.sql line by line, then open DBeaver and view the ER diagram.

```
sudo mariadb shop < practice.sql | head -n 30
flatpak install -y flathub io.dbeaver.DBeaverCommunity
sudo mariadb -e "SELECT user,host FROM mysql.user WHERE user='shop_app';"
```

**Step 4 – Give the web server write access only where needed.** PHP-FPM runs as www-data. The setgid web root already gives group www-data; allow group write on the storage folder only.

```
chmod 2775 storage storage/avatars
ls -ld storage/avatars                  # group should be www-data
sudo -u www-data test -w storage/avatars && echo "www-data can write"
```

**Step 5 – Test registration and injection attempts.** Use the browser for the CAPTCHA; use curl to prove that hostile input is harmless.

```
firefox http://webeng.test/week10/public/register.php &
curl -s -o /dev/null -w "%{http_code}\n"
"http://webeng.test/week10/public/avatar.php?id=1%20OR%201=1"
curl -s -o /dev/null -w "%{http_code}\n" http://webeng.test/week10/config/db.php
sudo mariadb shop -e "SELECT id,name,email,LEFT(password_hash,7) hash,avatar FROM users;"
ls -l storage/avatars
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `SELECT COUNT(*) FROM products;` | 8 |
| 2 | Register with wrong CAPTCHA | Error “CAPTCHA is incorrect.” and no row inserted |
| 3 | Register twice with the same e-mail | Error “E-mail is already registered.” (SQLSTATE 23000) |
| 4 | Stored password | `password_hash` begins with `$2y$` or `$argon2id$`, never plain text |
| 5 | Upload a `.php` file renamed `.png` | Rejected by `finfo` / `getimagesize` |
| 6 | `curl .../config/db.php` | 404; `avatar.php?id=1 OR 1=1` returns 404, not data |
| 7 | Rocky: avatar upload after relabel | Saved in `storage/avatars` with type `httpd_sys_rw_content_t` |

## Stretch Goals (Industry Practice)

- Add a 60-second CAPTCHA expiry and a limit of 5 attempts per IP using a database table.
- Integrate the self-hosted ALTCHA widget in place of the image CAPTCHA.
- Add `ALTER TABLE users ADD COLUMN email_verified_at DATETIME NULL` and write a migration script.

## Review Questions

1. Normalise a table `orders(order_id, customer_name, customer_city, product_name, qty)` to 3NF.
2. Why does a prepared statement stop `' OR '1'='1`?
3. Why is the application database user not `root`?
4. List four checks performed before accepting an uploaded image.
5. Compare an image CAPTCHA with reCAPTCHA in terms of security, privacy and accessibility.

# Week 11: Full-Stack PHP: Templates, Authentication, Products, Cart and Error Handling

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 21–22 | **CLO** | CLO_3 |
| **PLO** | PLO_4 | **Bloom's Level** | C4 (Analyzing) |
| **Reading** | Ch. 12–14 Advanced PHP, MySQL, E-Commerce | **Assessment** | A3 issued (due Week 12); Q3 (CLO_3, 6 marks) |

## Learning Outcomes

- Structure a PHP application with a bootstrap file, helpers and reusable templates.
- Implement registration/login/logout with password hashing, sessions and session-ID regeneration.
- Protect forms with CSRF tokens and authorise actions.
- Display database products, manage a session cart and place orders in a transaction.
- Handle errors with exceptions, central handlers and logging.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of PDO, CAPTCHA and uploads; A3 briefing |
| 0:10 – 0:45 | 35 | Concept Block 1 | Application structure, templates, authentication, sessions, CSRF |
| 0:45 – 1:20 | 35 | Concept Block 2 | Products, cart, transactions, error handling and logging |
| 1:20 – 1:30 | 10 | Break | Rest; copy week10 to week11 |
| 1:30 – 2:40 | 70 | Hands-on Lab | Lab 11 – Mini-shop with login, cart and checkout |
| 2:40 – 2:55 | 15 | Quiz Q3 | CLO_3 quiz (PDO, sessions, security) |
| 2:55 – 3:00 | 5 | Wrap-up | Collect questions, preview Week 12 |

## Lecture Notes

### 11.1 Application Structure

Even without a framework, separate **public entry points** (`public/*.php`) from **non-public code** (`src/`, `config/`, `storage/`). A `bootstrap.php` starts the session, configures error handling and loads helpers. Templates (header/footer) remove duplication. This is the seed of the MVC pattern used by CodeIgniter and Laravel in Week 13.

### 11.2 Authentication and Sessions

- **Hash passwords** with `password_hash($pw, PASSWORD_DEFAULT)` (bcrypt/argon2) and verify with `password_verify()`; never use MD5 or SHA-1.
- After login call `session_regenerate_id(true)` to prevent **session fixation**.
- Session cookie flags: `HttpOnly`, `Secure` (HTTPS), `SameSite=Lax`.
- Use one generic message ("E-mail or password incorrect") to avoid revealing which accounts exist.

- **Authentication** = who you are; **authorisation** = what you may do (check `role` on every protected action).

### 11.3 CSRF

Cross-Site Request Forgery tricks a logged-in browser into submitting a request. Defence: embed a random per-session token in every state-changing form, verify it with `hash_equals()` and use `SameSite` cookies. State-changing actions must use POST, never GET.

### 11.4 Cart and Checkout

A simple cart lives in `$_SESSION["cart"]` as `product_id => qty`. Prices must always be read from the database at checkout, never from the browser. Checkout runs in a **transaction**: lock the product rows (`SELECT … FOR UPDATE`), verify stock, insert the order and items, decrement stock, commit; on any exception roll back so the database never holds half an order.

### 11.5 Error Handling and Logging

- Configure PDO with `ERRMODE_EXCEPTION`; catch `PDOException` only where you can recover (for example duplicate keys).
- Install `set_exception_handler()` for everything else: log the full error, show users a short message and HTTP 500.
- Production: `display_errors=0`, `log_errors=1`, a log file outside the web root, and log rotation.

## Hands-on Lab 11: Mini-Shop: Login, Product List, Session Cart and Transactional Checkout

Extend the Week 10 application into a working shop with templates, login/logout, CSRF protection, a product list, cart and checkout that stores orders in MariaDB.

### Free & Open-Source Tools

- PHP 8.x + PDO
- MariaDB
- PHP-FPM error log and Apache logs
- PHPStan

### Deliverables

- `/var/www/webeng/week11` copied from week10 with new pages
- Working flow: register → login → add to cart → checkout → order row in DB
- Error log entry produced by a deliberate failure
- PHPStan level 6 clean

### Common Source Files (identical on both operating systems)

**`week11/src/bootstrap.php`**

```
<?php
declare(strict_types=1);
ini_set('display_errors', '0');
ini_set('log_errors', '1');
ini_set('error_log', __DIR__ . '/../storage/php-error.log');
session_set_cookie_params(['httponly' => true, 'samesite' => 'Lax', 'secure'
=> !empty($_SERVER['HTTPS'])]);
session_start();
```

```
require __DIR__ . '/db.php';
set_exception_handler(function (Throwable $t): void {
    error_log((string)$t);
    http_response_code(500);
    echo '<h1>Something went wrong</h1><p>Please try again later.</p>';
});
function csrf_token(): string { return $_SESSION['csrf'] ??= bin2hex(random_bytes(32)); }
function csrf_field(): string { return '<input type="hidden" name="csrf" value="' .
e(csrf_token()) . '">'; }
function csrf_check(): void {
    $t = (string)($_SESSION['csrf'] ?? '');
    if ($t === '' || !hash_equals($t, (string)($_POST['csrf'] ?? '')))
{ http_response_code(419); exit('Invalid CSRF token'); }
}
function current_user(): ?array {
    static $u = false;
    if ($u === false) {
        $u = null;
        if (isset($_SESSION['uid'])) {
            $st = db()->prepare('SELECT id, name, role FROM users WHERE id = ?');
$st->execute([$_SESSION['uid']]);
            $u = $st->fetch() ?: null;
        }
    }
    return $u;
}
function require_login(): array { $u = current_user(); if (!$u) { header('Location:
login.php'); exit; } return $u; }
function redirect(string $to): never { header('Location: ' . $to); exit; }
function page_start(string $title): void {
    $u = current_user(); $n = array_sum($_SESSION['cart'] ?? []);
    echo '<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width, initial-scale=1">';
    echo '<title>' . e($title) . ' - PK Shop</title></head><body><header><nav><a
href="products.php">Products</a> | <a href="cart.php">Cart (' . $n . ')</a> | ';
    echo $u ? e($u['name']) . ' <form method="post" action="logout.php"
style="display:inline">' . csrf_field() . '<button>Logout</button></form>'
            : '<a href="login.php">Login</a> | <a href="register.php">Register</a>';
    echo '</nav></header><main><h1>' . e($title) . '</h1>';
}
function page_end(): void { echo '</main><footer><small>&copy; 2026 PK
Shop</small></footer></body></html>'; }
```

**week11/public/login.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
$err = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $st = db()->prepare('SELECT id, password_hash FROM users WHERE email = ?');
    $st->execute([trim((string)($_POST['email'] ?? ''))]);
    $u = $st->fetch();
    if ($u && password_verify((string)($_POST['password'] ?? ''), $u['password_hash'])) {
        session_regenerate_id(true);                          // prevents session fixation
        $_SESSION['uid'] = (int)$u['id'];
        redirect('products.php');
    }
    $err = 'E-mail or password incorrect.';
}
page_start('Login');
if ($err) { echo '<p role="alert" style="color:#b00020">' . e($err) . '</p>'; }
?>
<form method="post"><?= csrf_field() ?>
  <p><label>E-mail <input type="email" name="email" required></label></p>
```

```
  <p><label>Password <input type="password" name="password" required></label></p>
  <button>Login</button></form>
<?php page_end();
```

**week11/public/logout.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
if ($_SERVER['REQUEST_METHOD'] === 'POST') { csrf_check(); $_SESSION = [];
session_destroy(); }
redirect('login.php');
```

**week11/public/products.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
$rows = db()->query('SELECT p.id, p.name, p.price, p.stock, c.name AS cat
                     FROM products p JOIN categories c ON c.id = p.category_id ORDER BY
c.name, p.name')->fetchAll();
page_start('Products');
echo '<table border="1"><tr><th>Name</th><th>Category</th><th>Price
(Rs)</th><th>Stock</th><th></th></tr>';
foreach ($rows as $r) {
    echo '<tr><td>' . e($r['name']) . '</td><td>' . e($r['cat']) . '</td><td>' .
number_format((float)$r['price'], 2) . '</td><td>' . (int)$r['stock'] . '</td><td>';
    echo $r['stock'] > 0 ? '<form method="post" action="cart.php">' . csrf_field() .
'<input type="hidden" name="action" value="add"><input type="hidden" name="id" value="' .
(int)$r['id'] . '"><button>Add</button></form>' : 'Sold out';
    echo '</td></tr>';
}
echo '</table>';
page_end();
```

**week11/public/cart.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
$cart = $_SESSION['cart'] ?? [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $id = (int)($_POST['id'] ?? 0);
    switch ($_POST['action'] ?? '') {
        case 'add':    $cart[$id] = ($cart[$id] ?? 0) + 1; break;
        case 'update': $q = max(0, min(99, (int)($_POST['qty'] ?? 0))); if ($q === 0)
{ unset($cart[$id]); } else { $cart[$id] = $q; } break;
        case 'remove': unset($cart[$id]); break;
    }
    $_SESSION['cart'] = $cart;
    redirect('cart.php');
}
page_start('Your cart');
if (!$cart) { echo '<p>Cart is empty.</p>'; page_end(); exit; }
$in = implode(',', array_fill(0, count($cart), '?'));
$st = db()->prepare("SELECT id, name, price FROM products WHERE id IN ($in)");
$st->execute(array_keys($cart));
$total = 0.0;
echo '<table border="1"><tr><th>Item</th><th>Qty</th><th>Line (Rs)</th></tr>';
foreach ($st as $p) {
    $q = $cart[$p['id']]; $line = $q * (float)$p['price']; $total += $line;
    echo '<tr><td>' . e($p['name']) . '</td><td><form method="post">' . csrf_field() .
'<input type="hidden" name="action" value="update"><input type="hidden" name="id"
value="' . (int)$p['id'] . '"><input type="number" name="qty" min="0" max="99" value="' .
$q . '"><button>Update</button></form></td><td>' . number_format($line, 2) . '</td></tr>';
```

```
}
echo '</table><p><strong>Total: Rs ' . number_format($total, 2) . '</strong></p>';
echo '<form method="post" action="checkout.php">' . csrf_field() .
'<button>Checkout</button></form>';
page_end();
```

**week11/public/checkout.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
$user = require_login();
if ($_SERVER['REQUEST_METHOD'] !== 'POST') { redirect('cart.php'); }
csrf_check();
$cart = $_SESSION['cart'] ?? [];
if (!$cart) { redirect('cart.php'); }
$pdo = db();
$pdo->beginTransaction();
try {
    $total = 0.0; $lines = [];
    $sel = $pdo->prepare('SELECT id, price, stock FROM products WHERE id = ? FOR UPDATE');
// row lock
    foreach ($cart as $id => $qty) {
        $sel->execute([$id]); $p = $sel->fetch();
        if (!$p || $p['stock'] < $qty) { throw new RuntimeException("Insufficient stock
for product $id"); }
        $total += $qty * (float)$p['price']; $lines[] = [$id, $qty, $p['price']];
    }
    $pdo->prepare('INSERT INTO orders (user_id, total) VALUES
(?, ?)')->execute([$user['id'], $total]);
    $oid = (int)$pdo->lastInsertId();
    $ins = $pdo->prepare('INSERT INTO order_items (order_id, product_id, qty, unit_price)
VALUES (?,?,?,?)');
    $upd = $pdo->prepare('UPDATE products SET stock = stock - ? WHERE id = ?');
    foreach ($lines as [$id, $qty, $price]) { $ins->execute([$oid, $id, $qty, $price]);
$upd->execute([$qty, $id]); }
    $pdo->commit();
    $_SESSION['cart'] = [];
    page_start('Order placed'); echo '<p>Order #' . $oid . ' total Rs ' .
number_format($total, 2) . '</p>'; page_end();
} catch (Throwable $t) {
    $pdo->rollBack();
    error_log($t->getMessage());
    page_start('Checkout failed'); echo '<p>Sorry, we could not complete your order.</p>';
page_end();
}
```

## Version A – Rocky Linux 10

**Step 1 – Copy the Week 10 project and grant order privileges.** The app user needs INSERT on orders; grants are already on the whole `shop` schema. Copy register.php and the shared source.

```
cd /var/www/webeng && cp -a week10 week11 && cd week11
rm -f storage/avatars/* && kate src/bootstrap.php public/login.php public/logout.php
public/products.php public/cart.php public/checkout.php &
sed -i "s#^session_start();#require __DIR__ . '/../src/bootstrap.php';#; /require
__DIR__ . '\/..\/src\/db.php';/d" public/register.php
sudo restorecon -Rv .
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/var/www/webeng/week11/storage(/.*)?"
&& sudo restorecon -Rv storage
```

**Step 2 – Create a test user and walk through the flow.** Register in the browser (CAPTCHA), then log in and buy.

```
firefox http://webeng.test/week11/public/register.php &
```

```
firefox http://webeng.test/week11/public/login.php &
sudo mariadb shop -e "SELECT id,email FROM users;"
```

**Step 3 – Verify CSRF protection from the shell.** A POST without a token must be refused with 419.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST -d 'action=add&id=1'
http://webeng.test/week11/public/cart.php
curl -sI -c - http://webeng.test/week11/public/login.php | grep -i -E 'set-cookie'
```

**Step 4 – Verify the order in the database.** Order, items and reduced stock must match.

```
sudo mariadb shop -e "SELECT * FROM orders; SELECT * FROM order_items; SELECT
id,name,stock FROM products LIMIT 3;"
```

**Step 5 – Trigger and read an error.** Stop the database, load the products page, read the log, restart.

```
sudo systemctl stop mariadb; curl -s -o /dev/null -w "%{http_code}\n"
http://webeng.test/week11/public/products.php
tail -n 5 storage/php-error.log; sudo systemctl start mariadb
vendor/bin/phpstan analyse -l 6 src public 2>/dev/null || (composer require --dev
phpstan/phpstan && vendor/bin/phpstan analyse -l 6 src public)
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Copy the Week 10 project.** Copy register.php and the shared source, and make register.php use the new bootstrap.

```
cd /var/www/webeng && cp -a week10 week11 && cd week11
rm -f storage/avatars/* && kate src/bootstrap.php public/login.php public/logout.php
public/products.php public/cart.php public/checkout.php &
sed -i "s#^session_start();#require __DIR__ . '/../src/bootstrap.php';#; /require
__DIR__ . '\/..\/src\/db.php';/d" public/register.php
chmod 2775 storage storage/avatars
```

**Step 2 – Create a test user and walk through the flow.** Register in the browser (CAPTCHA), then log in and buy.

```
firefox http://webeng.test/week11/public/register.php &
firefox http://webeng.test/week11/public/login.php &
sudo mariadb shop -e "SELECT id,email FROM users;"
```

**Step 3 – Verify CSRF protection from the shell.** A POST without a token must be refused with 419.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST -d 'action=add&id=1'
http://webeng.test/week11/public/cart.php
curl -sI -c - http://webeng.test/week11/public/login.php | grep -i -E 'set-cookie'
```

**Step 4 – Verify the order in the database.** Order, items and reduced stock must match.

```
sudo mariadb shop -e "SELECT * FROM orders; SELECT * FROM order_items; SELECT
id,name,stock FROM products LIMIT 3;"
```

**Step 5 – Trigger and read an error.** Stop the database, load the products page, read the log, restart.

```
sudo systemctl stop mariadb; curl -s -o /dev/null -w "%{http_code}\n"
http://webeng.test/week11/public/products.php
tail -n 5 storage/php-error.log; sudo systemctl start mariadb
composer require --dev phpstan/phpstan && vendor/bin/phpstan analyse -l 6 src public
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | POST to `cart.php` without token | `419` and message "Invalid CSRF token" |
| 2 | Login with wrong password | Generic error message; no session user |
| 3 | `session_id` before and after login | Cookie value changes (regeneration) |
| 4 | Checkout with 2 products | One `orders` row, two `order_items` rows, stock reduced |
| 5 | Checkout quantity above stock | "could not complete your order"; no rows inserted (rollback) |
| 6 | Database stopped | HTTP 500 and a stack trace in `storage/php-error.log`, none on screen |

## Stretch Goals (Industry Practice)

- Add an order-history page using a JOIN.
- Add a `role = admin` check and an admin page that edits product stock.
- Add login rate limiting (5 failures → 15 min lock) in a table.

## Assignment A3 (CLO_3, 6 marks) – issued this week, due Week 12

Extend the shop with (a) an order-history page for the logged-in user, (b) an admin-only page to add products with an image upload, (c) a flash-message helper and (d) PHPStan level 6 clean. Submit the repository, a 1-page security checklist (CSRF, XSS, SQL injection, session handling) and screenshots.

# Review Questions

1. Why must `session_regenerate_id(true)` be called after login?
2. Why does checkout re-read prices from the database instead of trusting the cart?
3. Explain what `SELECT … FOR UPDATE` prevents when two users buy the last item.
4. Why is logout implemented as POST with a CSRF token?
5. What is logged versus shown to the user when an exception occurs, and why?

# Week 12: Queries, Result Handling, File Handling and Hourly Test

| Syllabus Lectures | Lec 23–24 | CLO | CLO_3 |
|---|---|---|---|
| PLO | PLO_4 | Bloom's Level | C4 (Analyzing) |
| Reading | Ch. 13–14 MySQL, File handling | Assessment | A3 due; OHT (CLO_2, 10 marks) |

## Learning Outcomes

- Execute and tune queries with PDO: fetch modes, LIKE search, pagination, joins, aggregates.
- Read an `EXPLAIN` plan and add an index that removes a full table scan.
- Read, write, import and export CSV safely with `fgetcsv`, `SplFileObject` and `php://output`.
- Process uploaded images with GD (validate, resize, store as WebP).

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & OHT briefing | A3 submission; rules for the hourly test |
| 0:10 – 1:10 | 60 | Hourly Test (OHT) | Weeks 9–11, CLO_2, closed book, 10 marks |
| 1:10 – 1:20 | 10 | Break | Rest; discuss answers informally |
| 1:20 – 2:00 | 40 | Concept Block | Queries, fetch modes, pagination, EXPLAIN, file handling, GD images |
| 2:00 – 2:50 | 50 | Hands-on Lab | Lab 12 – search, CSV import/export, thumbnails (finish at home) |
| 2:50 – 3:00 | 10 | Wrap-up | Review of OHT solutions and lab questions |

## Lecture Notes

### 12.1 Executing Queries and Retrieving Results

| PDO call | Returns | Use |
|---|---|---|
| `$pdo->query($sql)` | Statement | Static SQL with no user input |
| `$st = $pdo->prepare(); $st->execute([...])` | Statement | Any SQL containing values |
| `$st->fetch()` | One row or `false` | Single record |
| `$st->fetchAll()` | All rows | Small result sets |
| `$st->fetchColumn()` | First column value | `COUNT(*)`, single value |
| `foreach ($st as $row)` | Streams rows | Large result sets (low memory) |
| `PDO::FETCH_KEY_PAIR`, `FETCH_COLUMN`, `FETCH_CLASS` | Shaped results | Dropdown maps, ID lists, objects |

### 12.2 Search and Pagination

- Build `WHERE` clauses from an **allow-list** of fragments and bind all values.
- Escape LIKE wildcards in user input: `addcslashes($q, "%_\\")`.
- `LIMIT n OFFSET m` is simple but slow for deep pages; **keyset pagination** (`WHERE id > :last ORDER BY id LIMIT n`) scales better.
- Avoid the **N+1 problem** (one query per row); use a JOIN or a single `IN (...)` query.

### 12.3 EXPLAIN and Indexes

`EXPLAIN SELECT …` shows how MariaDB executes a query. Look at `type` (`ALL` = full table scan, `ref` / `range` = index use), `key` (index chosen) and `rows` (estimate). An index on `name` helps `LIKE 'Item 12%'` (prefix match) but not `LIKE '%12%'` (leading wildcard).

### 12.4 File Handling in PHP

| Task | Function |
|---|---|
| Whole file | `file_get_contents`, `file_put_contents($f, $data, LOCK_EX)` |
| Line by line | `fopen`, `fgets`, `feof`, `fclose`, `SplFileObject` |
| CSV | `fgetcsv($h, null, ",", "\"", "")`, `fputcsv($h, $row, ",", "\"", "")` |
| Paths / dirs | `basename`, `realpath`, `is_file`, `mkdir`, `scandir`, `glob` |
| Move / delete | `rename`, `copy`, `unlink`, `move_uploaded_file` |
| Output stream | `php://output` for downloads; `readfile` for files |

> **Compatibility:** From PHP 8.4 the CSV functions warn when the `$escape` argument is omitted. Pass all arguments explicitly (as in the lab) so the code runs without deprecations on both PHP 8.3 and 8.5.

Never build a file path from user input without `basename()` and an allow-list; a path such as `../../etc/passwd` is a **path traversal** attack.

## Hands-on Lab 12: Product Search, CSV Import/Export and Thumbnail Generation

Generate 5,000 products, build a searchable paginated listing, tune it with an index, export products as CSV, import CSV transactionally and create WebP thumbnails.

### Free & Open-Source Tools

- MariaDB `seq` tables, `EXPLAIN`
- PHP `SplFileObject`, GD
- PHPStan
- curl, `head`, `wc`

### Deliverables

- `search.php`, `export.php`, `import.php`, `src/images.php` under `/var/www/webeng/week12`
- EXPLAIN output before and after the index (copied into a text file)
- Imported products from a sample CSV with one bad line that rolls the whole import back

### Common Source Files (identical on both operating systems)

**sql/seed-and-explain.sql (run as root; the second EXPLAIN should show type=range)**

```
INSERT INTO products (category_id, name, price, stock)
SELECT 1 + (seq % 4), CONCAT('Item ', seq), 100 + (seq % 900), seq % 50 FROM
seq_1_to_5000;
EXPLAIN SELECT id, name FROM products WHERE name LIKE 'Item 12%';
CREATE INDEX idx_prod_name ON products (name);
EXPLAIN SELECT id, name FROM products WHERE name LIKE 'Item 12%';
```

**public/search.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
function run(string $sql, array $args = []): PDOStatement { $s = db()->prepare($sql);
$s->execute($args); return $s; }
$q = trim((string)($_GET['q'] ?? ''));
$cat = filter_input(INPUT_GET, 'cat', FILTER_VALIDATE_INT) ?: null;
$page = max(1, (int)($_GET['page'] ?? 1)); $per = 10;
$where = []; $args = [];
if ($q !== '') { $where[] = 'p.name LIKE :q'; $args[':q'] = addcslashes($q, '%_\\') .
'%'; }
if ($cat)      { $where[] = 'p.category_id = :c'; $args[':c'] = $cat; }
$w = $where ? 'WHERE ' . implode(' AND ', $where) : '';
$total = (int)run("SELECT COUNT(*) FROM products p $w", $args)->fetchColumn();
$pages = max(1, (int)ceil($total / $per)); $page = min($page, $pages); $off = ($page - 1)
* $per;
$rows = run("SELECT p.id, p.name, p.price, c.name AS cat FROM products p JOIN categories c
ON c.id = p.category_id $w ORDER BY p.id LIMIT $per OFFSET $off", $args)->fetchAll();
$cats = db()->query('SELECT id, name FROM categories')->fetchAll(PDO::FETCH_KEY_PAIR);
page_start('Product search');
?>
<form method="get"><input name="q" value="<?= e($q) ?>" placeholder="Name starts with…">
<select name="cat"><option value="">All</option><?php foreach ($cats as $id =>
$n): ?><option value="<?= $id ?>" <?= $cat === $id ? 'selected' : '' ?>><?=
e($n) ?></option><?php endforeach; ?></select>
<button>Search</button></form>
<p><?= $total ?> result(s) – page <?= $page ?> of <?= $pages ?> | <a
href="export.php">Export CSV</a></p>
<table border="1"><?php foreach ($rows as $r): ?><tr><td><?= (int)$r['id'] ?></td><td><?=
e($r['name']) ?></td><td><?= e($r['cat']) ?></td><td><?= number_format((float)$r['price'],
2) ?></td></tr><?php endforeach; ?></table>
<p><?php if ($page > 1): ?><a href="?<?= e(http_build_query(['q' => $q, 'cat' => $cat,
'page' => $page - 1])) ?>">Prev</a><?php endif; ?>
<?php if ($page < $pages): ?> <a href="?<?= e(http_build_query(['q' => $q, 'cat' => $cat,
'page' => $page + 1])) ?>">Next</a><?php endif; ?></p>
<?php page_end();
```

**public/export.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
header('Content-Type: text/csv; charset=utf-8');
header('Content-Disposition: attachment; filename="products-' . date('Ymd') . '.csv"');
$out = fopen('php://output', 'w');
fputcsv($out, ['category_id', 'name', 'price', 'stock'], ',', '"', '');
foreach (db()->query('SELECT category_id, name, price, stock FROM products ORDER BY id')
as $row) {
    fputcsv($out, $row, ',', '"', '');
}
fclose($out);
```

**public/import.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/bootstrap.php';
$msg = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $f = $_FILES['csv'] ?? null;
    if (!$f || $f['error'] !== UPLOAD_ERR_OK || $f['size'] > 1_000_000) { $msg[] = 'Upload
a CSV smaller than 1 MB.'; }
    else {
        $file = new SplFileObject($f['tmp_name']);
```

```
        $file->setFlags(SplFileObject::READ_CSV | SplFileObject::READ_AHEAD |
SplFileObject::SKIP_EMPTY | SplFileObject::DROP_NEW_LINE);
        $file->setCsvControl(',', '"', '');
        $pdo = db(); $pdo->beginTransaction();
        $ins = $pdo->prepare('INSERT INTO products (category_id, name, price, stock)
VALUES (?, ?, ?, ?)');
        $header = null; $n = 0;
        foreach ($file as $i => $row) {
            if ($header === null) { $header = $row; continue; }
            [$cat, $name, $price, $stock] = array_pad($row, 4, null);
            if (!ctype_digit((string)$cat) || trim((string)$name) === ''
|| !is_numeric($price) || !ctype_digit((string)$stock)) {
                $msg[] = 'Line ' . ($i + 1) . ' is invalid.'; continue;
            }
            $ins->execute([(int)$cat, trim((string)$name), $price, (int)$stock]); $n++;
        }
        if ($msg) { $pdo->rollBack(); $msg[] = 'Import cancelled; nothing saved.'; }
        else { $pdo->commit(); $msg[] = "$n products imported."; }
    }
}
page_start('Import products');
foreach ($msg as $m) { echo '<p>' . e($m) . '</p>'; }
?>
<form method="post" enctype="multipart/form-data"><?= csrf_field() ?><input type="file"
name="csv" accept=".csv,text/csv" required><button>Import</button></form>
<?php page_end();
```

**src/images.php**

```
<?php
declare(strict_types=1);
function makeThumb(string $src, string $dst, int $max = 200): bool {
    $info = getimagesize($src);
    if ($info === false) { return false; }
    [$w, $h, $type] = $info;
    $im = match ($type) {
        IMAGETYPE_JPEG => imagecreatefromjpeg($src),
        IMAGETYPE_PNG  => imagecreatefrompng($src),
        IMAGETYPE_WEBP => imagecreatefromwebp($src),
        default        => false,
    };
    if ($im === false) { return false; }
    $r = min($max / $w, $max / $h, 1);
    $t = imagescale($im, max(1, (int)($w * $r)), max(1, (int)($h * $r)));
    return $t !== false && imagewebp($t, $dst, 80);
}
```

**bin/thumbs.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/../src/images.php';
foreach (glob(__DIR__ . '/../storage/avatars/*.{jpg,png,webp}', GLOB_BRACE) ?: [] as $f) {
    $dst = __DIR__ . '/../storage/thumbs/' . pathinfo($f, PATHINFO_FILENAME) . '.webp';
    echo basename($f), makeThumb($f, $dst) ? ' -> thumb OK' : ' -> FAILED', PHP_EOL;
}
```

**sample-import.csv (third data row is invalid on purpose)**

```
category_id,name,price,stock
1,Phone Stand,799.00,20
2,Clean Code,4500.00,6
9x,Broken Row,10,1
```

## Version A – Rocky Linux 10

**Step 1 – Copy the project and seed 5,000 rows.** Reuse the previous application and its relabelled storage folder.

```
cd /var/www/webeng && cp -a week11 week12 && cd week12 && mkdir -p sql storage/thumbs bin
kate sql/seed-and-explain.sql public/search.php public/export.php public/import.php
src/images.php bin/thumbs.php sample-import.csv &
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/var/www/webeng/week12/storage(/.*)?"
&& sudo restorecon -Rv .
sudo mariadb shop < sql/seed-and-explain.sql
```

**Step 2 – Compare the two EXPLAIN plans.** Record `type`, `key` and `rows` before and after the index.

```
sudo mariadb shop -e "EXPLAIN SELECT id,name FROM products WHERE name LIKE 'Item 12%'\G" |
grep -E 'type|key:|rows'
sudo mariadb shop -e "EXPLAIN SELECT id,name FROM products WHERE name LIKE '%12%'\G" |
grep -E 'type|key:|rows'
```

**Step 3 – Test search and CSV export.** Use the browser for pages that need a login session; curl can test export when the page does not require one.

```
curl -s "http://webeng.test/week12/public/search.php?q=Item%2012" | grep -E 'result|page'
curl -s http://webeng.test/week12/public/export.php | head -n 3
curl -s http://webeng.test/week12/public/export.php | wc -l
```

**Step 4 – Test the transactional import.** Import the sample file in the browser; the bad line must cancel everything. Then remove the bad line and retry.

```
firefox http://webeng.test/week12/public/import.php &
sudo mariadb shop -e "SELECT COUNT(*) FROM products;"
```

**Step 5 – Generate thumbnails from CLI.** CLI runs as your user, so it reads files created by php-fpm (group apache).

```
cp ~/Pictures/*.jpg storage/avatars/ 2>/dev/null; php bin/thumbs.php
ls -lh storage/thumbs
php -r 'print_r(array_intersect_key(gd_info(), array_flip(["WebP Support","JPEG
Support","PNG Support"])));'
sudo ausearch -m avc -ts recent | tail -n 3
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Copy the project and seed 5,000 rows.** Reuse the previous application and its storage folder.

```
cd /var/www/webeng && cp -a week11 week12 && cd week12 && mkdir -p sql storage/thumbs bin
kate sql/seed-and-explain.sql public/search.php public/export.php public/import.php
src/images.php bin/thumbs.php sample-import.csv &
chmod 2775 storage storage/thumbs
sudo mariadb shop < sql/seed-and-explain.sql
```

**Step 2 – Compare the two EXPLAIN plans.** Record `type`, `key` and `rows` before and after the index.

```
sudo mariadb shop -e "EXPLAIN SELECT id,name FROM products WHERE name LIKE 'Item 12%'\G" |
grep -E 'type|key:|rows'
sudo mariadb shop -e "EXPLAIN SELECT id,name FROM products WHERE name LIKE '%12%'\G" |
grep -E 'type|key:|rows'
```

**Step 3 – Test search and CSV export.** Use the browser for pages that need a login session; curl can test export when the page does not require one.

```
curl -s "http://webeng.test/week12/public/search.php?q=Item%2012" | grep -E 'result|page'
curl -s http://webeng.test/week12/public/export.php | head -n 3
curl -s http://webeng.test/week12/public/export.php | wc -l
```

**Step 4 – Test the transactional import.** Import the sample file in the browser; the bad line must cancel everything. Then remove the bad line and retry.

```
firefox http://webeng.test/week12/public/import.php &
sudo mariadb shop -e "SELECT COUNT(*) FROM products;"
```

**Step 5 – Generate thumbnails from CLI.** On PHP 8.5 the old `imagedestroy()` is deprecated, so the code relies on automatic cleanup.

```
cp ~/Pictures/*.jpg storage/avatars/ 2>/dev/null; php bin/thumbs.php
ls -lh storage/thumbs
php -r 'print_r(array_intersect_key(gd_info(), array_flip(["WebP Support","JPEG
Support","PNG Support"])));'
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | First EXPLAIN (no index) | `type = ALL`, rows ≈ 5,000 |
| 2 | EXPLAIN after index (prefix LIKE) | `type = range`, `key = idx_prod_name`, far fewer rows |
| 3 | EXPLAIN with `%12%` | Index not usable (`ALL`): explain why in your notes |
| 4 | Search `q=Item 12` | Result count and pagination links preserve the query |
| 5 | Import with invalid line | “Import cancelled; nothing saved.” and product count unchanged |
| 6 | Import valid file | 2 products imported; export contains them |
| 7 | Thumbnails | WebP files ≤ 200 px created in `storage/thumbs` |

## Stretch Goals (Industry Practice)

- Implement keyset pagination and compare page 400 timings.
- Stream a 100,000-row export using unbuffered queries (`PDO::MYSQL_ATTR_USE_BUFFERED_QUERY`).
- Add a unit test for `makeThumb()` with PHPUnit.

## Hourly Test (OHT) – Blueprint and Sample Questions

| Q | Topic (CLO_2) | Marks | Type |
|---|---|---|---|
| 1 | PHP syntax: arrays, loops, `match` | 2 | Write code |
| 2 | SQL: DDL + SELECT with JOIN and GROUP BY | 3 | Write queries |
| 3 | PDO: prepared statement, login check, error handling | 3 | Complete code |
| 4 | Sessions, CSRF, upload validation | 2 | Short answers |

1. Given `$marks = ["Ali" => [70, 80], "Sara" => [90, 85]]`, print each student’s average and the name of the top student.
2. Write SQL to list category names with the number of products priced above 1000, only categories with more than one such product.

3. Complete the PHP fragment that checks a login with PDO and `password_verify`, regenerating the session ID on success.
4. List three checks for an uploaded image and one reason a CSRF token is needed on the logout form.

## Review Questions

1. Why does `LIKE '%12%'` ignore an index on `name` while `LIKE '12%'` can use it?
2. When would you choose `fetchAll()` over iterating the statement?
3. Why does the import run inside a transaction?
4. What is a path-traversal attack and how does `basename()` help?
5. Explain the N+1 query problem with a shop example.

# Week 13: Email, Cookies, Sessions and MVC Frameworks (CodeIgniter 4 and Laravel)

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 25–26 | **CLO** | CLO_3 (revisits CLO_1: MVC) |
| **PLO** | PLO_4 | **Bloom's Level** | C4 (Analyzing) |
| **Reading** | Ch. 15 PHP Frameworks | **Assessment** | — |

## Learning Outcomes

- Explain how e-mail is delivered (SMTP) and why SPF, DKIM and DMARC matter.
- Send and inspect e-mail safely in development with Symfony Mailer and Mailpit.
- Configure secure cookies and sessions (flags, lifetime, storage, fixation protection).
- Map MVC onto the three-tier model and compare CodeIgniter 4 with Laravel.
- Build a CodeIgniter 4 CRUD application with migrations, validation, CSRF and flash messages.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | E-mail (SMTP), SPF/DKIM/DMARC, cookies and sessions in depth |
| 0:45 – 1:20 | 35 | Concept Block 2 | MVC pattern, CodeIgniter 4 architecture, Laravel overview, framework analysis |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 13 – Mailpit, secure sessions and CodeIgniter 4 notes app |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 13.1 How E-mail Works

A mail user agent hands a message to an **SMTP** server (port 587 with STARTTLS or 465 with TLS). The server looks up the recipient domain's **MX** record in DNS and relays the message; the recipient reads it through **IMAP** or **POP3**. PHP's built-in `mail()` function depends on a local mail program and offers little control, so applications use a library such as **Symfony Mailer** or PHPMailer with an authenticated SMTP service.

| Mechanism | What it proves | Where it lives |
|---|---|---|
| SPF | Which servers may send for the domain | TXT record |
| DKIM | The message is signed and not altered | TXT record with public key |
| DMARC | Policy when SPF / DKIM fail; reports | TXT record `_dmarc` |

Rules for transactional mail: send from a domain you control, include a plain-text alternative, never put passwords in mail, make reset tokens random, single-use and short-lived, and store only a hash of the token. In development, use a **mail catcher** (Mailpit) so that no real message leaves the machine.

## 13.2 Cookies and Sessions

| Cookie attribute | Purpose |
|---|---|
| `Expires` / `Max-Age` | Lifetime; omitted = session cookie (deleted when the browser closes) |
| `Path`, `Domain` | Scope of the cookie |
| `Secure` | Send only over HTTPS |
| `HttpOnly` | Not readable from JavaScript (limits XSS damage) |
| `SameSite=Lax\|Strict\|None` | Controls cross-site sending (CSRF defence) |
| `__Host-` prefix | Forces Secure, Path=/ and no Domain |

A **session** keeps data on the server and gives the browser only a random identifier in a cookie. Storage can be files (PHP default), a database or Redis (needed when several servers share users). Protect sessions by regenerating the ID at privilege changes, setting an idle timeout and destroying the session on logout. Cookies are for preferences and identifiers, never for storing prices, roles or passwords.

## 13.3 MVC and the Three-Tier Model (CLO_1 Revisited)

| MVC part | Responsibility | Tier it lives in |
|---|---|---|
| Model | Data access and business rules | Application tier (talks to the data tier) |
| View | Templates that produce HTML / JSON | Application tier (output sent to the presentation tier) |
| Controller | Receives the request, calls models, picks a view | Application tier |
| Router / filters | URL → controller; CSRF, auth, throttling | Application tier |

## 13.4 CodeIgniter 4 and Laravel

| Aspect | CodeIgniter 4 | Laravel |
|---|---|---|
| Size / speed | Small, fast start, few dependencies | Large ecosystem, more setup and memory |
| CLI tool | `php spark` | `php artisan` |
| ORM | Model + Query Builder (entities optional) | Eloquent ORM (active record, relations) |
| Templates | PHP views, View Cells | Blade |
| Migrations / validation | Built in | Built in |
| Auth, queues, scheduler | Add-ons (Shield) / basic | First-party packages and starter kits |
| Typical fit | Small and medium apps, shared hosting | Large products, APIs, teams |

CodeIgniter 4 layout: `app/Controllers`, `app/Models`, `app/Views`, `app/Config` (Routes, Filters, Database), `app/Database/Migrations`, `public/` (the only web-exposed folder), `writable/` (logs, sessions, cache) and `.env` for secrets. Request lifecycle: `public/index.php` → router → before-filters (CSRF) → controller → model → view → after-filters → response.

# Hands-on Lab 13: Mailpit, Secure Sessions and a CodeIgniter 4 Notes Application

Catch e-mail with Mailpit in a Podman container, observe cookie and session behaviour with curl, then build a CRUD “Notes” application with CodeIgniter 4 (migrations, model validation, CSRF, flash data, pagination). Finish with a guided Laravel demo.

## Free & Open-Source Tools

- Podman (Apache-2.0) + Mailpit (MIT)
- Symfony Mailer (MIT)
- CodeIgniter 4 (MIT)
- Laravel (MIT) – demo
- PHP built-in server `php -S`

## Deliverables

- `mail/send.php` message visible in the Mailpit inbox (HTML + text)
- `state.php` demo with cookie jar output explaining each flag
- CodeIgniter project `ci-notes` with working add / list / delete / pagination and CSRF
- Laravel `route:list` screenshot and a one-paragraph comparison

## Common Source Files (identical on both operating systems)

**mail/send.php**

```
<?php
declare(strict_types=1);
require __DIR__ . '/vendor/autoload.php';
use Symfony\Component\Mailer\Mailer;
use Symfony\Component\Mailer\Transport;
use Symfony\Component\Mime\Email;

$mailer = new Mailer(Transport::fromDsn('smtp://127.0.0.1:1025'));
$token = bin2hex(random_bytes(16));
$link = 'http://webeng.test/reset?token=' . $token;
$mail = (new Email())
    ->from('noreply@webeng.test')->to('student@example.com')
    ->subject('Reset your password')
    ->text("Open this link within 30 minutes: $link")
    ->html('<p>Open <a href="' . htmlspecialchars($link, ENT_QUOTES) . '">this link</a>
within 30 minutes.</p>');
$mailer->send($mail);
echo "Sent. Store only hash('sha256', token) = " . hash('sha256', $token) . PHP_EOL;
```

**public/state.php**

```
<?php
declare(strict_types=1);
session_set_cookie_params(['lifetime' => 0, 'path' => '/', 'httponly' => true, 'samesite'
=> 'Lax', 'secure' => false]);
session_start();
$_SESSION['visits'] = ($_SESSION['visits'] ?? 0) + 1;
$theme = in_array($_GET['theme'] ?? '', ['light', 'dark'], true) ? $_GET['theme'] :
($_COOKIE['theme'] ?? 'light');
setcookie('theme', $theme, ['expires' => time() + 86400 * 30, 'path' => '/', 'samesite' =>
'Lax']);
header('Content-Type: text/plain; charset=utf-8');
echo 'Session visits: ' . $_SESSION['visits'] . PHP_EOL . 'Theme cookie: ' .
htmlspecialchars((string)$theme) . PHP_EOL;
```

**ci-notes/app/Database/Migrations/<timestamp>_CreateNotes.php (keep the generated file name, replace the class body)**

```
<?php
namespace App\Database\Migrations;
use CodeIgniter\Database\Migration;

class CreateNotes extends Migration
```

```
{
    public function up(): void
    {
        $this->forge->addField([
            'id'         => ['type' => 'INT', 'constraint' => 11, 'unsigned' => true,
'auto_increment' => true],
            'title'      => ['type' => 'VARCHAR', 'constraint' => 120],
            'body'       => ['type' => 'TEXT', 'null' => true],
            'created_at' => ['type' => 'DATETIME', 'null' => true],
            'updated_at' => ['type' => 'DATETIME', 'null' => true],
        ]);
        $this->forge->addKey('id', true);
        $this->forge->createTable('notes');
    }
    public function down(): void { $this->forge->dropTable('notes'); }
}
```

**ci-notes/app/Models/NoteModel.php**

```
<?php
namespace App\Models;
use CodeIgniter\Model;

class NoteModel extends Model
{
    protected $table = 'notes';
    protected $primaryKey = 'id';
    protected $allowedFields = ['title', 'body'];
    protected $useTimestamps = true;
    protected $validationRules = [
        'title' => 'required|min_length[3]|max_length[120]',
        'body'  => 'permit_empty|max_length[2000]',
    ];
}
```

**ci-notes/app/Controllers/Notes.php**

```
<?php
namespace App\Controllers;
use App\Models\NoteModel;

class Notes extends BaseController
{
    public function index(): string
    {
        $model = new NoteModel();
        return view('notes/index', ['notes' => $model->orderBy('id', 'DESC')->paginate(5),
'pager' => $model->pager]);
    }
    public function store()
    {
        $model = new NoteModel();
        if (! $model->insert($this->request->getPost(['title', 'body']))) {
            return redirect()->back()->withInput()->with('errors', $model->errors());
        }
        return redirect()->to('/notes')->with('message', 'Note saved.');
    }
    public function delete(int $id)
    {
        (new NoteModel())->delete($id);
        return redirect()->to('/notes')->with('message', 'Note deleted.');
    }
}
```

**ci-notes/app/Views/notes/index.php**

```
<!DOCTYPE html><html lang="en"><head><meta charset="utf-
8"><title>Notes</title></head><body><main>
<h1>Notes</h1>
<?php if (session()->getFlashdata('message')): ?><p role="status"><?=
esc(session('message')) ?></p><?php endif; ?>
<?php foreach ((array) session('errors') as $err): ?><p role="alert"><?=
esc($err) ?></p><?php endforeach; ?>
<form action="<?= site_url('notes') ?>" method="post"><?= csrf_field() ?>
  <p><input name="title" value="<?= esc(old('title') ?? '') ?>" required
placeholder="Title"></p>
  <p><textarea name="body" placeholder="Body"><?= esc(old('body') ?? '') ?></textarea></p>
  <button>Save</button></form>
<?php foreach ($notes as $n): ?>
  <article><h2><?= esc($n['title']) ?></h2><p><?= esc($n['body'] ?? '') ?></p>
    <form action="<?= site_url('notes/' . $n['id'] . '/delete') ?>" method="post"><?=
csrf_field() ?><button>Delete</button></form></article>
<?php endforeach; ?>
<?= $pager->links() ?></main></body></html>
```

**add to ci-notes/app/Config/Routes.php**

```
$routes->get('notes', 'Notes::index');
$routes->post('notes', 'Notes::store');
$routes->post('notes/(:num)/delete', 'Notes::delete/$1');
```

## Version A – Rocky Linux 10

**Step 1 – Install Podman and start Mailpit.** Rootless container; ports are bound to the loopback interface only.

```
sudo dnf -y install podman
podman run -d --name mailpit -p 127.0.0.1:8025:8025 -p 127.0.0.1:1025:1025
docker.io/axllent/mailpit:latest
podman ps && curl -s http://127.0.0.1:8025/api/v1/messages | jq '.total'
```

**Step 2 – Send mail with Symfony Mailer.** Create the project folder, install the library and send.

```
mkdir -p ~/webeng/week13/{mail,public} && cd ~/webeng/week13/mail
composer require symfony/mailer
kate send.php && php send.php
curl -s http://127.0.0.1:8025/api/v1/messages | jq '.messages[0] | {Subject,
From: .From.Address, To: .To[0].Address}'
```

**Step 3 – Observe cookies and sessions.** Use a cookie jar; compare the first and second request, then change the theme.

```
cd ~/webeng/week13 && kate public/state.php && (php -S 127.0.0.1:8080 -t public >/dev/null
2>&1 &)
curl -si -c jar.txt http://127.0.0.1:8080/state.php | sed -n '1,12p'
curl -s -b jar.txt -c jar.txt "http://127.0.0.1:8080/state.php?theme=dark"; cat jar.txt
```

**Step 4 – Create the CodeIgniter database and project.** The application account may run migrations, so it owns the `notes` schema.

```
sudo dnf -y install php-intl php-mbstring php-xml php-mysqlnd php-pdo
sudo mariadb -e "CREATE DATABASE notes CHARACTER SET utf8mb4; CREATE USER
'notes_app'@'localhost' IDENTIFIED BY 'ChangeMe_2026!'; GRANT ALL ON notes.* TO
'notes_app'@'localhost';"
cd ~/webeng/week13 && composer create-project codeigniter4/appstarter ci-notes && cd ci-
notes
cp env .env && cat >> .env <<'EOF'
CI_ENVIRONMENT = development
```

```
database.default.hostname = localhost
database.default.database = notes
database.default.username = notes_app
database.default.password = ChangeMe_2026!
database.default.DBDriver = MySQLi
EOF
```

**Step 5 – Generate files, enable CSRF and migrate.** Use the generators, then replace the class bodies with the listings above.

```
php spark make:migration CreateNotes
php spark make:model NoteModel
php spark make:controller Notes
mkdir -p app/Views/notes && kate app/Models/NoteModel.php app/Controllers/Notes.php
app/Views/notes/index.php app/Config/Routes.php &
sed -i "s#// 'csrf',#'csrf',#" app/Config/Filters.php && grep -n "'csrf'"
app/Config/Filters.php
php spark migrate && php spark routes | grep -i notes
```

**Step 6 – Run and test.** The built-in server runs as your user, so SELinux does not confine it; Apache-hosted PHP would need booleans.

```
php spark serve --port 8081 &
sleep 2; curl -s -o /dev/null -w "POST without CSRF: %{http_code}\n" -X POST -d 'title=x'
http://127.0.0.1:8081/notes
firefox http://127.0.0.1:8081/notes &
getsebool httpd_can_sendmail httpd_can_network_connect httpd_can_network_connect_db
```

**Step 7 – Laravel demo (guided, 15 min).** Observe how the same MVC ideas appear with different tooling.

```
cd ~/webeng/week13 && composer create-project laravel/laravel lv-demo && cd lv-demo
php artisan route:list && php artisan make:controller NoteController --resource
php artisan serve --port=8082 &
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Install Podman and start Mailpit.** Rootless container; ports are bound to the loopback interface only.

```
sudo apt -y install podman
podman run -d --name mailpit -p 127.0.0.1:8025:8025 -p 127.0.0.1:1025:1025
docker.io/axllent/mailpit:latest
podman ps && curl -s http://127.0.0.1:8025/api/v1/messages | jq '.total'
```

**Step 2 – Send mail with Symfony Mailer.** Create the project folder, install the library and send.

```
mkdir -p ~/webeng/week13/{mail,public} && cd ~/webeng/week13/mail
composer require symfony/mailer
kate send.php && php send.php
curl -s http://127.0.0.1:8025/api/v1/messages | jq '.messages[0] | {Subject,
From: .From.Address, To: .To[0].Address}'
```

**Step 3 – Observe cookies and sessions.** Use a cookie jar; compare the first and second request, then change the theme.

```
cd ~/webeng/week13 && kate public/state.php && (php -S 127.0.0.1:8080 -t public >/dev/null
2>&1 &)
curl -si -c jar.txt http://127.0.0.1:8080/state.php | sed -n '1,12p'
curl -s -b jar.txt -c jar.txt "http://127.0.0.1:8080/state.php?theme=dark"; cat jar.txt
```

**Step 4 – Create the CodeIgniter database and project.** The application account may run migrations, so it owns the `notes` schema.

```
sudo apt -y install php-intl php-mbstring php-xml php-mysql php-sqlite3
sudo mariadb -e "CREATE DATABASE notes CHARACTER SET utf8mb4; CREATE USER
'notes_app'@'localhost' IDENTIFIED BY 'ChangeMe_2026!'; GRANT ALL ON notes.* TO
'notes_app'@'localhost';"
cd ~/webeng/week13 && composer create-project codeigniter4/appstarter ci-notes && cd ci-
notes
cp env .env && cat >> .env <<'EOF'
CI_ENVIRONMENT = development
database.default.hostname = localhost
database.default.database = notes
database.default.username = notes_app
database.default.password = ChangeMe_2026!
database.default.DBDriver = MySQLi
EOF
```

**Step 5 – Generate files, enable CSRF and migrate.** Use the generators, then replace the class bodies with the listings above.

```
php spark make:migration CreateNotes
php spark make:model NoteModel
php spark make:controller Notes
mkdir -p app/Views/notes && kate app/Models/NoteModel.php app/Controllers/Notes.php
app/Views/notes/index.php app/Config/Routes.php &
sed -i "s#// 'csrf',#'csrf',#" app/Config/Filters.php && grep -n "'csrf'"
app/Config/Filters.php
php spark migrate && php spark routes | grep -i notes
```

**Step 6 – Run and test.** Check that the framework refuses a POST without a CSRF token.

```
php spark serve --port 8081 &
sleep 2; curl -s -o /dev/null -w "POST without CSRF: %{http_code}\n" -X POST -d 'title=x'
http://127.0.0.1:8081/notes
firefox http://127.0.0.1:8081/notes &
sudo ss -tlnp | grep -E '8080|8081|1025|8025'
```

**Step 7 – Laravel demo (guided, 15 min).** Observe how the same MVC ideas appear with different tooling.

```
cd ~/webeng/week13 && composer create-project laravel/laravel lv-demo && cd lv-demo
php artisan route:list && php artisan make:controller NoteController --resource
php artisan serve --port=8082 &
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | Mailpit UI `http://127.0.0.1:8025` | One message with subject “Reset your password”, HTML and text parts |
| 2 | `cat jar.txt` | `PHPSESSID` marked HttpOnly (`#HttpOnly_` prefix); `theme` cookie with future expiry |
| 3 | Second request visits counter | Increments from 1 to 2 using the same session |
| 4 | `php spark migrate` | `notes` table exists in MariaDB |
| 5 | POST /notes without token | Rejected (403 / error page) because the CSRF filter is active |
| 6 | Add six notes | Pagination appears after five; flash message shown after save and delete |

| # | Check | Expected Result |
|---|---|---|
| 7 | Title shorter than 3 characters | Validation error displayed and old input preserved |

## Stretch Goals (Industry Practice)

- Use CodeIgniter's `service("email")` with `email.SMTPHost=127.0.0.1`, `email.SMTPPort=1025` in `.env` to mail a note.
- Switch sessions to the database handler and inspect the table.
- Create a Laravel migration and compare the generated code with CodeIgniter.

## Review Questions

1. Why are password-reset tokens stored hashed?
2. Explain `HttpOnly`, `Secure` and `SameSite` and which attack each reduces.
3. Place Model, View and Controller on a three-tier diagram and justify.
4. Give two reasons to choose CodeIgniter 4 and two to choose Laravel for a new project.
5. What problem does a database migration solve in a team?

# Week 14: WordPress: Architecture, Installation, Plugins and the Block Editor

| Syllabus Lectures | Lec 27–28 | CLO | CLO_3 |
|---|---|---|---|
| PLO | PLO_4 | Bloom's Level | C4 (Analyzing) |
| Reading | Ch. 16 Content Management Systems | Assessment | — |

## Learning Outcomes

- Explain WordPress architecture: core, database tables, themes, plugins and the hook system.
- Install WordPress from the command line with WP-CLI on a hardened LAMP server.
- Select, install and evaluate plugins using objective criteria.
- Build pages with the block editor, patterns and the Site Editor (drag-and-drop).
- Analyze when to use WordPress, CodeIgniter or Laravel.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | CMS concepts, WordPress architecture, hooks, template hierarchy, block themes |
| 0:45 – 1:20 | 35 | Concept Block 2 | Plugins and vetting, WP-CLI, block editor and patterns, WordPress vs frameworks |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 14 – WP-CLI install, plugins and a block-built home page |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 14.1 Why a CMS?

A content management system separates **content** from **presentation** and gives non-programmers an editor, user roles, media handling, revisions and an extension ecosystem. WordPress (GPL-2.0-or-later) is written in PHP on MariaDB/MySQL and runs a very large share of public websites, so WordPress skills are directly employable.

### 14.2 Architecture

| Component | Location / table | Role |
|---|---|---|
| Core | `wp-admin/`, `wp-includes/` | Never edited; replaced on update |
| Content & settings | `wp_posts`, `wp_postmeta`, `wp_options`, `wp_users`, `wp_terms` | All site data in the database |
| Themes | `wp-content/themes/` | Appearance; **block themes** use HTML templates + `theme.json` |
| Plugins | `wp-content/plugins/` | Features added through hooks |
| Uploads | `wp-content/uploads/` | Media; the only folder that must be writable in production |

| Component | Location / table | Role |
|---|---|---|
| Config | `wp-config.php` | DB credentials, salts, constants (keep outside web root if possible) |

Extensibility works through **hooks**: *actions* run code at a moment (`add_action("init", …)`), *filters* modify data (`add_filter("the_title", …)`). The **template hierarchy** decides which template renders a URL (`single-project.php` → `single.php` → `singular.php` → `index.php`; block themes use `templates/*.html`). The REST API (`/wp-json/`) exposes content as JSON, enabling headless use with Angular or React.

## 14.3 Block Editor, Patterns and Full-Site Editing

- Content is stored as HTML wrapped in block comments (`<!-- wp:paragraph -->`), so it stays readable and versionable.
- **Patterns** are reusable block layouts; **template parts** (header, footer) and **Styles** (colours, typography) are edited in the **Site Editor**.
- `theme.json` defines the design tokens (palette, font sizes, spacing) that appear as controls in the editor.

## 14.4 Choosing and Vetting Plugins

| Check | Good sign |
|---|---|
| Active installations | Large, stable user base |
| Last updated / tested up to | Updated in the last 6 months, tested with the current WordPress and PHP |
| Support forum | Developer answers issues |
| Licence / source | GPL-compatible and open source |
| Footprint | Does not add dozens of database queries (measure with Query Monitor) |
| Necessity | Could a block, pattern or a few lines of code replace it? |

## 14.5 WordPress vs a PHP Framework (CLO_3 Analysis)

| Requirement | Best fit | Reason |
|---|---|---|
| Blog / marketing site edited by non-developers | WordPress | Editor, themes and plugins ready-made |
| Custom business rules, complex workflows | CodeIgniter 4 / Laravel | Full control, testable code |
| Mobile app back-end API | Laravel / CodeIgniter 4 (or WP REST API for content) | Structured routing, auth, validation |
| Store with standard catalogue | WordPress + WooCommerce (GPL) | Common requirements already solved |

# Hands-on Lab 14: WordPress with WP-CLI, Plugins and a Home Page Built from Blocks

Install WordPress using WP-CLI on your LAMP stack (MariaDB + PHP-FPM + Apache), set the SELinux / permission model correctly, install and review plugins, and build a front page with the block editor.

## Free & Open-Source Tools

- WordPress (GPL-2.0+)
- WP-CLI (MIT)
- Plugins: Query Monitor (GPL), Contact Form 7 (GPL)
- MariaDB, Apache, PHP-FPM from earlier weeks

## Deliverables

- WordPress at `http://webeng.test/wp/` with pretty permalinks
- Two plugins active; a one-page review table (installs, last update, queries added)
- Home page created from `home.html` and edited by drag-and-drop in the editor, set as the static front page
- Screenshot of the Site Editor with one changed pattern

## Common Source Files (identical on both operating systems)

**home.html (block markup; save to /tmp/home.html so the web user can read it)**

```
<!-- wp:heading {"level":1} -->
<h1 class="wp-block-heading">Welcome to the WebEng Lab Blog</h1>
<!-- /wp:heading -->

<!-- wp:paragraph -->
<p>Notes, experiments and projects from Web Engineering at Hamdard University.</p>
<!-- /wp:paragraph -->

<!-- wp:buttons -->
<div class="wp-block-buttons"><!-- wp:button -->
<div class="wp-block-button"><a class="wp-block-button__link wp-element-button"
href="/wp/blog/">Read the blog</a></div>
<!-- /wp:button --></div>
<!-- /wp:buttons -->
```

**wp-db.sql**

```
# Database (run once, as database administrator)
CREATE DATABASE wp_site CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'ChangeMe_2026!';
GRANT ALL PRIVILEGES ON wp_site.* TO 'wp_user'@'localhost';
```

## Version A – Rocky Linux 10

**Step 1 – Install PHP extensions, MariaDB and the database.** Reuse the Week 9–10 packages; add WordPress extras.

```
sudo dnf -y install php-fpm php-mysqlnd php-gd php-intl php-mbstring php-xml php-opcache
mariadb-server
sudo systemctl enable --now php-fpm mariadb
sudo mariadb < wp-db.sql
```

**Step 2 – Install WP-CLI and define a helper.** WP-CLI must run as the web user so files get the right owner. The helper keeps commands short.

```
curl -sSLo /tmp/wp-cli.phar https://raw.githubusercontent.com/wp-cli/builds/gh-
pages/phar/wp-cli.phar
php /tmp/wp-cli.phar --info | head -n 4
sudo install -m 0755 /tmp/wp-cli.phar /usr/local/bin/wp
wpa() { sudo -u apache env WP_CLI_CACHE_DIR=/tmp/wp-cli-cache wp --path=/var/www/webeng/wp
"$@"; }
```

**Step 3 – Prepare the folder and SELinux.** Core files are read-only content; wp-content must be writable. Outbound connections are needed for plugin downloads.

```
sudo mkdir -p /var/www/webeng/wp && sudo chown apache:apache /var/www/webeng/wp
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/var/www/webeng/wp/wp-content(/.*)?"
sudo setsebool -P httpd_can_network_connect on
wpa core download && sudo restorecon -Rv /var/www/webeng/wp | tail -n 2
```

**Step 4 – Create the configuration and install.** The URL includes /wp because the site is inside the virtual host.

```
wpa config create --dbname=wp_site --dbuser=wp_user --dbpass='ChangeMe_2026!' --
dbhost=localhost
wpa core install --url=http://webeng.test/wp --title="WebEng Blog" --admin_user=siteadmin
\
  --admin_password='Str0ng!Pass-2026' --admin_email=admin@webeng.test --skip-email
wpa rewrite structure '/%postname%/' --hard
curl -sI http://webeng.test/wp/ | head -n 1
```

**Step 5 – Install plugins and the default block theme.** Query Monitor is a developer tool; Contact Form 7 adds forms.

```
wpa plugin install query-monitor contact-form-7 --activate
wpa theme install twentytwentyfive --activate || wpa theme activate twentytwentyfive
wpa plugin list && wpa theme list
```

**Step 6 – Create the home page from block markup.** Content comes from the file; then set it as the front page.

```
cp home.html /tmp/home.html && chmod 644 /tmp/home.html
ID=$(wpa post create /tmp/home.html --post_type=page --post_title='Home' --
post_status=publish --porcelain)
wpa option update show_on_front page && wpa option update page_on_front "$ID"
wpa post generate --count=5 --post_type=post
firefox http://webeng.test/wp/wp-admin/ &      # drag blocks, insert a pattern, open
Appearance → Editor
```

**Step 7 – Verify and troubleshoot.** A denial here usually means a label or boolean is missing.

```
curl -s http://webeng.test/wp/ | grep -io '<title>[^<]*'
curl -s http://webeng.test/wp/wp-json/ | jq -r .name
wpa core version && wpa db check | tail -n 2
sudo ausearch -m avc -ts recent | tail -n 3
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Install PHP extensions, MariaDB and the database.** Reuse the Week 9–10 packages; add WordPress extras.

```
sudo apt -y install php-fpm php-mysql php-gd php-intl php-mbstring php-xml php-curl
mariadb-server
PHPV=$(php -r 'echo PHP_MAJOR_VERSION.".".PHP_MINOR_VERSION;'); sudo systemctl enable --
now php${PHPV}-fpm mariadb
sudo mariadb < wp-db.sql
```

**Step 2 – Install WP-CLI and define a helper.** WP-CLI must run as the web user so files get the right owner. The helper keeps commands short.

```
curl -sSLo /tmp/wp-cli.phar https://raw.githubusercontent.com/wp-cli/builds/gh-
pages/phar/wp-cli.phar
php /tmp/wp-cli.phar --info | head -n 4
sudo install -m 0755 /tmp/wp-cli.phar /usr/local/bin/wp
wpa() { sudo -u www-data env WP_CLI_CACHE_DIR=/tmp/wp-cli-cache wp --
path=/var/www/webeng/wp "$@"; }
```

**Step 3 – Prepare the folder.** AppArmor does not restrict this layout; ownership is enough.

```
sudo mkdir -p /var/www/webeng/wp && sudo chown www-data:www-data /var/www/webeng/wp
wpa core download && ls /var/www/webeng/wp | head
```

**Step 4 – Create the configuration and install.** The URL includes /wp because the site is inside the virtual host.

```
wpa config create --dbname=wp_site --dbuser=wp_user --dbpass='ChangeMe_2026!' --
dbhost=localhost
wpa core install --url=http://webeng.test/wp --title="WebEng Blog" --admin_user=siteadmin
\
  --admin_password='Str0ng!Pass-2026' --admin_email=admin@webeng.test --skip-email
wpa rewrite structure '/%postname%/' --hard
curl -sI http://webeng.test/wp/ | head -n 1
```

**Step 5 – Install plugins and the default block theme.** Query Monitor is a developer tool; Contact Form 7 adds forms.

```
wpa plugin install query-monitor contact-form-7 --activate
wpa theme install twentytwentyfive --activate || wpa theme activate twentytwentyfive
wpa plugin list && wpa theme list
```

**Step 6 – Create the home page from block markup.** Content comes from the file; then set it as the front page.

```
cp home.html /tmp/home.html && chmod 644 /tmp/home.html
ID=$(wpa post create /tmp/home.html --post_type=page --post_title='Home' --
post_status=publish --porcelain)
wpa option update show_on_front page && wpa option update page_on_front "$ID"
wpa post generate --count=5 --post_type=post
firefox http://webeng.test/wp/wp-admin/ &      # drag blocks, insert a pattern, open
Appearance → Editor
```

**Step 7 – Verify and troubleshoot.** Check the logs when something fails.

```
curl -s http://webeng.test/wp/ | grep -io '<title>[^<]*'
curl -s http://webeng.test/wp/wp-json/ | jq -r .name
wpa core version && wpa db check | tail -n 2
sudo tail -n 5 /var/log/apache2/error.log
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `curl -sI http://webeng.test/wp/` | `HTTP/1.1 200 OK` |
| 2 | `wpa core version` | Current WordPress release |
| 3 | `wpa plugin list --status=active --field=name` | `query-monitor` and `contact-form-7` |
| 4 | Browser front page | Heading, paragraph and button from `home.html`; your drag-and-drop change visible |
| 5 | `curl .../wp-json/ \| jq -r .name` | “WebEng Blog” |

| # | Check | Expected Result |
|---|---|---|
| 6 | Query Monitor (admin bar) | Page generated with a recorded number of queries and time |

### Stretch Goals (Industry Practice)

- Create a Contact Form 7 form and send a test mail to Mailpit through an SMTP plugin pointing to 127.0.0.1:1025.
- Use the WordPress REST API from Angular (Week 6 project) to list posts.
- Compare page weight and query count with and without Contact Form 7 enabled.

## Review Questions

1. Which folders of a WordPress site must never be edited and which hold your own work?
2. Explain actions versus filters with one example of each.
3. List four criteria for accepting a plugin and one reason to avoid one.
4. How does a block theme differ from a classic theme?
5. For which of three requirements (brochure site, loan-approval workflow, mobile API) would you pick WordPress, and why?

# Week 15: Creating and Configuring a Blog: Themes, Custom Content, Security and Backups

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 29–30 | **CLO** | CLO_3 |
| **PLO** | PLO_4 | **Bloom's Level** | C4 (Analyzing) |
| **Reading** | Ch. 16 Content Management Systems (themes, customisation) | **Assessment** | — |

## Learning Outcomes

- Create and configure a blog: posts, pages, categories, tags, comments, media, permalinks and settings.
- Customise appearance safely with a child theme and `theme.json`.
- Extend WordPress with a small plugin that registers a custom post type and taxonomy.
- Harden the installation and automate backups with systemd timers.

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | Blog structure and configuration, SEO basics, comments, child themes, theme.json |
| 0:45 – 1:20 | 35 | Concept Block 2 | Custom post types and plugins, REST API, performance, security hardening, backups |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 15 – Child theme, custom post type, hardening and backup |
| 2:50 – 3:00 | 10 | Wrap-up | Demo of results, exit ticket, preview of next week |

## Lecture Notes

### 15.1 Blog Anatomy and Settings

| Element | Use |
|---|---|
| Posts | Time-ordered entries with author, categories and tags (appear in feeds) |
| Pages | Static content (About, Contact); hierarchical |
| Categories / tags | Category = broad topic (hierarchical); tag = specific keyword |
| Media library | Images and files; always add alt text |
| Comments | Moderation queue, "comment held for approval", disable on old posts |
| Settings | General (timezone Asia/Karachi), Reading (front page), Discussion, Permalinks (`/%postname%/`) |
| Roles | Administrator, Editor, Author, Contributor, Subscriber – give least privilege |

SEO and sharing basics: meaningful titles and headings, descriptive permalinks, excerpts, image alt text, fast pages, XML sitemap (built into core) and Open Graph metadata.

### 15.2 Child Themes and theme.json

Never edit a parent theme directly; an update would erase your work. A **child theme** inherits everything and overrides only what you change. In a block theme, the child's `theme.json` merges with the parent's, so you can replace palette, typography and spacing tokens without copying templates.

### 15.3 Custom Post Types, Taxonomies and Hooks

Content that is not a blog post (projects, events, courses) should be a **custom post type** registered by a **plugin**, not by the theme, so the content survives a theme change. `show_in_rest` makes it available in the block editor and the REST API.

### 15.4 Performance

- Enable PHP **OPcache**; use a page cache for anonymous visitors; serve images as WebP/AVIF with `width/height`.
- Keep plugins few; measure with Query Monitor and browser DevTools; target good Core Web Vitals (LCP, INP, CLS).

### 15.5 Security Hardening Checklist

| Control | How |
|---|---|
| Updates | Core, plugins and themes updated promptly (`wp core update`, `wp plugin update --all`) |
| Least privilege | Admin account only for administration; editors write content |
| File editing off | `DISALLOW_FILE_EDIT` stops code edits from the dashboard |
| File permissions | Directories 755, files 644, `wp-config.php` 640 |
| Attack surface | Remove unused plugins/themes; block `xmlrpc.php` if unused |
| Authentication | Strong unique passwords, 2FA plugin, login rate limiting |
| Transport | HTTPS everywhere (Let's Encrypt in production) |
| Backups | Database + files, off-machine copy, tested restore |

> **Operations:** A backup that has never been restored is only a hope. Schedule a restore test.

## Hands-on Lab 15: Configure the Blog, Child Theme, "Projects" Plugin, Hardening and Automatic Backup

Turn the Week 14 site into a configured blog, add a branded child theme, a custom post type plugin, apply a hardening checklist and schedule verified nightly backups with a systemd timer.

### Free & Open-Source Tools

- WP-CLI
- systemd timers, `runuser`, `tar`, `sha256sum`
- Query Monitor
- jq, curl

### Deliverables

- Blog configured (timezone, permalinks, comment moderation, categories, 3 posts)
- Active child theme `twentytwentyfive-child` with custom palette

- Plugin `webeng-projects` and 2 “Project” entries visible via the REST API
- Hardening applied and a timer that produces a verified backup

## Common Source Files (identical on both operating systems)

**wp-content/themes/twentytwentyfive-child/style.css**

```
/*
Theme Name: Twenty Twenty-Five Child
Template: twentytwentyfive
Version: 1.0.0
Description: WebEng Lab child theme
*/
```

**wp-content/themes/twentytwentyfive-child/theme.json**

```
{
  "$schema": "https://schemas.wp.org/trunk/theme.json",
  "version": 3,
  "settings": {
    "color": { "palette": [
      { "slug": "brand", "name": "Brand", "color": "#0f766e" },
      { "slug": "ink",   "name": "Ink",   "color": "#0f172a" }
    ] }
  },
  "styles": {
    "elements": { "link": { "color": { "text": "var(--wp--preset--color--brand)" } } }
  }
}
```

**wp-content/plugins/webeng-projects/webeng-projects.php**

```
<?php
/**
 * Plugin Name: WebEng Projects
 * Description: Custom post type and taxonomy for student projects.
 * Version: 1.0.0
 * License: GPL-2.0-or-later
 */
defined('ABSPATH') || exit;

function webeng_register_projects(): void {
    register_post_type('project', [
        'labels' => ['name' => 'Projects', 'singular_name' => 'Project'],
        'public' => true, 'has_archive' => true, 'show_in_rest' => true,
        'menu_icon' => 'dashicons-portfolio',
        'supports' => ['title', 'editor', 'thumbnail', 'excerpt'],
        'rewrite' => ['slug' => 'projects'],
    ]);
    register_taxonomy('skill', 'project', ['label' => 'Skills', 'hierarchical' => false,
'show_in_rest' => true]);
}
add_action('init', 'webeng_register_projects');
register_activation_hook(__FILE__, function (): void { webeng_register_projects();
flush_rewrite_rules(); });
add_shortcode('project_count', fn(): string => esc_html((string)
wp_count_posts('project')->publish));
```

**/usr/local/sbin/wp-backup.sh (chmod 750, owner root)**

```
#!/usr/bin/env bash
set -euo pipefail
WEBUSER="${WEBUSER:-apache}"; WP=/var/www/webeng/wp; OUT=/var/backups/webeng; TS=$(date
+%Y%m%d-%H%M)
runuser -u "$WEBUSER" -- env WP_CLI_CACHE_DIR=/tmp/wp-cli-cache wp --path="$WP" db export
"/tmp/db-$TS.sql"
```

```
mv "/tmp/db-$TS.sql" "$OUT/"
tar czf "$OUT/wp-files-$TS.tgz" -C /var/www/webeng wp
find "$OUT" -type f -mtime +14 -delete
(cd "$OUT" && sha256sum "db-$TS.sql" "wp-files-$TS.tgz" > "SHA256-$TS.txt")
echo "backup $TS complete"
```

**/etc/systemd/system/wp-backup.service (Kubuntu: set WEBUSER=www-data)**

```
[Unit]
Description=WordPress backup

[Service]
Type=oneshot
Environment=WEBUSER=apache
ExecStart=/usr/local/sbin/wp-backup.sh
```

**/etc/systemd/system/wp-backup.timer**

```
[Unit]
Description=Nightly WordPress backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

## Version A – Rocky Linux 10

**Step 1 – Define the helper and configure the blog.** Settings are changed from the command line so they are repeatable.

```
wpa() { sudo -u apache env WP_CLI_CACHE_DIR=/tmp/wp-cli-cache wp --path=/var/www/webeng/wp
"$@"; }
wpa option update timezone_string Asia/Karachi
wpa option update blogdescription "Web Engineering lab notes"
wpa option update comment_moderation 1 && wpa option update default_comment_status closed
wpa term create category "Lab Notes" --porcelain
for t in "Linux" "PHP" "WordPress"; do wpa post create --post_title="$t basics" --
post_content="<!-- wp:paragraph --><p>Notes on $t.</p><!-- /wp:paragraph -->" --
post_status=publish --post_category=$(wpa term list category --name="Lab Notes" --
field=term_id) --tags_input="$t" --porcelain; done
```

**Step 2 – Create and activate the child theme and plugin.** Files must belong to the web user and carry the right SELinux type (inherited from wp-content).

```
T=/var/www/webeng/wp/wp-content
sudo -u apache mkdir -p $T/themes/twentytwentyfive-child $T/plugins/webeng-projects
sudo -u apache tee $T/themes/twentytwentyfive-child/style.css < style.css >/dev/null
sudo -u apache tee $T/themes/twentytwentyfive-child/theme.json < theme.json >/dev/null
sudo -u apache tee $T/plugins/webeng-projects/webeng-projects.php < webeng-
projects.php >/dev/null
wpa theme activate twentytwentyfive-child && wpa plugin activate webeng-projects
wpa post create --post_type=project --post_title="Capstone Shop" --post_status=publish
wpa post create --post_type=project --post_title="CSV Explorer" --post_status=publish
curl -s http://webeng.test/wp/wp-json/wp/v2/project | jq 'map(.title.rendered)'
```

**Step 3 – Apply hardening.** Constants, permissions and a block on XML-RPC at the web-server level.

```
wpa config set DISALLOW_FILE_EDIT true --raw && wpa config set WP_DEBUG false --raw
sudo find /var/www/webeng/wp -type d -exec chmod 755 {} + && sudo find /var/www/webeng/wp
-type f -exec chmod 644 {} +
```

```
sudo chmod 640 /var/www/webeng/wp/wp-config.php
sudo tee /etc/httpd/conf.d/wp-hardening.conf >/dev/null <<'EOF'
<Directory /var/www/webeng/wp>
    <Files xmlrpc.php>
        Require all denied
    </Files>
    Header always set X-Content-Type-Options "nosniff"
</Directory>
EOF
sudo apachectl configtest && sudo systemctl reload httpd
curl -s -o /dev/null -w "xmlrpc: %{http_code}\n" http://webeng.test/wp/xmlrpc.php
```

**Step 4 – Install the backup script and timer.** The script runs as root from systemd and drops privileges for WP-CLI.

```
sudo install -d -m 0750 /var/backups/webeng
kate wp-backup.sh wp-backup.service wp-backup.timer &
sudo install -m 0750 wp-backup.sh /usr/local/sbin/wp-backup.sh
sudo install -m 0644 wp-backup.service wp-backup.timer /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now wp-backup.timer
sudo systemctl start wp-backup.service && sudo journalctl -u wp-backup.service -n 5 --no-
pager
systemctl list-timers wp-backup.timer
```

**Step 5 – Verify the backup.** Integrity first; then check that the archive and dump contain what you expect.

```
cd /var/backups/webeng && sudo sha256sum -c SHA256-*.txt
sudo tar tzf wp-files-*.tgz | grep -c . && sudo grep -c 'CREATE TABLE' db-*.sql
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Define the helper and configure the blog.** Settings are changed from the command line so they are repeatable.

```
wpa() { sudo -u www-data env WP_CLI_CACHE_DIR=/tmp/wp-cli-cache wp --
path=/var/www/webeng/wp "$@"; }
wpa option update timezone_string Asia/Karachi
wpa option update blogdescription "Web Engineering lab notes"
wpa option update comment_moderation 1 && wpa option update default_comment_status closed
wpa term create category "Lab Notes" --porcelain
for t in "Linux" "PHP" "WordPress"; do wpa post create --post_title="$t basics" --
post_content="<!-- wp:paragraph --><p>Notes on $t.</p><!-- /wp:paragraph -->" --
post_status=publish --post_category=$(wpa term list category --name="Lab Notes" --
field=term_id) --tags_input="$t" --porcelain; done
```

**Step 2 – Create and activate the child theme and plugin.** Files must belong to the web user.

```
T=/var/www/webeng/wp/wp-content
sudo -u www-data mkdir -p $T/themes/twentytwentyfive-child $T/plugins/webeng-projects
sudo -u www-data tee $T/themes/twentytwentyfive-child/style.css < style.css >/dev/null
sudo -u www-data tee $T/themes/twentytwentyfive-child/theme.json < theme.json >/dev/null
sudo -u www-data tee $T/plugins/webeng-projects/webeng-projects.php < webeng-
projects.php >/dev/null
wpa theme activate twentytwentyfive-child && wpa plugin activate webeng-projects
wpa post create --post_type=project --post_title="Capstone Shop" --post_status=publish
wpa post create --post_type=project --post_title="CSV Explorer" --post_status=publish
curl -s http://webeng.test/wp/wp-json/wp/v2/project | jq 'map(.title.rendered)'
```

**Step 3 – Apply hardening.** Constants, permissions and a block on XML-RPC at the web-server level.

```
wpa config set DISALLOW_FILE_EDIT true --raw && wpa config set WP_DEBUG false --raw
```

```
sudo find /var/www/webeng/wp -type d -exec chmod 755 {} + && sudo find /var/www/webeng/wp
-type f -exec chmod 644 {} +
sudo chmod 640 /var/www/webeng/wp/wp-config.php
sudo tee /etc/apache2/conf-available/wp-hardening.conf >/dev/null <<'EOF'
<Directory /var/www/webeng/wp>
    <Files xmlrpc.php>
        Require all denied
    </Files>
    Header always set X-Content-Type-Options "nosniff"
</Directory>
EOF
sudo a2enconf wp-hardening && sudo apache2ctl configtest && sudo systemctl reload apache2
curl -s -o /dev/null -w "xmlrpc: %{http_code}\n" http://webeng.test/wp/xmlrpc.php
```

**Step 4 – Install the backup script and timer.** The script runs as root from systemd and drops privileges for WP-CLI. Set WEBUSER to www-data in the service file.

```
sudo install -d -m 0750 /var/backups/webeng
kate wp-backup.sh wp-backup.service wp-backup.timer &
sed -i 's/WEBUSER=apache/WEBUSER=www-data/' wp-backup.service
sudo install -m 0750 wp-backup.sh /usr/local/sbin/wp-backup.sh
sudo install -m 0644 wp-backup.service wp-backup.timer /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now wp-backup.timer
sudo systemctl start wp-backup.service && sudo journalctl -u wp-backup.service -n 5 --no-
pager
systemctl list-timers wp-backup.timer
```

**Step 5 – Verify the backup.** Integrity first; then check that the archive and dump contain what you expect.

```
cd /var/backups/webeng && sudo sha256sum -c SHA256-*.txt
sudo tar tzf wp-files-*.tgz | grep -c . && sudo grep -c 'CREATE TABLE' db-*.sql
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | Settings in the admin | Timezone Asia/Karachi, permalinks `/%postname%/`, comments held for moderation |
| 2 | `wpa theme list --status=active --field=name` | `twentytwentyfive-child` |
| 3 | REST: `/wp-json/wp/v2/project` | Two projects returned as JSON |
| 4 | `curl .../xmlrpc.php` | 403 Forbidden |
| 5 | Dashboard theme/plugin editor | Menu items absent (`DISALLOW_FILE_EDIT`) |
| 6 | `systemctl list-timers wp-backup.timer` | Next run 02:30; last manual run produced `SHA256-*.txt` with OK on check |

## Stretch Goals (Industry Practice)

- Restore the backup into a second database and site path and compare post counts.
- Install `fail2ban` and add a jail for repeated `wp-login.php` failures.
- Add `rsync` to push the backup to another machine over SSH keys.

## Review Questions

1. Why create a child theme instead of editing Twenty Twenty-Five directly?
2. Why should a custom post type live in a plugin and not in the theme?

3. Name four items on a WordPress hardening checklist and the command or setting for each.
4. What does `Persistent=true` do in a systemd timer?
5. How would you test that a backup is restorable?

# Week 16: Capstone Deployment with Containers, Course Revision and Final-Exam Preparation

| | | | |
|---|---|---|---|
| **Syllabus Lectures** | Lec 31–32 | **CLO** | CLO_1, CLO_2, CLO_3 |
| **PLO** | PLO_2, PLO_4, PLO_5 | **Bloom's Level** | C2–C4 |
| **Reading** | Selected chapters based on course syllabus | **Assessment** | Final examination follows (Subj 10 + Obj 15 + Subj 25) |

## Learning Outcomes

- Package and run a multi-service web stack with rootless Podman and a compose file.
- Apply a Git-based delivery workflow and automatic checks.
- Review the OWASP Top 10 against the techniques used in the course.
- Revise CLO_1 (classification, MVC vs tiers), CLO_2 (development) and CLO_3 (analysis with WordPress / CodeIgniter).

## 3-Hour Lecture Plan

| Time (h:mm) | Min | Segment | Activity |
|---|---|---|---|
| 0:00 – 0:10 | 10 | Recap & attendance | Recap of last week, attendance, follow-up on assignment / quiz |
| 0:10 – 0:45 | 35 | Concept Block 1 | Containers, Podman, compose, Quadlet, Git workflow and CI concepts |
| 0:45 – 1:20 | 35 | Concept Block 2 | OWASP Top 10 mapping, CLO revision and mock final exam |
| 1:20 – 1:30 | 10 | Break | Rest; students start VM / packages for the lab |
| 1:30 – 2:50 | 80 | Hands-on Lab | Lab 16 – Containerised capstone stack |
| 2:50 – 3:00 | 10 | Wrap-up | Course feedback, final-exam logistics, CLO self-assessment |

## Lecture Notes

### 16.1 The Full Stack in One Picture

| Tier | Technology used in this course | Weeks |
|---|---|---|
| Presentation (browser) | HTML5, CSS3, JavaScript, Bootstrap, Angular | 1–7 |
| Application | Apache + PHP-FPM, Node.js, CodeIgniter 4 / Laravel, WordPress | 6, 9–15 |
| Data | MariaDB, files, sessions | 10–13 |
| Platform | Linux (Rocky / Kubuntu), TLS, firewall, SELinux / AppArmor, systemd, Podman | 1, 8–16 |

### 16.2 Containers and Podman

A **container** is a normal Linux process isolated with namespaces and cgroups; it is created from an **image** (layers described by a `Containerfile`). **Podman** is daemonless and runs containers **rootless**, so a breakout gives an attacker only your user's rights. Data that must survive container replacement lives in **volumes**. `podman-compose` reads a Compose file and creates the services on a private network where each service is reachable by its name (the database host is simply `db`).

| Concept | Command / file | Purpose |
|---|---|---|
| Image build | `podman build -t web ./web` | Create an image from a Containerfile |
| Run / list | `podman run -d …`, `podman ps` | Start and inspect containers |
| Compose | `podman-compose up -d --build` | Start the multi-service stack |
| Volumes | `podman volume ls` | Persistent data |
| Quadlet | `~/.config/containers/systemd/*.container` | Run containers as systemd services at boot |
| SELinux label | `:Z` on bind mounts | Relabel a host folder for one container (Rocky) |

## 16.3 Delivery Workflow

- **Git:** small commits with clear messages, feature branches, pull / merge requests, tags for releases (`v1.0.0`).
- **Automatic checks:** `php -l`, PHPStan, ESLint, html-validate, unit tests, run in a pre-commit hook locally and in a CI service (Forgejo Actions, GitLab CE runner or GitHub Actions) on every push.
- **Configuration:** secrets in environment variables or files outside Git (12-factor); never commit passwords.
- **Observability:** application logs to stdout/files, health endpoint, `podman logs`, `journalctl`.

## 16.4 OWASP Top 10 Mapped to the Course

| Risk | Where you defended against it |
|---|---|
| Broken access control | Role checks, `require_login()`, files outside web root (Weeks 10–11) |
| Cryptographic failures | HTTPS (Week 1), `password_hash`, random tokens (Weeks 11, 13) |
| Injection (SQL, XSS) | PDO prepared statements, output escaping `e()` (Weeks 9–12) |
| Insecure design | Transactions, least-privilege DB user, CAPTCHA (Weeks 10–11) |
| Security misconfiguration | SELinux, firewall, `display_errors=0`, hardening config (Weeks 1, 9, 15) |
| Vulnerable components | Composer / npm / WordPress updates, plugin vetting (Weeks 14–15) |
| Authentication failures | Session regeneration, rate limiting, generic messages (Week 11) |
| Integrity & logging failures | CSRF tokens, upload validation, error logs, backups with hashes (Weeks 11–15) |

## 16.5 Revision Map by CLO

| CLO | Marks in course | Weeks | Final-exam style |
|---|---|---|---|
| CLO_1 Classify web components; MVC vs 2/3-tier | 28 | 1–3, 13 | Subjective 10 marks: define, compare, draw architecture |
| CLO_2 Develop with HTML, CSS, JS, PHP, DB | 35 | 2–12 | Objective 15 marks: MCQ and code reading |
| CLO_3 Analyze / solve with WordPress, CodeIgniter | 37 | 10–15 | Subjective 25 marks: scenarios, design and justification |

## 16.6 Mock Final Exam – Sample Questions

| # | CLO | Question | Marking guide |
|---|---|---|---|
| 1 | CLO_1 | Draw and explain a three-tier architecture for an online admission portal and show where MVC fits. | Tiers (3), technologies (2), MVC mapping (3), one trade-off (2) |

| # | CLO | Question | Marking guide |
|---|---|---|---|
| 2 | CLO_2 | Write PHP that accepts a search term, queries products by prefix with PDO and prints escaped results with pagination. | Prepared statement (3), LIKE escaping (1), escaping (2), pagination (2) |
| 3 | CLO_2 | Predict output of a JavaScript snippet using closures, `let` in loops and `await`. | Trace table with reasons |
| 4 | CLO_3 | A client wants a news site edited by journalists plus a custom subscription workflow. Compare WordPress, CodeIgniter 4 and a hybrid; recommend one. | Criteria (4), comparison (6), decision and risks (5) |
| 5 | CLO_3 | Analyse a CodeIgniter controller with no validation and a raw SQL string; list defects and fix them. | SQL injection, CSRF, validation, error handling (5 each) |

## Hands-on Lab 16: Containerised Capstone: Apache/PHP + MariaDB with Podman Compose and a Quadlet Service

Build and run the shop stack as containers, prove persistence and recovery, run Mailpit as a boot-time systemd (Quadlet) service, and package the project in Git with a pre-commit check.

### Free & Open-Source Tools

- Podman + podman-compose (Apache-2.0 / GPL)
- Official `php:apache` and `mariadb:lts` images
- Mailpit (MIT)
- Git + shell hooks

### Deliverables

- Git repository `capstone` with `compose.yaml`, `web/Containerfile`, `app/`, `schema.sql`, `.gitignore`, tag `v1.0.0`
- Stack reachable at `http://127.0.0.1:8090/` showing “DB OK” and the product count
- Evidence of persistence (`podman volume ls`, data survives `down` / `up`)
- Mailpit started as a user systemd service

### Common Source Files (identical on both operating systems)

**`compose.yaml`**

```
services:
  db:
    image: docker.io/library/mariadb:lts
    environment:
      MARIADB_DATABASE: shop
      MARIADB_USER: shop_app
      MARIADB_PASSWORD: "ChangeMe_2026!"
      MARIADB_RANDOM_ROOT_PASSWORD: "1"
    volumes:
      - dbdata:/var/lib/mysql
      - ./schema.sql:/docker-entrypoint-initdb.d/10-schema.sql:${MOUNT_OPTS}
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 10
  web:
    build: ./web
    ports:
      - "127.0.0.1:8090:80"
```

```
    environment:
      DB_HOST: db
      DB_NAME: shop
      DB_USER: shop_app
      DB_PASS: "ChangeMe_2026!"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./app:/var/www/html:${MOUNT_OPTS}
volumes:
  dbdata:
```

**web/Containerfile**

```
FROM docker.io/library/php:8-apache
RUN docker-php-ext-install pdo_mysql \
 && a2enmod rewrite headers \
 && { echo 'expose_php=0'; echo 'display_errors=0'; echo 'log_errors=1'; } >
/usr/local/etc/php/conf.d/zz-app.ini
```

**app/index.php**

```
<?php
declare(strict_types=1);
try {
    $pdo = new PDO(sprintf('mysql:host=%s;dbname=%s;charset=utf8mb4', getenv('DB_HOST'),
getenv('DB_NAME')),
                   (string)getenv('DB_USER'), (string)getenv('DB_PASS'),
[PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
    $n = (int)$pdo->query('SELECT COUNT(*) FROM products')->fetchColumn();
    $status = "DB OK – $n products";
} catch (Throwable $t) {
    error_log((string)$t); http_response_code(503); $status = 'DB unavailable';
}
?>
<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"><title>Capstone</title></head>
<body><h1>Capstone stack</h1><p><?= htmlspecialchars($status) ?></p><p>PHP <?=
PHP_VERSION ?></p></body></html>
```

**~/.config/containers/systemd/mailpit.container**

```
[Unit]
Description=Mailpit mail catcher

[Container]
Image=docker.io/axllent/mailpit:latest
PublishPort=127.0.0.1:8025:8025
PublishPort=127.0.0.1:1025:1025

[Install]
WantedBy=default.target
```

**.git/hooks/pre-commit (chmod +x)**

```
#!/usr/bin/env bash
set -e
for f in $(git diff --cached --name-only --diff-filter=ACM | grep '\.php$' || true); do
php -l "$f" > /dev/null; done
echo "pre-commit OK"
```

## Version A – Rocky Linux 10

**Step 1 – Install tools and create the repository.** EPEL provides podman-compose; the fallback installs it with pip.

```
sudo dnf -y install podman git epel-release
sudo dnf -y install podman-compose || { sudo dnf -y install python3-pip && pip install --
user podman-compose; }
mkdir -p ~/webeng/capstone/{web,app} && cd ~/webeng/capstone && git init -q -b main
cp /var/www/webeng/week10/schema.sql . && kate compose.yaml web/Containerfile
app/index.php &
printf '.env\n*.log\n' > .gitignore
```

**Step 2 – Set the SELinux-aware mount option.** On Rocky the `Z` option relabels the mounted host folders so that the container may read them.

```
echo 'MOUNT_OPTS=ro,Z' > .env
podman info --format 'rootless={{.Host.Security.Rootless}}
selinux={{.Host.Security.SELinuxEnabled}}'
```

**Step 3 – Build and start the stack.** First start downloads images and initialises the database from schema.sql.

```
podman-compose up -d --build
podman-compose ps
sleep 20 && curl -s http://127.0.0.1:8090/ | grep -E 'DB|PHP'
```

**Step 4 – Prove persistence and recovery.** Data lives in the named volume; a stopped database yields HTTP 503 and recovers.

```
podman volume ls
podman-compose stop db; curl -s -o /dev/null -w "db down: %{http_code}\n"
http://127.0.0.1:8090/
podman-compose start db; sleep 15; curl -s -o /dev/null -w "db up: %{http_code}\n"
http://127.0.0.1:8090/
podman-compose down && podman-compose up -d && sleep 20 && curl -s http://127.0.0.1:8090/
| grep products
```

**Step 5 – Run Mailpit as a boot-time service (Quadlet).** Remove the Week 13 container first so the ports are free. Linger lets the user service start at boot.

```
podman rm -f mailpit 2>/dev/null; mkdir -p ~/.config/containers/systemd && kate
~/.config/containers/systemd/mailpit.container
systemctl --user daemon-reload && systemctl --user start mailpit.service
sudo loginctl enable-linger $USER && systemctl --user status mailpit.service --no-pager |
head -n 5
curl -s -o /dev/null -w "mailpit UI: %{http_code}\n" http://127.0.0.1:8025/
```

**Step 6 – Commit with the hook and tag the release.** The hook blocks commits that contain PHP syntax errors.

```
install -m 0755 /dev/stdin .git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
set -e
for f in $(git diff --cached --name-only --diff-filter=ACM | grep '\.php$' || true); do
php -l "$f" > /dev/null; done
echo "pre-commit OK"
EOF
git add -A && git commit -qm "Capstone: containerised shop stack" && git tag v1.0.0 && git
log --oneline --decorate | head -n 3
```

## Version B – Kubuntu 26.04 LTS

**Step 1 – Install tools and create the repository.** Both podman and podman-compose are in the Ubuntu archive.

```
sudo apt -y install podman podman-compose git
mkdir -p ~/webeng/capstone/{web,app} && cd ~/webeng/capstone && git init -q -b main
cp /var/www/webeng/week10/schema.sql . && kate compose.yaml web/Containerfile
app/index.php &
printf '.env\n*.log\n' > .gitignore
```

**Step 2 – Set the mount option.** Without SELinux no relabelling is needed; the mounts are read-only.

```
echo 'MOUNT_OPTS=ro' > .env
podman info --format 'rootless={{.Host.Security.Rootless}}'
```

**Step 3 – Build and start the stack.** First start downloads images and initialises the database from schema.sql.

```
podman-compose up -d --build
podman-compose ps
sleep 20 && curl -s http://127.0.0.1:8090/ | grep -E 'DB|PHP'
```

**Step 4 – Prove persistence and recovery.** Data lives in the named volume; a stopped database yields HTTP 503 and recovers.

```
podman volume ls
podman-compose stop db; curl -s -o /dev/null -w "db down: %{http_code}\n"
http://127.0.0.1:8090/
podman-compose start db; sleep 15; curl -s -o /dev/null -w "db up: %{http_code}\n"
http://127.0.0.1:8090/
podman-compose down && podman-compose up -d && sleep 20 && curl -s http://127.0.0.1:8090/
| grep products
```

**Step 5 – Run Mailpit as a boot-time service (Quadlet).** Remove the Week 13 container first so the ports are free. Linger lets the user service start at boot.

```
podman rm -f mailpit 2>/dev/null; mkdir -p ~/.config/containers/systemd && kate
~/.config/containers/systemd/mailpit.container
systemctl --user daemon-reload && systemctl --user start mailpit.service
sudo loginctl enable-linger $USER && systemctl --user status mailpit.service --no-pager |
head -n 5
curl -s -o /dev/null -w "mailpit UI: %{http_code}\n" http://127.0.0.1:8025/
```

**Step 6 – Commit with the hook and tag the release.** The hook blocks commits that contain PHP syntax errors.

```
install -m 0755 /dev/stdin .git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
set -e
for f in $(git diff --cached --name-only --diff-filter=ACM | grep '\.php$' || true); do
php -l "$f" > /dev/null; done
echo "pre-commit OK"
EOF
git add -A && git commit -qm "Capstone: containerised shop stack" && git tag v1.0.0 && git
log --oneline --decorate | head -n 3
```

## Verification Checklist

| # | Check | Expected Result |
|---|---|---|
| 1 | `curl http://127.0.0.1:8090/` | "DB OK – 8 products" and the PHP version |
| 2 | `podman info` format line | `rootless=true` (Rocky also `selinux=true`) |
| 3 | Stop `db` and request the page | HTTP 503 with "DB unavailable"; recovers after `start db` |
| 4 | `down` then `up` | Product count unchanged (named volume kept the data) |
| 5 | `systemctl --user status mailpit.service` | `active (running)` |
| 6 | Commit with a PHP syntax error | Hook prints the parse error and aborts the commit |
| 7 | `git tag` | `v1.0.0` present |

## Stretch Goals (Industry Practice)

- Add a `phpmyadmin`-free admin check: a read-only `/health.php` returning JSON for monitoring.
- Add a CI pipeline file (Forgejo / GitLab CE / GitHub Actions) that runs `php -l` and PHPStan.
- Scan the image for known vulnerabilities with Trivy (Apache-2.0).

## Course Wrap-up Checklist

- Submit the capstone repository link and the README listing commands for **your** operating system.
- Complete the CLO self-assessment (1–5 scale) for CLO_1, CLO_2 and CLO_3 and note one weakness to revise.
- Bring the final-exam permitted items (calculator, ID). Cell phones in silent mode; at least 75% attendance is required.

# Review Questions

1. Explain the difference between an image, a container and a volume.
2. Why bind published ports to `127.0.0.1` in development?
3. Name five OWASP Top 10 risks and the countermeasure you implemented for each.
4. Compare 2-tier, 3-tier and MVC in a short paragraph.
5. Choose WordPress, CodeIgniter 4 or Laravel for a university event-registration system and justify with three criteria.

# Appendix A: Rocky 10 vs Kubuntu 26.04 Cheat Sheet

| Task | Rocky Linux 10 | Kubuntu 26.04 LTS |
|---|---|---|
| Install package | `sudo dnf install pkg` | `sudo apt install pkg` |
| Search package | `dnf search term` | `apt search term` |
| Which package owns file | `dnf provides /path` | `dpkg -S /path` |
| Start / enable web server | `systemctl enable --now httpd` | `systemctl enable --now apache2` |
| Reload web server | `sudo systemctl reload httpd` | `sudo systemctl reload apache2` |
| Test web config | `sudo apachectl configtest` | `sudo apache2ctl configtest` |
| Site config location | `/etc/httpd/conf.d/*.conf` | `/etc/apache2/sites-available/` + `a2ensite` |
| Error log | `/var/log/httpd/error_log` | `/var/log/apache2/error.log` |
| Web user | `apache` | `www-data` |
| PHP-FPM service | `php-fpm` | `php8.5-fpm` (use `php${PHPV}-fpm`) |
| PHP-FPM socket | `/run/php-fpm/www.sock` | `/run/php/php8.5-fpm.sock` |
| php.ini | `/etc/php.ini`, `/etc/php.d/` | `/etc/php/8.5/fpm/php.ini` |
| MariaDB service / log | `mariadb` / `/var/log/mariadb/` | `mariadb` / `journalctl -u mariadb` |
| Open firewall port | `firewall-cmd --permanent --add-port=8080/tcp` | `ufw allow 8080/tcp` |
| Security module | SELinux: `getenforce`, `ls -Z`, `restorecon`, `semanage`, `setsebool`, `ausearch -m avc` | AppArmor: `aa-status`, profiles in `/etc/apparmor.d/` |
| Container engine | `podman` (in AppStream) | `podman` (apt) |

## SELinux Troubleshooting Routine (Rocky)

```
getenforce                                    # Enforcing / Permissive
sudo ausearch -m avc -ts recent               # recent denials
sudo journalctl -t setroubleshoot --since "10 min ago"
ls -Z /var/www/webeng/weekNN                  # check file types
sudo restorecon -Rv /var/www/webeng           # reset to policy default
getsebool -a | grep httpd                     # list web booleans
```

> **Industry practice:** Never fix a problem by running `setenforce 0` permanently. Find the denial and apply the correct label or boolean. This is the habit employers expect.

# Appendix B: Lab Marking Rubric (10 points per lab)

| Criterion | Points | Full-mark description |
|---|---|---|
| Functionality | 4 | All verification-checklist items pass on the student's own OS |
| Code quality | 2 | Linter / static analysis clean, meaningful names, comments where needed |
| Security & configuration | 2 | Least privilege, escaped output, prepared statements, correct labels / firewall |
| Documentation & Git | 1 | README with commands run, committed with clear messages |
| Viva / explanation | 1 | Student explains one design decision and one failure they fixed |

# Appendix C: References and Free / Open-Source Tool Index

| Resource | Use in this course | Address |
|---|---|---|
| MDN Web Docs | HTML, CSS, JavaScript, HTTP reference | developer.mozilla.org |
| PHP Manual | Language and extension reference | php.net/manual |
| MariaDB Knowledge Base | SQL reference and administration | mariadb.com/kb |
| Node.js Docs | Core APIs, test runner | nodejs.org/docs |
| Angular Docs | Signals, components, HttpClient | angular.dev |
| Bootstrap Docs / Icons | Components, utilities, icons | getbootstrap.com, icons.getbootstrap.com |
| CodeIgniter 4 User Guide | MVC framework | codeigniter.com/user_guide |
| WordPress Developer Resources | Themes, plugins, WP-CLI | developer.wordpress.org |
| Podman Docs | Rootless containers, Quadlet | docs.podman.io |
| OWASP Top 10 / Cheat Sheets | Secure coding checklist | owasp.org |
| Rocky Linux / Kubuntu Docs | OS administration | docs.rockylinux.org, kubuntu.org |

Course texts: Sebesta, Programming the World Wide Web; Ullman, PHP and MySQL for Dynamic Web Sites; Bayross, HTML, DHTML, JavaScript, Perl CGI; Scobey & Lingras, Web Programming and Internet Technologies (as listed in the course outline).